



UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO

DIPARTIMENTO DI INFORMATICA

CORSO DI LAUREA IN INFORMATICA E COMUNICAZIONI DIGITALI

TESI DI LAUREA
IN
SVILUPPO DI VIDEOGIOCHI

Integrazione di mission graph,
grammatiche generative e algoritmi
evolutivi per la creazione
procedurale di livelli videoludici

RELATORI:

Prof. Pierpaolo Basile

LAUREANDO:

Danilo Ivone

ANNO ACCADEMICO 2024 - 2025

Dedico questo lavoro a me stesso

Indice

1	Introduzione	1
1.1	Obiettivi della Ricerca	2
1.2	Problematiche	2
1.3	Struttura della Tesi	2
2	Stato dell'arte	5
2.1	Procedural Content Generation (PCG)	5
2.1.1	Giochi che utilizzano la PCG	6
2.1.2	Caratteristiche di un algoritmo di PCG	8
2.2	Generazione di dungeon	9
2.2.1	Grammatiche generative	11
2.2.2	Modelli basati su grafi - Mission Graph	13
2.2.3	Search-Based PCG	16
2.2.4	Algoritmi evolutivi	17
2.2.5	Diagrammi di Voronoi	21
3	Metodologia	25
3.1	Definizione della struttura logica	25
3.1.1	Definizione del grafo di base - Main Path	25
3.1.2	Applicazione della grammatica generativa	26
3.1.3	Regole di produzione	27
3.1.4	Espansione del Main path	29
3.2	Solvibilità del grafo	30
3.3	Generazione dei template di stanze (offline)	32
3.3.1	Generazione del layout	33

3.3.2	Ottimizzazione evolutiva	36
3.3.3	Intervento manuale e salvataggio	37
3.4	Istanziamento del livello fisico	38
3.4.1	Generazione delle posizioni	38
3.4.2	Scelta dei prefabbricati	41
3.4.3	Costruzione dei corridoi	44
3.5	Riempimento e gestione delle stanze	46
3.5.1	Individuare punti di spawn (Voronoi)	47
3.5.2	Decorazioni	49
3.5.3	Gestione trappole	51
3.5.4	Gestione nemici	53
3.5.5	Gestione oggetti	56
4	A Bug Fix	59
4.1	Descrizione	59
4.2	Personaggi	59
4.3	Storia	60
4.3.1	Tema	61
4.4	Trama	61
4.5	Gameplay	62
4.5.1	Obiettivi	62
4.5.2	Abilità del giocatore	63
4.5.3	Meccaniche di gioco	63
4.5.4	Oggetti	66
4.6	Progressione	68
4.7	Sconfitta	68
4.8	Art Style	69
4.9	Musica e Suono	69
5	Implementazione	71
5.1	Moduli - Generazione dei livelli	73
5.1.1	WorldGen	73
5.1.2	GraphPipeline	75
5.1.3	GridPlacerBFS	81

5.1.4	RoomSpawner	85
5.1.5	CorridorBuilder	89
5.1.6	WorldFinalize	91
5.1.7	RoomsManager	91
5.2	Moduli - Generazione delle stanze (Offline)	96
5.2.1	RoomLayoutMask	96
5.2.2	EvolutionEngine	96
5.2.3	LayoutFitnessEvaluator	99
5.3	TopKSelection	105
5.4	ElitistHybridReproduction	105
5.5	LayoutGenomeMutator	106
5.5.1	LayoutToRoomTemplateBuilder	107
6	Sperimentazione	109
6.1	Obiettivi della sperimentazione	109
6.2	Metodologia sperimentale	110
6.2.1	Design sperimentale	110
6.2.2	Partecipanti	111
6.2.3	Protocollo sperimentale	111
6.3	Strumenti di valutazione	112
6.3.1	Game Experience Questionnaire	112
6.3.2	Scala di misura	117
6.4	Analisi dei dati	117
6.5	Risultati della valutazione	118
6.5.1	Risultati del Core Module	118
6.5.2	Risultati del Post Game Module	122
6.5.3	Interpretazione dei risultati	125
7	Conclusioni	127
7.1	Considerazioni	128
7.1.1	Punti di Forza	128
7.1.2	Punti di Debolezza	128
7.1.3	Sviluppi Futuri	129
	Bibliografia	131

Capitolo 1

Introduzione

Fin dall'alba della civiltà umana, l'individuo ha costantemente ricercato modalità per esprimere pensieri ed emozioni, individuando nell'arte un mezzo privilegiato per rappresentare una profonda espressione dell'animo umano. Nel corso dei secoli, sono state sviluppate numerose forme d'arte che spaziano dall'arte rupestre e dalle prime forme di artefatti manifatturieri all'avvento del cinema, della fotografia e dell'arte digitale dei giorni nostri. La rapida evoluzione tecnologica iniziata nel secolo scorso ha reso possibile la creazione di un mezzo di comunicazione unico nel suo genere: i videogiochi. Essi integrano diverse forme d'arte, il cui studio può essere riassunto sotto il termine di *game design*. I videogiochi catturano e stimolano l'attenzione dei giocatori, offrendo esperienze sempre più immersive grazie all'incremento della potenza di calcolo dei dispositivi dedicati. Tale progresso ha consentito lo sviluppo e l'implementazione di algoritmi sofisticati in grado di generare universi complessi, convincenti e imprevedibili, noti collettivamente come *Procedural Content Generation (PCG)*.

Cosa si intende per PCG? Secondo la definizione fornita da Togelius, essa comprende “*algoritmi in grado di generare contenuti di gioco a partire da input limitati o indiretti forniti dall'utente*” [14] e si è affermata come un asset di fondamentale importanza nello sviluppo di videogiochi, dal momento che consente di ridurre significativamente i costi di produzione dei contenuti. Inoltre, stimola la creatività e la produttività dei designer, permette un controllo più preciso della difficoltà del gioco e contribuisce al suo bilanciamento. Infine, essa incrementa la ri-giocabilità di un titolo offrendo contenuti inaspettati e variabili [15].

1.1 Obiettivi della Ricerca

La presente tesi di laurea si propone di sviluppare e integrare un sistema di generazione procedurale dei livelli in un prototipo denominato “*A Bug Fix*”, un *action-platform* bidimensionale a visuale laterale con struttura *dungeon-like*. L’obiettivo primario è la valutazione empirica dell’impatto di tale sistema sull’esperienza utente. Un campione di utenti verrà selezionato per la valutazione del gioco, ciascun utente valuterà le due versioni del gioco. Ai partecipanti non verrà comunicata la natura dei livelli. Successivamente, i partecipanti saranno invitati a compilare il *Game Experience Questionnaire (GEQ)*. I dati raccolti saranno sottoposti ad analisi statistica e i risultati ottenuti dalle esperienze dei due gruppi di utenti saranno confrontati al fine di valutare se i livelli generati producono differenze statistiche significative rispetto a quelli progettati. In base ai risultati ottenuti, verranno formulate raccomandazioni per il miglioramento dell’applicazione della *PCG* nel contesto di giochi appartenenti alla stessa categoria strutturale del prototipo.

1.2 Problematiche

Il presente lavoro si articola su due dimensioni complementari: la progettazione di un sistema in grado di generare strutture esplorabili di tipo *dungeon-like* che integrino vincoli di progressione basati sulla cinematica del giocatore, sulle abilità e sugli oggetti, tenendo conto della natura ibrida del prototipo; e la valutazione empirica dell’impatto che tale sistema esercita sull’esperienza utente, al fine di verificare l’eventuale presenza di differenze significative rispetto al level design manuale.

1.3 Struttura della Tesi

La tesi è strutturata come segue:

- **Capitolo 2 - Stato dell’arte:** Analizzeremo lo stato attuale della *Procedural Content Generation (PCG)* applicata alla generazione di livelli esplorativi strutturati, con particolare attenzione ai modelli *dungeon-like*, fornendo una panoramica dei principali approcci algoritmici e un breve excursus storico sui videogiochi che ne fanno uso.

- **Capitolo 3 - Metodologia:** Illustreremo l'impiego degli algoritmi selezionati per affrontare la problematica della generazione procedurale dei livelli.
- **Capitolo 4 - A Bug Fix:** Descriveremo il prototipo che ha costituito la base sulla quale sono stati implementati gli algoritmi procedurali per la generazione dei livelli.
- **Capitolo 5 - Implementazione:** Dettaglieremo l'implementazione tecnica degli algoritmi utilizzati nel motore di gioco.
- **Capitolo 6 - Sperimentazione:** Questa sezione illustra la metodologia sperimentale adottata, il campione di partecipanti coinvolto, gli strumenti di valutazione impiegati e la presentazione dei risultati ottenuti. L'analisi si concentra sulle differenze nell'esperienza di gioco tra le due versioni, utilizzando le metriche del Game Experience Questionnaire.
- **Capitolo 7 - Conclusioni:** Illustreremo le conclusioni della ricerca e i dati ottenuti saranno utilizzati per delineare i punti di forza e di debolezza della *PCG* applicata al prototipo. Verranno inoltre delineati eventuali sviluppi futuri e potenziali direzioni di ricerca.

Capitolo 2

Stato dell'arte

Il presente capitolo si propone di analizzare lo stato attuale della generazione procedurale di contenuti nel settore videoludico, fornendo esempi concreti che ne giustificano l'adozione; si preoccupa, inoltre, di esaminare il concetto di *dungeon-like* e di come tali strutture possano essere correlate alla risoluzione del problema in oggetto; infine, esplora le tecniche utilizzate nella letteratura contemporanea per affrontare la complessa problematica della generazione dei *dungeon*.

2.1 Procedural Content Generation (PCG)

La PCG rappresenta un insieme di *software* in grado di generare contenuti di gioco in maniera autonoma o con il supporto limitato di uno o più designer che forniscono input esterni, i quali fungono da parametri di controllo per l'algoritmo utilizzato. Per “contenuti” si intende l'insieme di elementi che costituiscono un prodotto videoludico, tra cui: livelli, mappe, regole di gioco, *textures*, oggetti, missioni, musiche, armi, personaggi e molti altri.

Hendrikx et al. definiscono la Procedural Content Generation (PCG) come l'impiego di sistemi informatici per la generazione di contenuti di gioco, distinguendo le istanze di interesse da quelle generate e selezionando quelle potenzialmente rilevanti per l'utente finale. Hendrik sottolinea inoltre l'importanza non solo della potenza computazionale dei dispositivi, ma anche della capacità di valutare gli aspetti tecnici e culturali delle istanze generate. Viene inoltre approfondito il motivo per cui l'interesse verso questo strumento sia cresciuto significativamente negli ultimi anni

a causa di diversi fattori (tra i quali l'aumento della complessità dei giochi moderni) che impone agli sviluppatori di contenere i costi di produzione di software raffinati ed estremamente complessi come i videogiochi, e la continua ricerca di esperienze innovative e personalizzate [8].

2.1.1 Giochi che utilizzano la PCG

Nei primi anni '80, la limitata capacità di memoria dei computer domestici imponeva restrizioni significative alle capacità di memorizzazione dei contenuti videoludici. Tale vincolo ha incentivato gli sviluppatori a ricercare metodi alternativi per la generazione e la memorizzazione di tali contenuti. Uno dei primi giochi che risolse efficacemente questo problema fu *Elite* [5]. Questo software era in grado di generare proceduralmente 8 galassie, ognuna contenente 256 pianeti, ciascuno dei quali presentava proprietà uniche. La generazione veniva effettuata mediante la memorizzazione di un *seed* (numero intero).



Figura 2.1: Screenshot dall'originale *Elite*

Con l'avanzare degli anni e l'incremento delle capacità tecnologiche, la PCG è stata applicata con crescente sofisticazione. *Diablo* (Blizzard, 1996), un *action role-playing game hack-and-slash*, distribuisce nemici e oggetti all'interno di mappe generate proceduralmente. *Spore* (Maxis, 2008) consente ai giocatori di creare le proprie creature attraverso una vasta gamma di parametri modificabili, quali: forma e dimensione del corpo, diverse fauci, ali, zampe, ecc. Successivamente, vengono applicate tecniche di animazione procedurale per animare i design creati.

Minecraft (Mojang, 2011), un gioco estremamente popolare e di recente creazione, utilizza ampiamente la PCG per generare l'intero spazio di gioco, oltre che i suoi contenuti.



Figura 2.2: Una porzione di uno dei mondi di *Minenecraft*

The Binding of Isaac (Edmund McMillen, 2011), uno dei *roguelike* più celebri degli ultimi anni, impiega un algoritmo di generazione che per iniziare posiziona una prima stanza, successivamente espande il livello attraverso la creazione di altre stanze adiacenti, infine, posiziona oggetti, nemici e altri elementi in modo casuale, garantendo una rigiocabilità pressoché infinita.



Figura 2.3: Stanza iniziale del primo livello di una genercia partita in *The Binding of Isaac*

2.1.2 Caratteristiche di un algoritmo di PCG

Questi software avanzati richiedono che la soluzione al problema della generazione dei contenuti possieda specifiche caratteristiche, desiderabili o imprescindibili, al fine di garantire una risoluzione efficace del problema in questione. Nell'opera "Procedural Content Generation in Games"[14], Togelius fornisce un'analisi approfondita delle caratteristiche e una classificazione degli algoritmi. Tra tali caratteristiche, si annoverano:

- **Velocità:** è il tempo massimo necessario per generare i contenuti e può oscillare da millisecondi a mesi, a seconda che l'operazione sia eseguita durante la fase di *gameplay* o durante lo sviluppo del gioco.
- **Affidabilità:** la procedura deve soddisfare specifici criteri di qualità che variano a seconda del contenuto generato. Ad esempio, un *dungeon* privo di ingressi o uscite rappresenta un evento bloccante per il giocatore, mentre un oggetto come un cespuglio con una forma insolita non ha effetti catastrofici sul gioco.
- **Controllabilità:** questi algoritmi devono possedere un certo grado di controllabilità, che consenta agli utenti o ad altri algoritmi di generare aspetti specifici.
- **Espressività e diversità:** è fondamentale che i risultati della generazione presentino diversità l'uno dall'altro, evitando minuscole variazioni. Consideriamo due estremi: un generatore di livelli che ad ogni generazione restituisca lo stesso livello, modificando casualmente solo il colore di una singola pietra, e un generatore che assembla componenti totalmente a caso, producendo livelli ingiocabili. Entrambi sono esempi di non espressività.
- **Creatività e credibilità:** per aumentare l'immersività, è necessario evitare i casi in cui il contenuto risulti creato da un algoritmo e sia più simile al lavoro prodotto da un designer.

È fondamentale accettare eventuali compromessi tra le proprietà, ad esempio tra velocità e qualità.

Gli algoritmi generativi possono essere classificati secondo i seguenti criteri:

- **Online/Offline:** la generazione online si verifica quando i contenuti vengono creati durante l'interazione del giocatore con il gioco. Al contrario, la generazione offline avviene durante la fase di sviluppo del gioco o prima dell'inizio di una sessione di gioco.
- **Necessaria/Opzionale:** alcuni contenuti generati sono essenziali per il progresso del giocatore all'interno di un livello o per la sua conclusione e, pertanto, devono essere sempre accurati. Altri contenuti, invece, sono di natura ausiliaria.
- **Gradi di controllo:** la generazione dei contenuti può essere gestita attraverso diverse opzioni, tra cui l'utilizzo di un seed casuale o un vettore multidimensionale che consente di specificare i parametri dell'algoritmo.
- **Generica/Adattiva:** i contenuti possono essere generati tenendo conto (o meno) del comportamento del giocatore durante una sessione di gioco. In tal caso, alcuni parametri, come ad esempio la difficoltà, possono essere modificati.
- **Stocastica/Deterministica:** la generazione deterministica si manifesta quando si ottengono contenuti identici a seguito dell'utilizzo degli stessi parametri di input.
- **Costruttiva/Generated and Test:** la generazione costruttiva avviene in un unico passaggio. In alternativa, l'output passa attraverso un processo di validazione mediante un ciclo di generazione/verifica.
- **Automatic/MixedAuthorship:** storicamente, gli sviluppatori erano gli unici a poter modificare un set limitato di parametri durante la fase di sviluppo per gestire la generazione. Tuttavia, è emerso un nuovo paradigma che coinvolge anche i giocatori nel processo, creando una cooperazione tra designer e giocatori.

2.2 Generazione di dungeon

Nel contesto reale, il termine “dungeon” designa prigioni o celle sotterranee. Nel contesto videoludico e dei giochi da tavolo, invece, rappresenta un metodo comune

per esprimere dei livelli caratterizzati da specifiche peculiarità. Tali livelli, generalmente strutturati come stanze collegate da corridoi (room-based), contengono numerosi elementi di interesse per il giocatore, tra cui: tesori, personaggi non giocanti (nemici, alleati e neutrali), decorazioni (tipicamente di ambientazione fantasy) coerenti con l'ambiente circostante, trappole e altri elementi. Tuttavia, questa definizione non è esaustiva, poiché il concetto si è evoluto nel tempo e ora comprende anche complessi di caverne, rovine antiche, costruzioni artificiali e altri elementi. Un tratto distintivo delle strutture *dungeon-like* risiede nel bilanciamento tra la libertà di esplorazione, il controllo della progressione e il ritmo dell'esperienza. Tali strutture promuovono un'esplorazione libera, consentendo al giocatore di scegliere tra le diverse stanze disponibili, scoraggiandolo, al contempo, dall'affrontare sfide che eccedano le proprie attuali capacità.

La generazione procedurale di questi livelli può essere concettualizzata come un processo articolato su tre livelli distinti ma interdipendenti:

- **Modello Rappresentativo:** Un'astrazione impiegata per semplificare la struttura del livello, in grado di definire la rappresentazione di un dungeon, fornendo una visione d'insieme della configurazione finale.
- **Metodo di Costruzione del Modello:** Un insieme di regole o algoritmi incaricati di generare la rappresentazione astratta del modello, determinando la struttura e la disposizione dell'ambiente di gioco.
- **Realizzazione Geometrica:** La traduzione dell'astrazione progettata in una configurazione spaziale concreta, che determina l'effettiva geometria dell'ambiente di gioco.

La separazione tra dimensione logica e implementazione geometrica riveste un'importanza cruciale, in quanto consente di disgiungere il problema della struttura topologica dalla sua realizzazione spaziale concreta. Pur essendo tradizionalmente considerati distinti dai platform, i dungeon-like possono presentare alcune analogie. Nei platform, la progettazione dello spazio è fortemente condizionata dalla cinematica del personaggio, in particolare dalla presenza della gravità e dalle limitazioni imposte dal salto verticale. Al contrario, i dungeon pongono l'accento sulla connettività topologica e sulla sequenza delle sfide. Nei sistemi ibridi, la generazione procedurale deve integrare entrambe le dimensioni. Pertanto, non è sufficiente

garantire la semplice connessione tra le stanze a livello astratto: ogni transizione spaziale deve essere compatibile con le capacità motorie previste per il giocatore. È necessario, quindi, adottare metodi di valutazione in grado di verificare la coerenza della struttura generata sia in termini topologici sia rispetto a vincoli di percorribilità previsti dal sistema di movimento del giocatore. Di seguito sono presentati alcuni degli approcci algoritmici sviluppati nel corso degli anni per affrontare questa complessa sfida.

2.2.1 Grammatiche generative

Le grammatiche generative sono un formalismo matematico introdotto negli anni '50 dal linguista Noam Chomsky e consentono la descrizione della struttura sintattica delle lingue naturali [2]. Formalmente, una grammatica generativa G è definita come una quadrupla

$$G = (V_N, V_T, P, S)$$

dove:

- V_N rappresenta un insieme finito di simboli non terminali, suscettibili di sostituzione;
- V_T costituisce un insieme finito di simboli terminali, disgiunto da V_N , e forma la stringa finale;
- S è il simbolo iniziale, appartenente a V_N , dal quale ha origine il processo di derivazione;
- P è un insieme finito di regole di produzione, in cui ogni regola associa una stringa di simboli appartenente a

$$V = V_S \cup V_T.$$

Questa definizione consente alle grammatiche generative di configurarsi come un sistema in grado di generare esclusivamente le stringhe valide di un linguaggio.

Ogni regola grammaticale è costituita da una parte sinistra, denominata *Left-Hand Side (LHS)*, e da una parte destra, indicata come *Right-Hand Side (RHS)*. L'applicazione della grammatica prevede una ricerca all'interno della stringa, in

cui ogni volta che un simbolo o una sequenza di simboli compare nella LHS, viene sostituito dalla corrispondente regola di sostituzione specificata dalla RHS. Tale processo viene ripetuto un numero predeterminato di volte o fino a quando tutti i simboli della stringa risultano essere terminali.

Di seguito è riportato un esempio di grammatica generativa e la relativa soluzione:

Grammatica:

1. $S \Rightarrow BAnA$
2. $B \Rightarrow s$
3. $A \Rightarrow t$
4. $n \Rightarrow r$

Soluzione:

Passo	Simbolo espanso	RHS	Forma risultante
1	S	$BAnA$	$BAnA$
2	B	s	$sAnA$
3	A	t	$stnA$
4	n	r	$strA$
5	A	t	$strt$

Le grammatiche generative si classificano in due archetipi principali:

- **Grammatiche di forma (Shape Grammars):** nelle quali le regole definiscono le modalità con cui le forme geometriche di base possono essere sostituite da configurazioni più complesse, garantendo una coerenza spaziale;
- **Grammatiche a grafo (Graph Grammars):** queste grammatiche rappresentano un'estensione delle grammatiche formali, in cui le regole di produzione vengono applicate su grafi anziché su stringhe (ogni produzione specifica la sostituzione di un sottografo con un altro).

2.2.2 Modelli basati su grafi - Mission Graph

Come precedentemente discusso, le regole di produzione delle grammatiche generative possono essere applicate ai grafi (*Graph Grammar*). Tali strutture risultano particolarmente efficaci nella rappresentazione dello spazio di gioco e degli obiettivi che, congiuntamente, lo costituiscono. La loro notevole flessibilità consente la creazione di livelli con gradi di complessità differenti, a seconda delle scelte di game design. Ad esempio, un gioco lineare potrebbe richiedere la semplice esistenza del percorso principale (solitamente risolto tramite una stringa), mentre per categorie di giochi che richiedono strutture più complesse, quali i dungeon, l'adozione del grafo come astrazione permette una rappresentazione agevole di diverse informazioni.

Nelle grammatiche basate sui grafi, l'elemento *LHS* (*L*) rappresenta un sottografo che deve essere sostituito o modificato, mentre l'elemento *RHS* è il sottografo che sostituisce *L* una volta individuato il pattern matcher.

L'applicazione delle regole di riscrittura su un grafo si realizza mediante i seguenti cinque passaggi:

1. identificazione del sottografo che coincide con la *LHS* e memorizzazione degli identificatori dei nodi di tale sottografo;
2. rimozione di tutti gli archi tra i nodi memorizzati;
3. sottoposizione dei nodi memorizzati a trasformazioni secondo le regole definite dalla *RHS*;
4. aggiornamento dei collegamenti tra i nodi secondo la *RHS*;
5. rimozione dei nodi memorizzati.

Nei suoi studi, Joris Dormans [4] ha proposto una strategia che prevede l'impiego della grammatica generativa basata sui grafi per la generazione di livelli appartenenti al genere action-adventure. La scelta di questo approccio da parte di Dormans è motivata dalla natura intrinsecamente non lineare dei grafi, che si rivela particolarmente adatta per i giochi che prevedono elementi di esplorazione. Dormans sostiene, in questa ricerca, che la creazione di layout sofisticati richieda un processo di generazione separato. È necessario, pertanto, generare una mappa topologica, denominata “missione”, che rappresenti i passi necessari per il completamento del

livello e sulla base della quale venga costruita la geometria del livello, denominata “spazio”. La generazione del grafo di missione (*Mission Graph*) avviene mediante l'utilizzo delle *Graph Grammars*, mentre la generazione dello spazio avviene mediante l'utilizzo delle *Shape Grammars*. Questo approccio consente di sfruttare i punti di forza di entrambi i metodi al fine di generare livelli in grado di adattarsi dinamicamente alle azioni del giocatore.

Il concetto è stato ulteriormente sviluppato da van Der Linden et al. [10] attraverso la proposta di “grammatiche di gameplay” per la generazione di livelli di dungeon. In questo contesto, i designer definiscono i vincoli di progettazione mediante un vocabolario specifico per il gameplay. Tale vocabolario comprende le azioni eseguibili dal giocatore durante il gioco, la loro sequenza e composizione, nonché le relazioni e i contenuti associati. I limiti imposti dagli sviluppatori conducono direttamente alla creazione di una *Graph Grammars*, denominata “grammatica di gameplay”, in grado di strutturare grafi di azioni del giocatore, traducibili in layout di livelli. Tale approccio consente ai designer di specificare e controllare la generazione del dungeon attraverso un vocabolario più naturale e orientato alla progettazione, permettendo inoltre la definizione di parametri personalizzati (ad esempio, difficoltà) che influenzano la riscrittura delle regole della grammatica.

Un approccio più recente prevede l'impiego congiunto di *Generative Adversarial Networks* (*GAN*) e *Mission-Graph* [7]. Nell'articolo, gli autori utilizzano le *GAN*, addestrate per la generazione di stanze a partire dal *Video Game Level Corpus* (*VGLC*), una vasta raccolta open-source di livelli di videogiochi, strutturata e formattata per agevolare l'analisi da parte di algoritmi e sistemi di intelligenza artificiale. Le stanze generate vengono organizzate in un dungeon attraverso una grammatica generativa, mappando una missione di alto livello progettata da un designer, creando così dungeon nuovi e interessanti, di dimensione arbitraria. Il grafo è organizzato su una backbone iniziale, una sequenza di simboli non-terminali rappresentanti stanze che devono essere presenti all'interno del dungeon, i quali vengono sostituiti finché il grafo non contiene solo nodi terminali. Le stanze vengono quindi posizionate in ordine breadth-first, posizionando la stanza iniziale; tutte le successive vengono posizionate in una posizione ortogonale casuale adiacente alla stanza precedentemente posizionata. Potrebbe verificarsi la situazione in cui non vi sia spazio attorno ad una stanza per ospitare i suoi vicini. Per garantire che

tutte le stanze vengano posizionate, l'algoritmo mantiene una lista di archi (tra le stanze), anch'essa ordinata in breadth-first dalla stanza iniziale. Quando si verifica questa problematica, la lista viene iterata e si tenta di posizionare la stanza; se ciò non è possibile, l'algoritmo effettua backtracking, annulla l'ultimo posizionamento e tenta un'alternativa non ancora provata. Questo processo si ripete finché la lista degli archi non è stata interamente visitata. È importante sottolineare che, sebbene il grafo rappresenti la connettività delle stanze, il layout bidimensionale tende a non rappresentare fedelmente i collegamenti presenti nel grafo originale. Il layout si prefigge di corrispondere al grafo nel modo più accurato possibile. I risultati di questo studio dimostrano come questo approccio ibrido consenta di creare livelli generati simili a quelli creati manualmente.

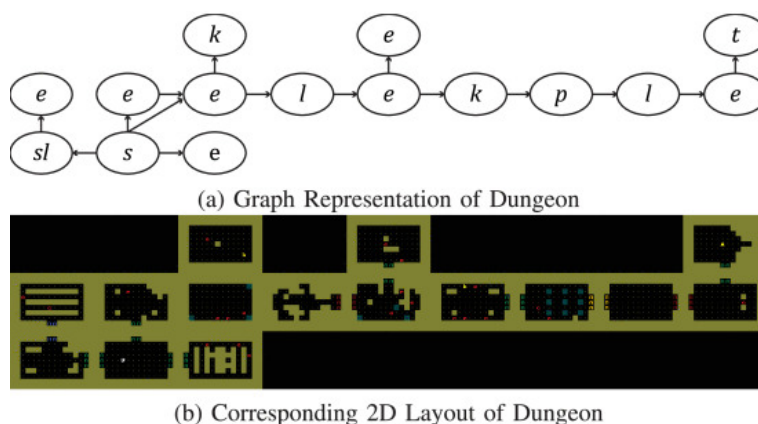


Figura 2.4: Un grafo (a) genera un dungeon (b): ogni nodo è una stanza, ogni arco una porta. La GAN determina il layout delle stanze. Notiamo che un bordo (da s a e) viene perso nella generazione. Fonte: [7].

L'impiego di tali strutture consente di conseguire diversi vantaggi, tra cui:

- la produzione di strutture complesse attraverso specifiche compatte;
- la capacità di controllare la generazione attraverso parametri di alto livello;
- l'integrazione di considerazioni di gameplay mediante vocaboli naturali nella struttura generata;
- la flessibilità e l'adattabilità a diversi stili di gioco e requisiti di design.

2.2.3 Search-Based PCG

Sono disponibili diverse metodologie per affrontare un problema di PCG, tra cui l'approccio *Search-Based*, ampiamente utilizzato e oggetto di studio. Questa metodologia impiega algoritmi di ricerca stocastici e funzioni di valutazione per individuare contenuti che presentano specifiche qualità. L'approccio si basa sull'assunto che nello spazio delle soluzioni ne esista una discretamente buona. Attraverso iterazioni e modifiche di una o più soluzioni, preservando le caratteristiche che migliorano il campione e scartando quelle dannose, si giunge progressivamente alla soluzione più appropriata.

Questo metodo richiede tre componenti fondamentali [13]:

1. **Un algoritmo di ricerca:** questo componente definisce la metodologia per la ricerca del contenuto al fine di individuare la soluzione ottimale. Gli algoritmi evolutivi si dimostrano spesso particolarmente adatti a questo scopo.
2. **Un metodo di rappresentazione del contenuto:** per consentire la ricerca e la valutazione del contenuto, è necessario che esso sia rappresentato in una forma specifica. È importante sottolineare che il tipo di rappresentazione influisce sia sull'efficienza degli algoritmi di generazione sia sullo spazio dei contenuti che il metodo è in grado di coprire.
3. **Una o più funzioni di valutazione:** Queste funzioni assegnano un punteggio (un valore di fitness o di valutazione) a ciascuna rappresentazione della soluzione. Generalmente, tali funzioni sono progettate per valutare la qualità dell'artefatto in esame, come ad esempio la giocabilità, l'intrattenimento, l'estetica di un livello e molti altri aspetti.

Le funzioni di valutazione possono essere classificate in tre categorie [13]:

1. **Funzioni di valutazione diretta:** le caratteristiche estratte dal contenuto generato vengono mappate a un valore di fitness. Queste funzioni tendono a valutare un singolo aspetto del contenuto di gioco, risultando facili da implementare e veloci da eseguire. Tuttavia, può risultare complesso definire una funzione di valutazione diretta per alcuni aspetti del contenuto.

2. **Funzioni di valutazione Simulation-Based:** questo approccio utilizza agenti di Intelligenza Artificiale per valutare la qualità dei contenuti. Il punteggio viene calcolato statisticamente in base al comportamento dell'agente e al suo stile di gioco. Le competenze e le capacità dell'agente possono essere modificate in base al tipo di valutazione e all'obiettivo desiderato.
3. **Funzioni di valutazione interattive:** consentono la valutazione del contenuto attraverso la raccolta diretta di dati dagli utenti che interagiscono con esso. Questi dati si distinguono tra impliciti ed espliciti. I dati impliciti vengono raccolti mediante l'analisi statistica delle azioni e dei comportamenti degli utenti (ad esempio, il tempo impiegato per completare un livello o il numero di monete raccolte). I dati espliciti, invece, richiedono la partecipazione attiva dell'utente attraverso la compilazione di moduli. Entrambi gli approcci presentano delle limitazioni. La raccolta di dati espliciti, se non adeguatamente integrata, può interrompere le sessioni di gioco, mentre la raccolta di dati impliciti è spesso basata su ipotesi e soggetta a rumore. Pertanto, si adotta comunemente una strategia ibrida.

2.2.4 Algoritmi evolutivi

Gli algoritmi evolutivi si ispirano ai principi della teoria dell'evoluzione di Darwin, in particolare al concetto di selezione naturale. Tali algoritmi si concentrano sul mantenimento di una popolazione di individui (detti cromosomi o soluzioni candidate) che vengono valutati. Ad ogni generazione, vengono conservati esclusivamente gli individui più adatti, detti "*fittest*" (individui con un valore di fitness maggiore), i quali hanno la possibilità di riprodursi. Il processo di riproduzione prevede la duplicazione degli individui già adatti, con l'introduzione di modifiche specifiche (mutazioni) e/o la combinazione di elementi provenienti da più individui (ricombinazione o crossover). Gli individui meno adatti vengono eliminati dalla popolazione.

Al fine di approfondire la comprensione del funzionamento di un algoritmo evolutivo, si analizzerà la strategia evolutiva $\mu + \lambda$ [13]. Il parametro μ indica la dimensione della popolazione mantenuta tra le generazioni, costituita dall'élite. Il parametro λ , d'altro canto, rappresenta la dimensione della porzione di popolazione

generata attraverso il processo riproduttivo in ciascuna generazione. Per agevolare la descrizione dei passaggi successivi, si considererà $\mu = \lambda = 50$.

1. **Inizializzazione:** viene costituita una popolazione iniziale di individui, $\mu + \lambda$, che possono essere generati casualmente o includere individui progettati manualmente o derivanti da precedenti processi evolutivi.
2. **Permutazione:** la popolazione viene sottoposta a permutazione casuale (sebbene questa fase sia facoltativa, essa contribuisce a prevenire la perdita di gradiente).
3. **Valutazione:** gli individui vengono valutati mediante una funzione di fitness (o una combinazione di più funzioni), a ciascuno dei quali viene assegnato un valore di fitness.
4. **Ordinamento:** la popolazione viene ordinata in ordine crescente in base al valore di fitness.
5. **Selezione:** i λ individui con il valore di fitness più basso vengono eliminati.
6. **Riproduzione:** i λ individui rimossi vengono sostituiti da copie dei μ individui rimasti. I nuovi individui aggiunti (le copie) sono denominati “discendenti”.
7. **Mutazione:** i λ discendenti vengono sottoposti a mutazioni (perturbazioni casuali). Le mutazioni da implementare dipendono dal tipo di rappresentazione e, in una certa misura, anche dal panorama di fitness. Ad esempio, se la rappresentazione utilizza un vettore di numeri reali, un operatore di mutazione efficace è la mutazione gaussiana, che consiste nell’aggiungere numeri casuali estratti da una distribuzione gaussiana con una piccola deviazione standard al vettore.
8. **Terminazione:** l’algoritmo termina quando viene raggiunto il numero massimo di generazioni o viene generato un individuo di qualità sufficiente. In alternativa, si procede con la generazione successiva tornando al passo 2.

Pur presentando una struttura relativamente semplice, questo algoritmo si dimostra altamente efficace anche in configurazioni degenerate in cui $\mu = \lambda = 1$.

Al fine di catturare efficacemente le molteplici caratteristiche rilevanti di un individuo, si ricorre frequentemente all'utilizzo di più funzioni di valutazione. Le strategie evolutive non costituiscono l'unico algoritmo impiegabile negli algoritmi evolutivi; un'altra tipologia ampiamente diffusa negli studi recenti è quella degli algoritmi denominati *Quality-Diversity* (QD) [6]. Tali algoritmi sono in grado di preservare simultaneamente la qualità e la diversità delle soluzioni generate individuando il più ampio insieme possibile di soluzioni eterogenee e di alta qualità in un'unica esecuzione. Questo obiettivo viene perseguito premiando la divergenza tra individui, intesa come novità o sorpresa degli artefatti prodotti, e, contemporaneamente, mantenendo un rigoroso controllo sulla qualità delle soluzioni attraverso la competizione locale tra individui che presentano tratti comportamentali simili o mediante l'implementazione di vincoli stringenti. Benché gli algoritmi QD possano essere molto pesanti computazionalmente le loro esecuzioni possono fornire risultati di alta qualità.

Nell'articolo intitolato "Empowering Quality Diversity in Dungeon Design with Interactive Constrained MAP-Elites" [1], gli autori presentano l'approccio Interactive Constrained MAP-Elites, che integra la *Evolutionary Dungeon Designer* (EDD), uno strumento di *Mixed-initiative co-creativity* (MI-CC) [16], con l'implementazione dell'algoritmo MAP-Elites [11], uno degli algoritmi di Quality Diversity (QD) più riconosciuti. La MI-CC si riferisce a un processo di creazione in cui un elaboratore e un utente umano si ispirano reciprocamente, generando stimoli iterativi. L'EDD è uno strumento che utilizza un approccio evolutivo di tipo QD per fornire suggerimenti generati proceduralmente, guidando il designer umano nella creazione di dungeon 2D e delle stanze che li compongono. MAP-Elites mantiene una mappa di soluzioni valide, suddivisa in celle in base a una o più dimensioni delle caratteristiche, tipicamente due. Ogni cella conserva una singola soluzione. Ad ogni iterazione, viene generato un discendente a partire da una o più soluzioni preesistenti. Il nuovo individuo viene assegnato a una cella in base alle sue caratteristiche, che possono differire da quelle delle celle occupate dai suoi genitori. Qualora la prole presenti una soluzione superiore rispetto a quella esistente nella cella, essa sostituisce l'individuo della generazione precedente. Tale processo si conclude con la generazione di una mappa di soluzioni in cui ciascuna cella contiene la soluzione

ottimale individuata per la specifica dimensione delle caratteristiche. Nell'esperimento, gli autori hanno combinato un totale di cinque dimensioni (due per volta), utilizzando stanze di dimensioni 13 x 7. La popolazione iniziale di individui mutati è stata impostata a 1000, distribuiti in popolazioni fattibili e infattibili, con una capacità massima di 25 individui per cella. L'algoritmo evolutivo (EA) operava in modo continuo, selezionando 5 genitori per popolazione ogni 100 generazioni. La prole veniva ottenuta tramite crossover a due punti e mutazione del 30%. I risultati ottenuti dimostrano soluzioni con alta diversità qualitativa, creando stanze aperte o intricate.

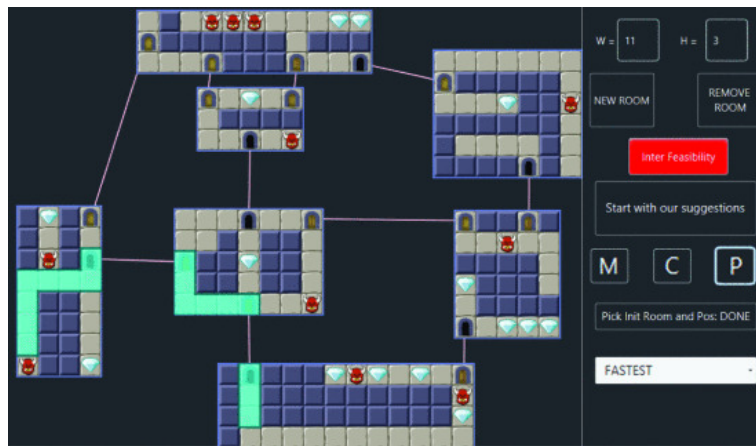


Figura 2.5: L'editor di dungeon **EDA** mostra un dungeon a sette stanze. Il pannello destro offre opzioni per modificare il dungeon: “M” per muovere le stanze, “C” per connetterle e “P” per calcolare il percorso. Il percorso più breve tra due tile è evidenziato in blu. Fonte: [1].

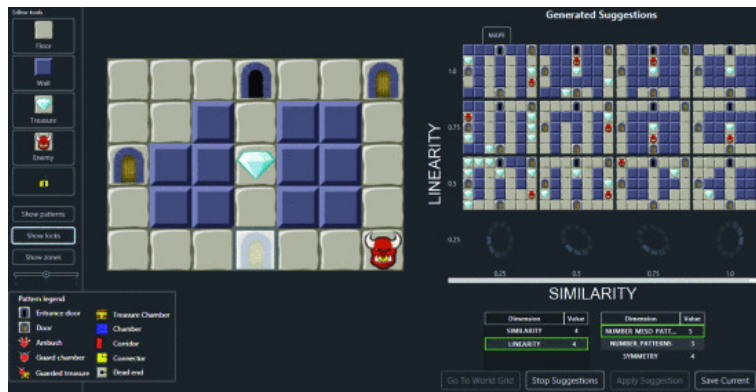


Figura 2.6: L'editor della stanza in **EDD** ha due sezioni: a sinistra, le opzioni per modificare manualmente la stanza visualizzata al centro; a destra, i suggerimenti generati proceduralmente. Fonte: [1].

Gli algoritmi evolutivi costituiscono uno strumento altamente versatile nella PCG per dungeon, consentendo un'esplorazione efficiente dello spazio delle possibili soluzioni al fine di individuare contenuti che soddisfino simultaneamente criteri strutturali, quali connettività e dimensione delle stanze, e obiettivi di gameplay, come il bilanciamento della difficoltà e la varietà. A differenza degli approcci costruttivi tradizionali, gli algoritmi evolutivi permettono l'ottimizzazione dei dungeon rispetto a molteplici funzioni di fitness, generando contenuti che bilanciano diversi aspetti dell'esperienza di gioco desiderata.

2.2.5 Diagrammi di Voronoi

I diagrammi di Voronoi, definiti formalmente da Georgy Voronoi nel 1908, sono strumenti utilizzati per rappresentare una struttura geometrica che consente la suddivisione di un piano in regioni specifiche determinate dalla distanza da un insieme finito di punti, denominati *siti*. Tali diagrammi generano partizioni dello spazio, note come *celle*; ogni cella comprende tutti i punti che risultano più vicini al proprio sito rispetto a qualsiasi altro sito. Formalmente, preso l'insieme dei punti finiti $S = \{s_1, s_2, \dots, s_n\}$, facenti parti del piano considerato, la cella di Voronoi $V(s_i)$ associata al sito s_i è definita come l'insieme dei punti dello spazio che sono più vicini a s_i rispetto che a un qualsiasi altro s_j con $j \neq i$:

$$V(s_i) = \{p \in \mathbb{R}^d \mid d(p, s_i) \leq d(p, s_j), \forall j \neq i\}$$

Dove $d(p, s)$ indica la distanza euclidea tra un punto p e un sito s nello spazio d -dimensionale $\mathbb{R}^d$. L'insieme delle celle di Voronoi di ciascun sito in S costituisce il **diagramma o il tassellamento di Voronoi** [3].

Nell'ambito della generazione procedurale di dungeon, i diagrammi di Voronoi consentono la creazione di mappe più realistiche e meno strutturate a griglia, conferendo ai dungeon un aspetto più naturale e variegato. Tale tecnica trova ampia applicazione nei seguenti ambiti:

1. **Generazione di aree organiche:** i diagrammi di Voronoi producono aree dall'aspetto più naturale e irregolare, a differenza degli approcci basati sulla costruzione di griglie regolari;

2. **Distribuzione di punti di interesse:** a partire dai siti generati, è possibile utilizzare l'insieme dei punti che compongono una cella per assegnare le posizioni delle entità di gioco (nemici, oggetti, decorazioni, ecc.);
3. **Flessibilità nella definizione degli spazi:** ciascuna cella di Voronoi può essere interpretata in modo differente a seconda degli obiettivi prefissati (celle come stanze, aree funzionali del dungeon o della stessa stanza).

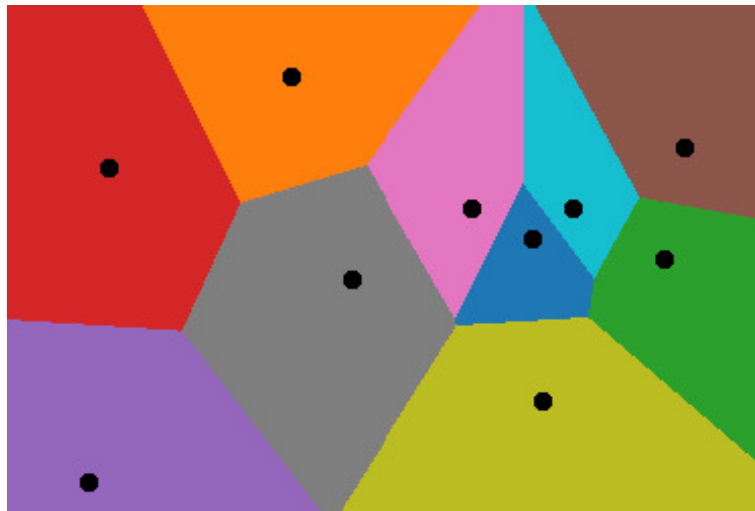


Figura 2.7: Rappresentazione di un diagramma di Voronoi: i punti neri rappresentano i *siti*, le aree colorate rappresentano le *celle*.

Tali tecniche si rivelano particolarmente efficaci nella generazione di dungeon caratterizzati da un aspetto meno rigido e meccanico, garantendo un equilibrio ottimale tra il controllo algoritmico e la variabilità organica.

La generazione dei tassellamenti di Voronoi è un problema computazionale ben studiato. Alcuni dei più usati approcci includono:

- **Approccio basato sulla triangolazione di Delaunay:** si avvale della dualità intrinseca tra i diagrammi di Voronoi e la triangolazione di Delaunay. La costruzione della triangolazione si realizza a partire dall'insieme dei siti. I vertici dei triangoli coincidono con i centri delle celle di Voronoi che, per definizione, sono i siti stessi. Questa caratteristica intrinseca implica che i punti connessi nella triangolazione presentano celle di Voronoi adiacenti. Tale proprietà consente la creazione di un grafo di connettività delle celle di Voronoi. Questa tecnica è ampiamente utilizzata nella generazione di maglie.

- **Algoritmo di Fortune:** si configura come uno dei metodi più efficienti per la generazione di diagrammi di Voronoi. Questo algoritmo impiega una tecnica di sweepline (linea di scansione) che consiste nell'attraversare il piano bidimensionale dal vertice superiore a quello inferiore. Durante tale attraversamento una sequenza di parabole, denominata *beach line*, viene generata a partire da ciascun sito incontrato. L'intersezione di queste parabole determina la formazione delle celle di Voronoi. L'Algoritmo di Fortune presenta una complessità $O(n \log n)$, dove n rappresenta il numero di siti.
- **Approccio per forza bruta:** si rivela adeguato per applicazioni caratterizzate da un numero limitato di siti o per implementazioni di diagrammi di Voronoi di semplicità moderata. In questo approccio, le celle del diagramma vengono determinate confrontando la distanza del punto considerato da ciascun sito e assegnando il punto al sito più vicino. Tale processo viene ripetuto per ogni punto del piano. Pur essendo meno efficiente rispetto ad altri algoritmi, l'approccio per forza bruta risulta spesso sufficiente nel contesto videoludico, dove non è richiesta la massima precisione, e si integra naturalmente con rappresentazioni basate su tile.

Un approccio innovativo viene impiegato in “Procedural Playable Cave Systems Based on Voronoi Diagram and Delaunay Triangulation” [12]. In questo articolo, gli autori sfruttano i diagrammi Voronoi e la triangolazione di Delaunay per sviluppare un sistema in grado di generare caverne bidimensionali e tridimensionali per terreni volumetrici. Il processo di generazione si basa su punti di interesse (POI), ovvero punti all'interno della caverna che presentano un valore significativo in termini di giocabilità (un esempio potrebbe essere un incontro con un nemico o la scoperta di un tesoro); questi punti vengono definiti dal designer. Prima della generazione della caverna, il designer che utilizza questo strumento deve inserire una serie di parametri, raggruppati in tre macro categorie: informazioni sui POI, informazioni sulle relazioni tra i POI e parametri globali. Successivamente, l'algoritmo genera un diagramma di Voronoi e una triangolazione di Delaunay. Viene quindi generato il primo punto di interesse e assegnato alla cella di Voronoi con le caratteristiche ottimali per il POI generato, sulla base delle informazioni fornite dal designer. In seguito, la caverna viene generata estraendo un POI da una lista che tiene traccia degli elementi non ancora assegnati. Una volta selezionato viene quindi posizionato

nella cella corrispondente e, in seguito, vengono tracciati i percorsi tra questi punti. Infine, il modello generato viene utilizzato per realizzare la caverna effettiva all'interno del terreno. Questo approccio consente la generazione di elementi e strutture inaspettate utilizzando un insieme di parametri direttamente correlati alla grotta stessa, anziché a concetti tecnici e matematici, eliminando così la necessità di competenze tecniche per definire tali parametri. Tuttavia, il tempo di esecuzione di questo metodo è influenzato dalla crescita della grotta, determinata da vari fattori, tra i quali il numero di gallerie e l'affinità tra i tipi di punti di interesse. Ciò rende il tempo di esecuzione variabile. Viene riportato un tempo di generazione complessivo di circa 11 secondi, che risulta eccessivamente elevato per garantire la generazione in tempo reale.

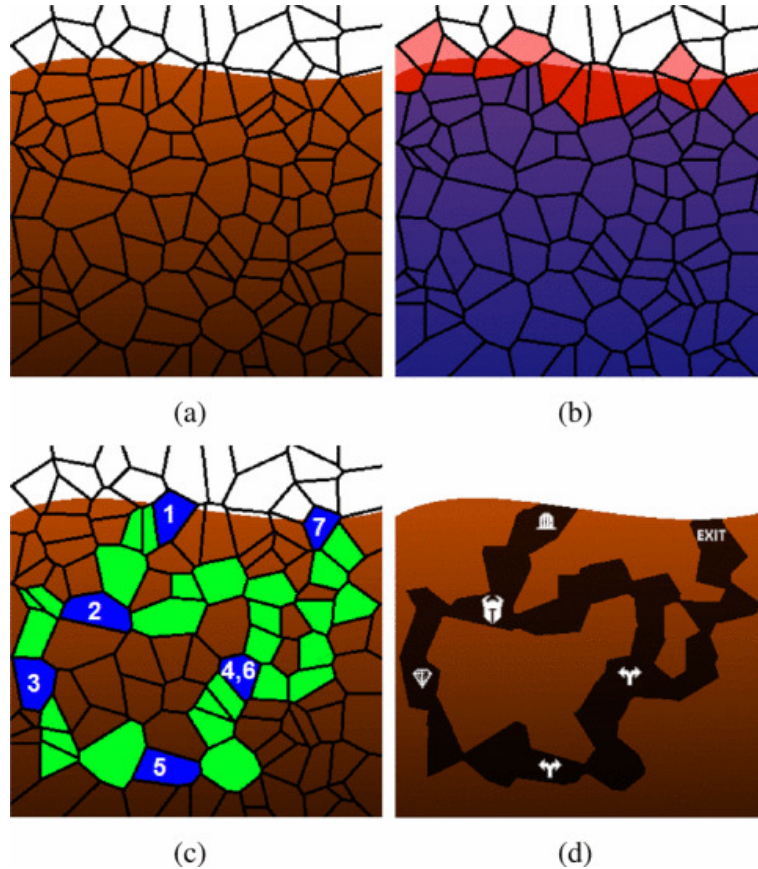


Figura 2.8: Il processo di generazione della grotta. (a) Un diagramma di Voronoi su terreno bidimensionale. (b) Classificazione delle celle: superficie (rosso) e interno (blu). (c) Cella selezionate per generare la grotta: 7 POI (numerati per ordine di generazione). (d) Grotta finale scavata sul terreno con POI assegnati. Fonte: [12].

Capitolo 3

Metodologia

Nel presente capitolo verrà fornita una descrizione astratta e facilmente comprensibile delle metodologie implementate per affrontare la problematica della generazione di livelli procedurali nell'ambito di questa tesi.

3.1 Definizione della struttura logica

Per avviare il processo di generazione del livello è necessario definire una struttura astratta che ne rappresenti la topologia. Tale struttura deve consentire al progettista di esercitare un controllo sui contenuti e sulla struttura stessa del livello. Questa rappresentazione viene realizzata mediante l'utilizzo di un grafo non orientato, il quale viene successivamente espanso attraverso l'applicazione delle regole di produzione definite dalla grammatica progettuale. Ciascun nodo del grafo rappresenta una missione che il giocatore dovrà compiere nel gioco e, contemporaneamente, il nodo mantiene traccia dei contenuti della stanza, mentre gli archi del grafo rappresentano i collegamenti che uniscono le stanze ed eventuali blocchi tra una stanza e l'altra.

3.1.1 Definizione del grafo di base - Main Path

Ogni livello generato nell'ambito di questo progetto è caratterizzato da un percorso principale, denominato *Main Path*, che costituisce la struttura fondamentale e imprescindibile che il giocatore deve percorrere per il completamento del dungeon.

Tale percorso viene definito mediante il grafo delle missioni e comprende le seguenti tipologie di stanze: *Start*, *Hub*, *Arena*, *Key Room*, *Challenge* e *Boss*. Ciascuna stanza ha specifici obiettivi. Durante la costruzione del grafo vengono specificate informazioni relative ai contenuti di ciascuna stanza, tra cui le abilità concesse o richieste, la presenza di eventuali nemici e le modalità di interazione con essi (il tipo di combattimento), nonché gli oggetti disponibili. Inoltre, vengono definiti eventuali vincoli esistenti tra i passaggi, rappresentati dagli archi, che consentono il transito tra le stanze, impedendo temporaneamente al giocatore l'accesso alle stanze vincolate fino al soddisfacimento dei requisiti imposti.



Figura 3.1: Rappresentazione del *Main Path* mediante grafo. Le frecce verdi indicano la presenza di un collegamento libero tra i nodi, mentre le frecce rosse segnalano un collegamento vincolato. Per consentire il transito su un collegamento vincolato, è necessario soddisfare specifici requisiti. **Nota:** il grafo in figura è orientato esclusivamente per descrivere tali vincoli.

3.1.2 Applicazione della grammatica generativa

La grammatica generativa opera in maniera deterministica sui nodi del grafo (*Graph Grammar*), definendo ulteriori missioni (*Mission Graph*). Prima della generazione del grafo, il progettista seleziona un numero che specifica la quantità massima di stanze che il grafo può contenere, denominato *Budget Nodes*. L'espansione del *Main Path* prosegue finché il numero di nodi nel grafo è inferiore al *Budget Nodes*. Ad ogni iterazione, viene selezionato un nodo (*LHS*), si individua la regola da applicare (*RHS*) e la si applica al nodo. Successivamente, gli archi dei nodi generati vengono

collegati opportunamente, come definito dalle regole stesse, e i nodi vengono memorizzati per una successiva espansione. Il nodo che ha innescato la generazione diventa un simbolo terminale e viene rimosso dalla lista dei nodi che possono essere sottoposti ad espansione. È importante sottolineare che l'applicazione di una regola su un nodo può generare uno o più nodi aggiuntivi; pertanto, *l'ordine* del grafo finale non corrisponde necessariamente al Budget Nodes e può risultare leggermente più ampio a seconda delle regole definite.

Simbolo	Abbreviazione	Descrizione
<i>Start</i>	<i>S</i>	Stanza iniziale del dungeon con un checkpoint.
<i>Hub</i>	<i>H</i>	Stanza priva di oggetti e nemici, funge da punto di interconnessione tra diverse stanze (incrocio).
<i>Arena</i>	<i>A</i>	Stanza con nemici e trappole.
<i>Key Room</i>	<i>K</i>	Stanza con nemici e trappole, fornisce un oggetto chiave, un'abilità o entrambe.
<i>Challenge</i>	<i>C</i>	Stanza con nemici e trappole, richiede l'utilizzo dell'abilità acquisita nella Key Room.
<i>Side</i>	<i>R</i>	Stanza di transito, dotata di trappole utilizzate per incrementare il senso di esplorazione e rendere il livello più labirintico.
<i>Save</i>	<i>SV</i>	Stanza con nemici e un checkpoint.
<i>Boss</i>	<i>B</i>	Stanza con nemici o boss e trappole, fine del dungeon.

Tabella 3.1: Simboli della grammatica.

3.1.3 Regole di produzione

Le specifiche di produzione definiscono con chiarezza gli output prodotti a seguito di un match tra il nodo generatore e la regola associata. Ogni specifica comprende:

- i nodi generati con una descrizione dettagliata degli oggetti, nemici e trappole associati unitamente ai parametri specifici;
- gli archi che collegano i nodi generati al nodo generatore, inclusi i vicoli associati;

- gli archi che collegano i nodi generati tra loro, inclusi i vicoli associati.

Regola	LHS	RHS
R1	Hub	Arena
R2	Arena	Side
R3	Arena	Key Room
R4	Arena	Challenge
R5	Challenge	Save
R6	Challenge	Hub

Tabella 3.2: Regole della *Graph Grammar* usate per la generazione del dungeon.

Per ogni regola definita si illustra l'espansione corrispondente al nodo generatore nelle figure seguenti. Si prega di notare che le immagini rispettano la stessa convenzione descritta nella figura 3.1. Inoltre, i nodi di colore celeste rappresentano il nodo generato prima della generazione, mentre i nodi di colore viola rappresentano lo stato che ne consegue all'applicazione delle regole.

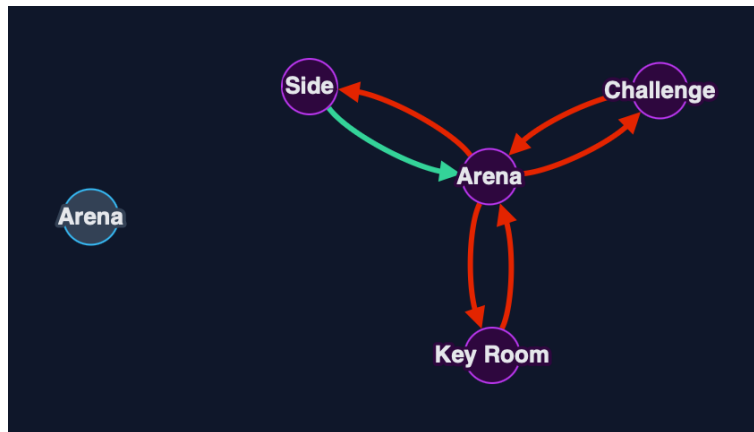


Figura 3.2: Applicazione della regola definita per il nodo generatore: Arena.



Figura 3.3: Applicazione della regola definita per il nodo generatore: Challenge.

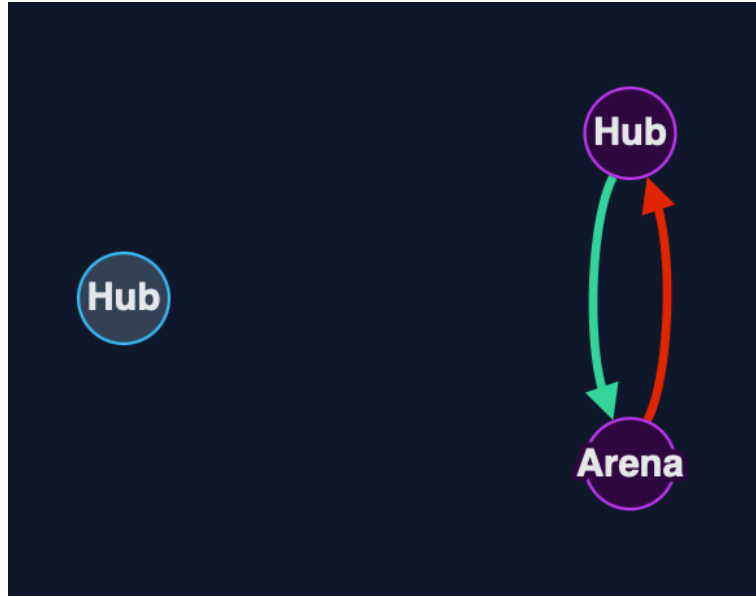


Figura 3.4: Applicazione della regola definita per il nodo generatore: Hub.

Si osserva che la progettazione delle regole è stata realizzata in modo tale che ciascuna di esse generi almeno un nuovo nodo non terminale; tale accorgimento assicura la continuità del processo di generazione attraverso le successive iterazioni dell’espansione fino al raggiungimento del limite di nodi stabilito.

L’implementazione di tali regole, pur nella loro apparente semplicità, consente la realizzazione di strutture complesse e facilmente gestibili attraverso parametri di alto livello. Inoltre, queste ultime risultano facilmente estendibili, consentendo l’integrazione di nuove regole e di nuove categorie di stanze.

3.1.4 Espansione del Main path

L’espansione del Main Path si realizza in conformità alle regole precedentemente stabilite. I nodi del Main Path vengono prioritariamente inseriti in una lista di nodi designati all’espansione e successivamente rimossi al termine del processo. Tale metodologia assicura che tutti i nodi presenti sul Main Path vengano espansi prioritariamente, prevenendo l’espansione in profondità del percorso di un singolo nodo. In seguito, verrà espanso il successivo “livello” del grafo, costituito dalla nuova lista di nodi da elaborare, popolata dai nodi generati durante l’espansione precedente. Tale processo si ripete fino al raggiungimento del limite definito dal Budget Nodes. Il risultato dell’espansione fornirà una struttura in grado di rappresentare il layout

di un livello e i suoi contenuti attraverso la manipolazione di semplici parametri, rendendo il livello facilmente controllabile e personalizzabile dal designer (integrazione di nuove categorie di stanze, nuove regole di produzione, nuovi elementi di gioco quali oggetti, nemici, trappole e nuovi vincoli tra stanze).

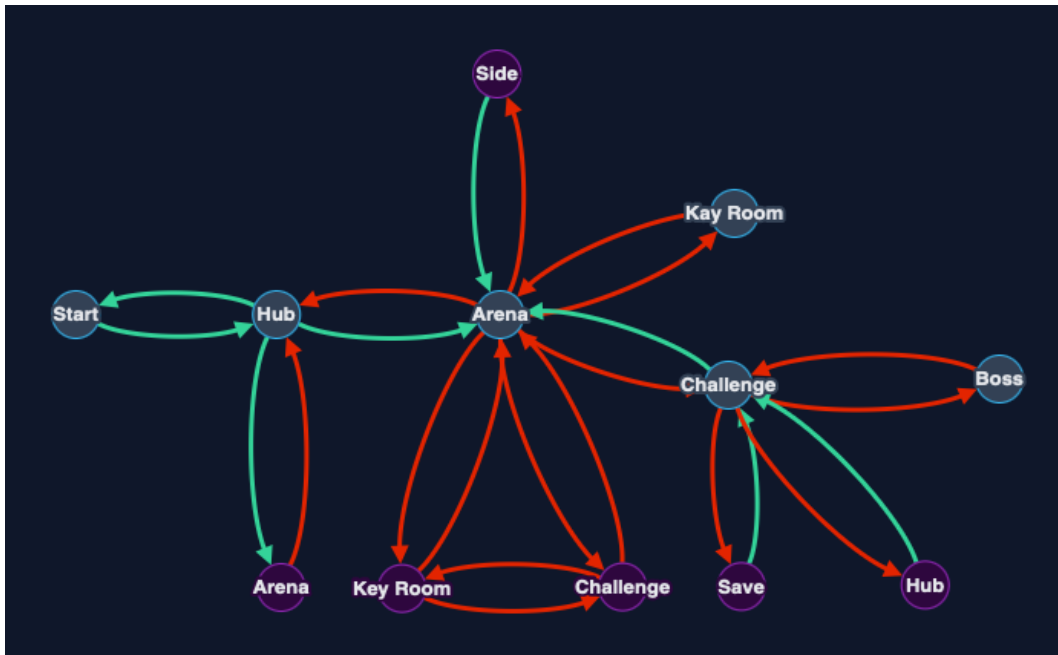


Figura 3.5: Un esempio di grafo espanso derivante dall'applicazione della grammatica definita con un *Budget Nodes* pari a 10. I nodi azzurri rappresentano il *Main Path*, mentre i nodi viola sono stati generati tramite l'applicazione delle regole di espansione. Si osservi che, nonostante il budget di nodi sia fissato a 10, il grafo presenta un totale di 12 nodi.

3.2 Solvibilità del grafo

A seguito della generazione del grafo è necessario verificare la fattibilità di completamento del livello da parte del giocatore. Ciò implica la simulazione del potenziale avanzamento del giocatore all'interno del grafo, al fine di confermare l'esistenza di almeno un percorso valido tra il punto di partenza e l'obiettivo finale, rappresentato dalla Boss Room. Partendo dal nodo iniziale, il grafo viene esplorato progressivamente, transitando per i nodi adiacenti e verificando, ad ogni passaggio, la disponibilità dei requisiti necessari da parte del giocatore per attraversare i collegamenti

tra le stanze. Durante l'esplorazione, si tiene traccia delle informazioni relative allo stato del giocatore, tra cui:

1. abilità acquisite che consentono il superamento di specifici ostacoli;
2. numero di chiavi possedute, utilizzate per l'apertura di porte che impediscono il proseguimento del gioco;
3. passaggi già sbloccati durante l'esplorazione.

All'arrivo del giocatore in un nodo contenente ricompense (abilità e/o oggetti), lo stato viene aggiornato di conseguenza (con l'aggiunta dell'abilità o dell'oggetto chiave). In tal modo, si simula la progressione del giocatore nel livello. Tale esplorazione prosegue finché:

- non viene raggiunto il nodo finale, nel qual caso il livello è considerato risolvibile;
- non esistono più percorsi esplorabili, indicando che il livello generato non può essere completato (in tal caso viene generato un nuovo grafo).

Questo controllo garantisce che il grafo generato offra almeno un percorso valido che conduca dall'inizio alla fine del livello.

Una volta convalidata la solvibilità del grafo, viene eseguita un'ulteriore simulazione. A ciascun nodo viene assegnato l'elenco delle abilità iniziali possedute dal giocatore al momento del suo ingresso in una stanza. Durante il gioco, al completamento di una stanza, il giocatore potrebbe acquisire una nuova abilità che viene aggiunta all'insieme delle abilità disponibili. Da quel momento in poi, tutte le stanze successivamente percorse includeranno tale nuova abilità tra quelle disponibili. Questo ulteriore passaggio consente, durante la conversione dalla rappresentazione astratta al layout fisico, di selezionare template che richiedono una specifica abilità per essere superati. Ciò permette una selezione dinamica dei template in base alle abilità precedentemente possedute dal giocatore e a quelle concesse dal livello.

Per una migliore comprensione di questo meccanismo, si consideri che il giocatore inizi il gioco con l'abilità *Grapple*. Di conseguenza, la stanza iniziale includerà tra le abilità richieste per la selezione del template quella appena menzionata. Continuando la simulazione, il giocatore attraversa l'*Hub*, l'*Arena* e infine la *Key Room*.

In tutte queste aree, l'insieme delle abilità disponibili rimane invariato rispetto a quello iniziale. Nella *Key Room*, il giocatore acquisisce una nuova abilità, il *double jump*. Da quel momento in poi, le stanze successive, comprese quelle generate dall'applicazione della grammatica, potranno opzionalmente includere tra le abilità richieste quella appena acquisita. È fondamentale evidenziare che i nodi che necessitano intrinsecamente di un'abilità specifica devono essere trasformati in un template che imponga l'utilizzo di tale abilità. Ad esempio, qualora un nodo *Challenge* richieda l'abilità *double jump*, il template corrispondente dovrà necessariamente includerla. Al contrario, nel caso precedentemente descritto, i nodi possono essere convertiti sia in template che richiedono l'abilità specifica, sia in template che non la richiedono, consentendo una maggiore diversificazione nella progettazione dei livelli.

3.3 Generazione dei template di stanze (offline)

Al fine di tradurre la topologia astratta in un contesto concreto, il designer deve innanzitutto progettare dei prefabbricati. Tale progettazione può essere realizzata attraverso due approcci distinti:

1. **Approccio completamente manuale:** il designer, nel rispetto di convenzioni predefinite, progetta autonomamente il proprio template modificando specifici parametri e costruendo la geometria della stanza.
2. **Approccio ibrido:** il designer ha la possibilità di utilizzare uno strumento specificamente implementato all'interno dell'ambiente di lavoro. Tale strumento consente la generazione di template modificabili o direttamente salvabili come template da utilizzare durante il processo di generazione, definendo, al contempo, le convenzioni necessarie per la creazione di un template valido.

La decisione di generare stanze offline è dettata dai seguenti fattori:

- Per garantire un livello di qualità adeguato, si è optato per un approccio *search-base*, in particolare un algoritmo evolutivo con una funzione multi-obiettivo, che verrà dettagliata nel successivo capitolo di questa tesi. Questa scelta evidenzia chiaramente che la generazione di stanze in runtime avrebbe comportato tempi di attesa significativamente più lunghi per il giocatore.

- Un ulteriore motivo risiede nella maggiore controllabilità offerta al designer. Lavorando direttamente sui template, il designer può progettare stanze che rispettino specifici vincoli di movimento, superabili dalle abilità che il giocatore possiede o acquisirà durante il gioco. In tal modo, con l'implementazione di nuove abilità, il designer potrà generare nuove stanze e adattare le più agevolmente alle nuove sfide proposte.

3.3.1 Generazione del layout

Per la generazione del layout di una stanza, è necessario innanzitutto definire un metodo di rappresentazione. Tale rappresentazione viene realizzata mediante l'utilizzo di una *mask* (una matrice). Inizialmente, la *mask* rappresenta la stanza come un unico blocco privo di spazi percorribili, in cui ogni cella della *mask* corrisponde a un elemento solido (terreno/muro). Le dimensioni della *mask* vengono selezionate casualmente all'interno di un range di valori prestabilito, che il designer ha la facoltà di modificare.

Successivamente alla sua creazione, viene implementato un operatore che specifica le posizioni dei punti cardinali lungo il perimetro delle stanze, in corrispondenza delle quali verranno generate aperture che consentiranno il passaggio dalla stanza corrente ad altre stanze. Le posizioni e le dimensioni delle aperture vengono memorizzate in quanto, tali informazioni, saranno fondamentali per la transizione dalla rappresentazione al layout fisico della stanza.

A questo punto, viene applicato un ulteriore operatore strutturale che scava corridoi interni, collegando le direzioni indicate. Su questi corridoi vengono eseguite lievi variazioni casuali nella traiettoria, al fine di generare forme meno artificiali. Questo processo fornisce uno spazio percorribile per il giocatore e garantisce percorsi tra le diverse aperture. La larghezza dei corridoi scavati e dei connettori viene determinata casualmente all'interno di un intervallo di parametri predefiniti, che possono essere modificati dal designer.

Il sistema supporta diverse configurazioni strutturali, tra cui: una struttura centralizzata con un hub principale, una configurazione lineare che collega sequenzialmente i punti di accesso/uscita e una configurazione multi-pivot che introduce più nodi di smistamento interni. Infine, è possibile creare collegamenti aggiuntivi

tra corridoi per introdurre percorsi alternativi e incrementare la complessità della navigazione.

Vengono poi applicati operatori secondari che riempiono alcuni spazi vuoti generati con piattaforme di dimensione casuale, rispettando un certo *padding* (la dimensione e il *padding* sono modificabili e controllabili dal designer). Così facendo, si introduce una variazione verticale nel layout, creando dei campioni che richiedono meccaniche di salto da parte del giocatore.

Di seguito vengono presentati alcuni esempi di layout che sono stati generati e successivamente modificati per essere adattati alle esigenze del designer.

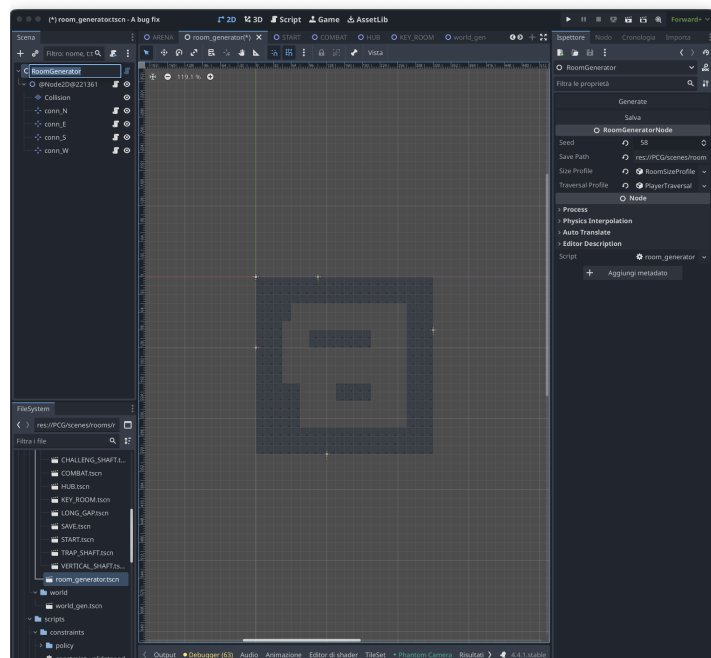


Figura 3.6: Esempio stanza con configurazione lineare.

3.3 – Generazione dei template di stanze (offline)

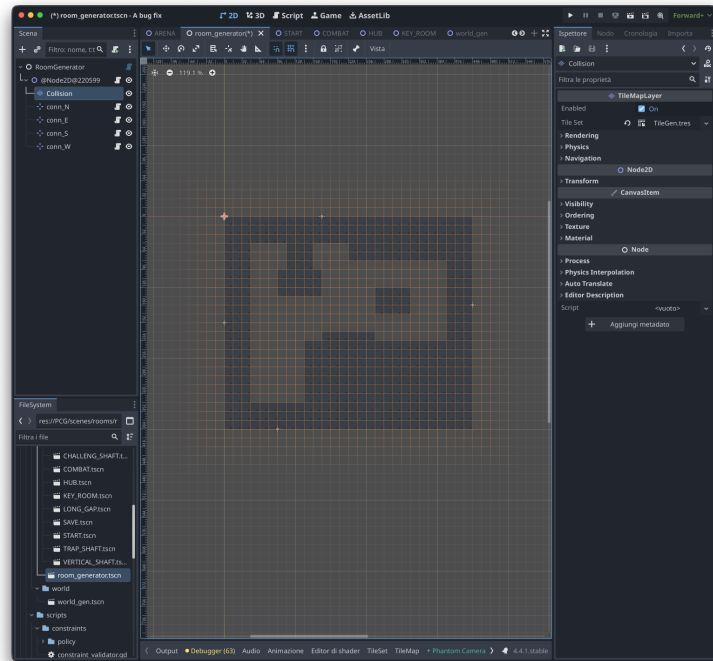


Figura 3.7: Esempio stanza con configurazione Hub.

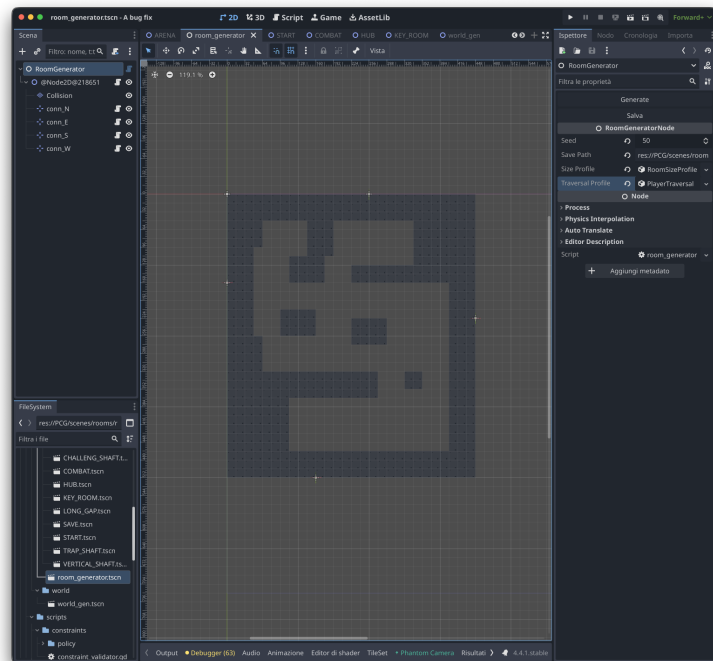


Figura 3.8: Esempio stanza con configurazione Multi-hub.

3.3.2 Ottimizzazione evolutiva

Come precedentemente indicato, la generazione del layout non si limita alla costruzione della singola stanza, bensì si configura come un processo evolutivo volto alla valutazione di diversi genomi (rappresentati dai layout generati) all'interno della popolazione corrente e delle generazioni successive, al fine di individuare il fenotipo che soddisfi le metriche di qualità definite dalle funzioni obiettivo. Inizialmente, viene generata la prima popolazione di genomi con parametri decisi casualmente all'interno dei limiti inferiori e superiori stabiliti dal designer. Ciascun individuo viene validato attraverso un algoritmo di *pathfinding* basato su *Breadth-First Search* (*BFS*). L'algoritmo esplora lo spazio degli stati del giocatore, rappresentato dalla coppia composta dalla posizione nella griglia e dallo stato fisico. Le transizioni tra gli stati sono definite dal modello di movimento del giocatore, che include camminata orizzontale, salto, caduta e il *wall jump* (salto dal muro) che consente al giocatore di scalare i muri. Durante l'esplorazione, l'algoritmo verifica che l'agente che simula il giocatore possa raggiungere tutte le posizioni considerate come terreno calpestabile e tutte le uscite definite. Qualora ciò non si verifichi, il template è considerato invalido e viene scartato; altrimenti, procede al processo di valutazione definito mediante la funzione di *fitness*. La funzione di *fitness* definita verifica le seguenti qualità della stanza:

1. **Densità dello spazio:** valuta la quantità di celle solide rispetto al totale delle celle del layout, prevenendo situazioni con stanze eccessivamente affollate o eccessivamente vuote;
2. **Spazio verticale utile:** misura l'effettivo utilizzo dello spazio verticale, incentivandone l'esplorazione;
3. **Asimmetria strutturale:** calcola il grado di simmetria orizzontale del layout, favorendo layout più asimmetrici al fine di ottenere strutture più variegate;
4. **Irregolarità orizzontale:** questa metrica penalizza segmenti solidi orizzontali prolungati, riducendo così la monotonia geometrica su terreni e muri;
5. **Superfici interne calpestabili:** questa metrica promuove la presenza di piattaforme e terreno calpestabile accessibile al giocatore;

6. **Stratificazione verticale:** questa metrica premia layout che presentano più livelli verticali distinti, incentivando stanze multi-livello e incrementando la complessità spaziale.

A queste metriche strutturali viene aggiunta una componente di *novelty*, che misura la distanza morfologica tra il layout corrente e quelli generati in precedenza. Tale componente ha l'obiettivo di prevenire la convergenza prematura dell'algoritmo evolutivo e favorire la produzione di soluzioni strutturalmente differenziate. Tutti i punteggi ottenuti dalle funzioni di fitness vengono normalizzati. I genomi della popolazione vengono quindi ordinati sulla base dello *score* ottenuto e solo una percentuale dei genomi con *score* più alto vengono selezionati, mentre i restanti genomi vengono scartati. Infine, i genomi sopravvissuti sono sottoposti a riproduzione. In ciascuna iterazione, i discendenti potranno casualmente essere una semplice coppia del genitore oppure verrà eseguito un *crossover* tra il genitore attualmente considerato e uno scelto casualmente tra i sopravvissuti. Ancora casualmente, il nuovo individuo generato viene sottoposto a mutazione (piccole perturbazioni di alcuni parametri) e poi aggiunto alla nuova popolazione di individui. Questo processo viene ripetuto per ciascuna generazione, ottenendo così, al suo termine, l'individuo che secondo le qualità descritte risulta il più adatto.

3.3.3 Intervento manuale e salvataggio

A seguito della fase evolutiva, il genoma con la fitness più elevata viene selezionato per la costruzione del fenotipo vincitore del processo evolutivo. Tale processo prevede l'applicazione di operatori strutturali e secondari, che sfruttano i parametri dell'individuo *fittest*. Successivamente, la *mask* viene convertita nel layout fisico della stanza, trasformando la rappresentazione delle celle solide nella *mask* in *tiles* fisiche della stanza. Il designer, qualora lo ritenga opportuno, potrà apportare modifiche al nuovo layout per ottimizzare il gameplay, intervenendo sulla forma della stanza mediante l'aggiunta o la rimozione di nuove forme geometriche, nonché sulla posizione e la dimensione dei varchi. Al termine di questo processo, il designer specificherà la categoria di stanza a cui apparterrà quella appena generata, le abilità richieste e il livello di difficoltà. Infine, il designer avrà la possibilità di salvare il nuovo template, che verrà utilizzato in combinazione con quelli preesistenti per la transizione dalla rappresentazione astratta mediante grafo al livello fisico.

3.4 Istanziamento del livello fisico

Per tradurre la rappresentazione astratta del grafo nel livello fisico, è necessario definire:

1. un algoritmo che generi coordinate all'interno dello spazio di gioco per l'allocation dei prefabbricati;
2. un algoritmo che trasformi i nodi del grafo in stanze prefabbricate;
3. un algoritmo che costruisca i collegamenti tra le stanze mantenendo la traccia delle relazioni definite dal grafo.

3.4.1 Generazione delle posizioni

Per tradurre il grafo generato in una rappresentazione spaziale, è necessario associare ogni nodo del grafo a una coordinata specifica all'interno di una griglia logica bidimensionale. Tale griglia non rappresenta direttamente la posizione nello spazio di gioco, bensì la disposizione relativa della stanza all'interno della struttura del livello. Questo approccio consente di organizzare il posizionamento dei nodi senza dover considerare immediatamente le dimensioni effettive dei prefabbricati che saranno selezionati successivamente. Il problema in questione può essere interpretato come un processo di *embedding* di un grafo non orientato in una griglia discreta, garantendo la preservazione della connettività tra i nodi e assicurando che ogni stanza occupi una cella distinta.

L'algoritmo risolve questo problema posizionando inizialmente il nodo del grafo, che rappresenta la stanza iniziale, nella cella centrale della griglia. Successivamente, il grafo viene attraversato mediante una visita in ampiezza (*BFS*). Durante tale attraversamento, per ogni nodo già posizionato, vengono analizzati i suoi nodi adiacenti.

Qualora un nodo adiacente non sia ancora stato posizionato, il sistema verifica la disponibilità di celle libere tra le quattro posizioni adiacenti (nord, sud, est, ovest) rispetto alla posizione del nodo corrente. In presenza di più celle disponibili, una di esse viene selezionata casualmente, al fine di introdurre variabilità nella struttura spaziale del livello.

La posizione selezionata viene quindi assegnata al nodo e registrata in una mappa che associa ciascun nodo alla cella occupata nella griglia. Per prevenire sovrapposizioni tra stanze, il sistema mantiene una struttura dati che registra le celle già occupate nella griglia. Tale struttura consente di verificare in tempo reale la disponibilità di una determinata posizione.

Questo approccio assicura un'esecuzione efficiente del processo di assegnazione delle celle, anche per grafi di dimensioni relativamente elevate. Durante la fase di piazzamento, può verificarsi la situazione in cui un nodo del grafo non disponga di celle libere adiacenti ai nodi già posizionati. In tale circostanza, il nodo non può essere collocato immediatamente nella griglia.

Per gestire tale situazione, i nodi in questione vengono temporaneamente inseriti in una lista di nodi rimandati (*deferred nodes*). Una fase successiva dell'algoritmo tenta quindi di assegnare una posizione anche a questi nodi ricercando nuove possibili connessioni con nodi già presenti nella griglia. Al termine della prima passata di piazzamento, l'algoritmo avvia una fase di recupero finalizzata a individuare una posizione alternativa per questi nodi.

La ricerca ha inizio dal nodo che originariamente avrebbe dovuto essere adiacente al nodo non piazzato. L'algoritmo estende progressivamente la ricerca ai nodi già piazzati che risultano vicini a quello corrente. In questo modo, la ricerca si espande radialmente nel grafo, analizzando iterativamente i nodi già posizionati e verificando la presenza di una cella cardinale libera nella griglia.

Non appena viene individuata una posizione disponibile, il nodo rimandato viene collocato in quella cella. Successivamente, la struttura del grafo viene aggiornata introducendo un nuovo collegamento tra il nodo appena piazzato e il nodo utilizzato come ancoraggio per il posizionamento, e vengono eliminate le relazioni precedenti.

Tale operazione consente la modifica localizzata della struttura delle connessioni tra i nodi, generando un percorso alternativo tra il nodo originatore del problema e il nodo che avrebbe dovuto essergli adiacente. Questo approccio previene la perdita di informazione strutturale all'interno del grafo e garantisce la connettività del livello, anche in presenza di vincoli spaziali della griglia che impediscono il posizionamento diretto dei nodi. L'implementazione di questa strategia preserva la connettività complessiva del livello, evitando il fallimento del processo di generazione.

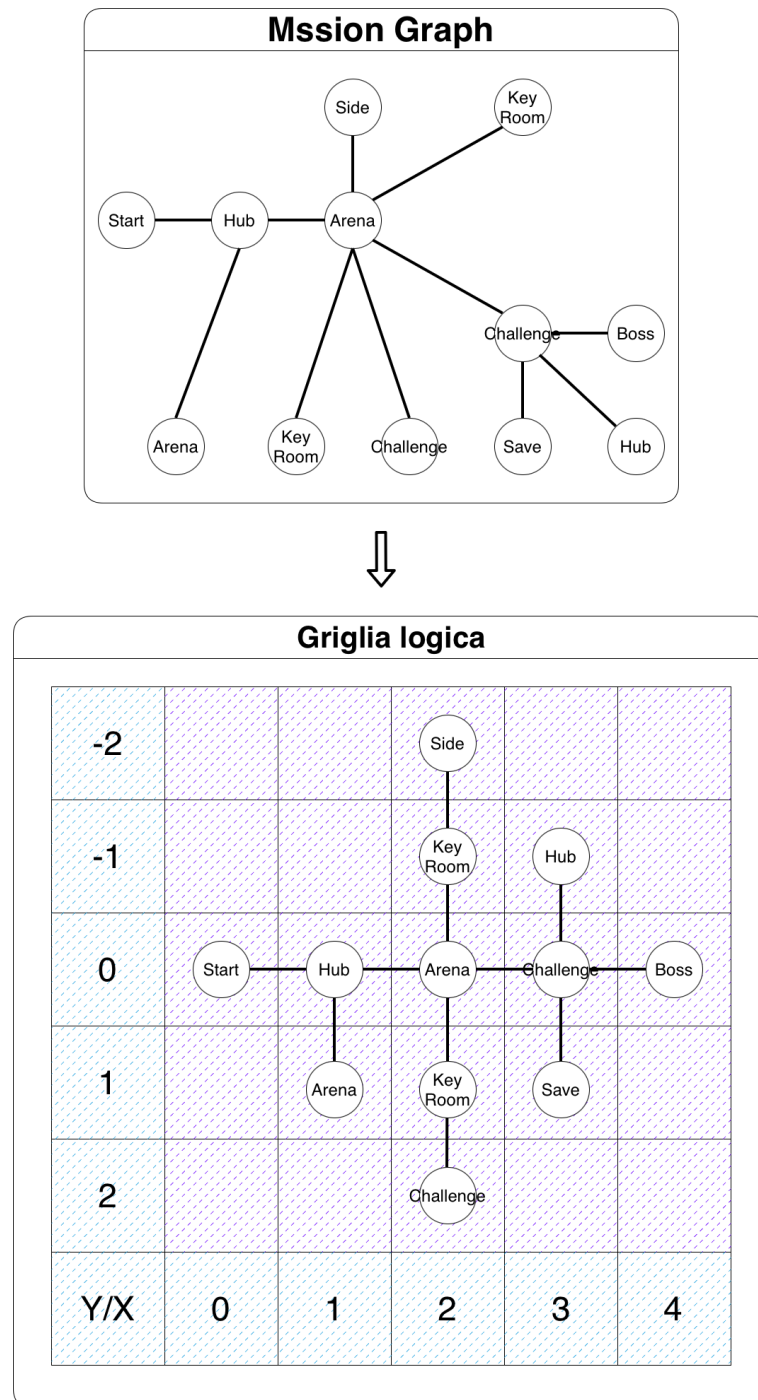


Figura 3.9: Processo di *embedding* del *Mission Graph* nella griglia logica.

Al termine del piazzamento logico, le coordinate della griglia vengono convertite in posizioni nello spazio di gioco. Tale conversione richiede la stima dello spazio

massimo occupabile da una stanza. Il sistema analizza il catalogo dei template disponibili, determinando la larghezza e l'altezza massime tra tutte le stanze definite. A tali dimensioni viene applicato un margine di *padding* per prevenire sovrapposizioni tra stanze adiacenti. La dimensione risultante definisce la dimensione di una cella della griglia nel mondo di gioco. Le coordinate logiche vengono quindi scalate utilizzando tale valore, assicurando che ogni stanza disponga dello spazio necessario per l'istanzamento senza interferenze con le altre.

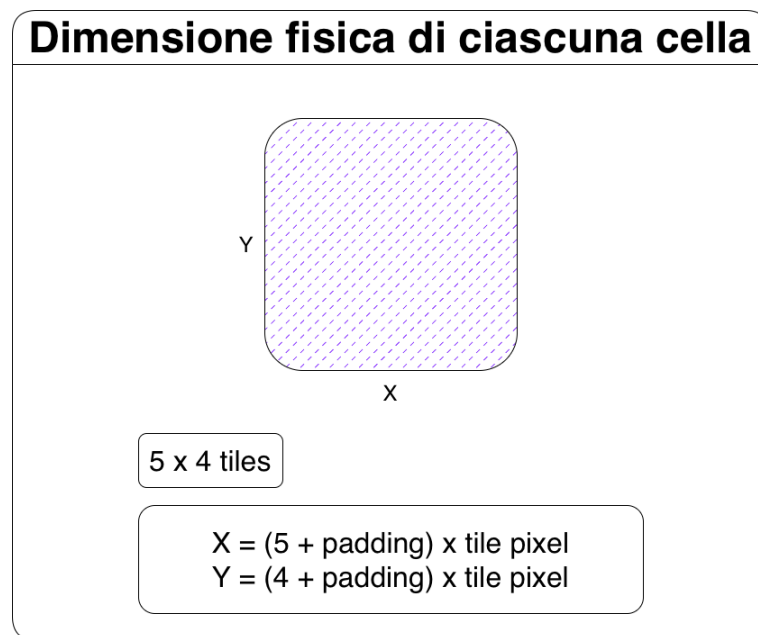


Figura 3.10: Determinazione delle dimensioni effettive di ogni cella della griglia logica destinata all'allocazione dei prefabbricati selezionati.

3.4.2 Scelta dei prefabbricati

La selezione del template più idoneo si realizza mediante un processo gerarchico di valutazione, articolato nei seguenti punti:

1. **Compatibilità Semantica:** il template in esame viene designato quale rappresentazione fisica del nodo qualora i loro tipi coincidano, oppure se nella lista di tipi alternativi che il template può assumere sia presente quello del nodo.
2. **Compatibilità di Abilità:** il template può richiedere diverse competenze per essere completato. La scelta del template è determinata da due fattori

principali: le abilità già possedute dal giocatore e le abilità richieste da un nodo. Per quanto concerne la valutazione delle abilità del giocatore, come specificato alla fine del paragrafo 3.2, il giocatore che accede a una stanza possiede un insieme di abilità. Il template può possedere tutte, alcune o nessuna delle abilità già possedute dal giocatore ed essere selezionato come stanza appropriata. Tuttavia, qualora in questo insieme della stanza sia presente anche una sola abilità non posseduta dal giocatore, tale template viene scartato. Nel caso in cui il nodo richieda una o più abilità, il template selezionato dovrà necessariamente possedere quella specifica competenza (anche se non è posseduta dal giocatore) e potrà opzionalmente includere anche altre competenze secondarie.

3. **Compatibilità dei Connettori:** un connettore rappresenta l'interfaccia attraverso la quale si stabilisce un collegamento tra stanze adiacenti. Ogni stanza può essere equipaggiata da uno o quattro connettori, disposti secondo le direzioni cardinali. Durante il posizionamento dei prefabbricati nello spazio fisico è possibile che la nuova stanza selezionata non disponga di un connettore compatibile con quello della stanza precedentemente posizionata, interrompendo così il collegamento rappresentato dal grafo e determinando una perdita di informazioni. In casi estremi, tale interruzione può provocare un blocco che impedisce al giocatore di progredire nel livello. Per mitigare tale problematica, la selezione del template deve tenere conto sia dei connettori disponibili offerti dal template stesso, sia di quelli richiesti dalla stanza precedentemente posizionata. Per una migliore comprensione di questo meccanismo, si consideri il posizionamento della stanza iniziale, seguito dal posizionamento di una stanza successiva ad est di quest'ultima. In tale scenario, il template selezionato dovrà necessariamente includere il connettore ovest.

Dopo aver applicato i vincoli di ammissibilità, il sistema può ancora disporre di diversi template validi per la generazione della stanza. In questa situazione, è necessario introdurre un criterio di selezione che privilegi i template più adatti al contesto senza compromettere completamente la variabilità del livello generato. Per questo motivo, viene adottato un meccanismo di selezione probabilistica pesata. A ciascun template candidato viene assegnato un punteggio (peso) calcolato sulla

base di diverse euristiche progettuali. La probabilità di selezione di un template è proporzionale al suo peso rispetto agli altri candidati.

Il peso assegnato a ciascun template viene determinato da una funzione di *scoring* che combina diversi fattori progettuali:

- un peso base associato dal designer al template;
- la compatibilità con la difficoltà logica del nodo;
- il grado di copertura dei connettori richiesti dalla struttura della mappa;
- un parametro di diversità finalizzato a prevenire la ripetizione di ruoli analoghi.

Il peso complessivo di un template è determinato mediante la moltiplicazione di tali contributi, con l'obiettivo di penalizzare i template che si discostano significativamente da uno dei criteri valutati.

La selezione finale del template avviene attraverso un'estrazione casuale ponderata, in cui i template con punteggi superiori presentano una probabilità di selezione maggiore, pur non garantendo un esito deterministico. Questo approccio consente di mantenere un equilibrio tra il controllo progettuale e la variabilità procedurale, evitando la generazione di livelli identici pur rispettando i vincoli strutturali e di gameplay.

Per prevenire situazioni in cui l'insieme dei template candidati risulti vuoto, il sistema implementa una fase di *fallback* che allenta progressivamente alcuni vincoli di selezione. In particolare, qualora nessun template soddisfi simultaneamente tutti i criteri di compatibilità, il sistema considera un sottoinsieme più permissivo basato esclusivamente sui requisiti di abilità richiesti dal nodo. Questo approccio garantisce la coerenza minima con la progressione delle abilità del giocatore, evitando il fallimento della generazione.

Al termine del processo decisionale, le stanze vengono posizionate all'interno del mondo di gioco, nelle celle precedentemente definite.

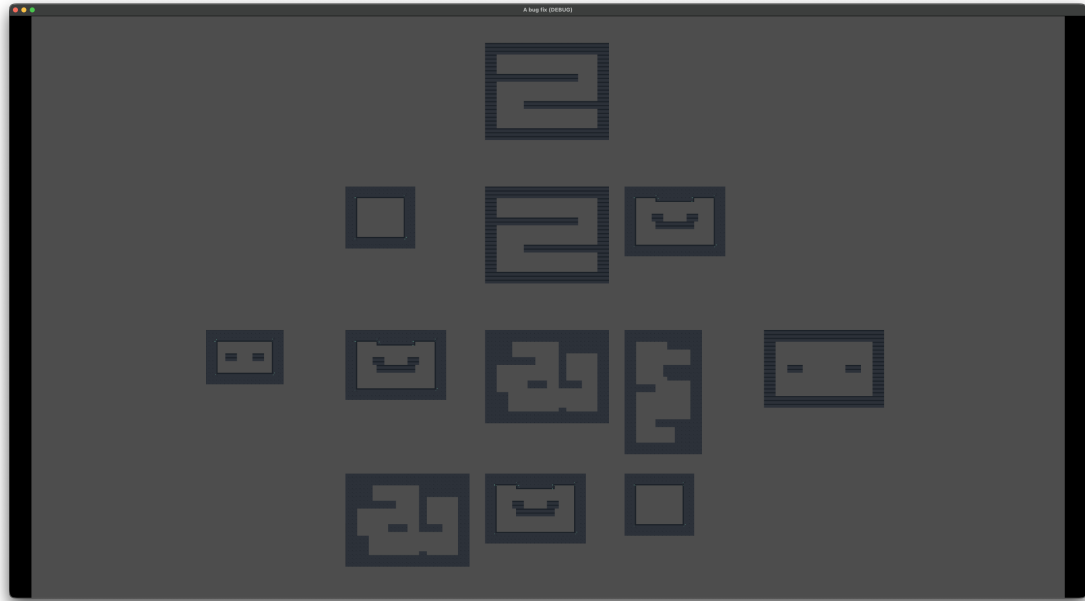


Figura 3.11: Esempio della disposizione delle stanze all'interno dell'ambiente di gioco.

3.4.3 Costruzione dei corridoi

La fase di generazione dei corridoi consiste nella trasformazione degli archi logici definiti nel *Mission Graph* in una configurazione spaziale navigabile. In particolare, tale fase si prefigge di realizzare i collegamenti fisici tra le stanze posizionate nella griglia logica, assicurando la coerenza tra la topologia del grafo e la geometria del livello.

Al termine della fase di posizionamento delle stanze all'interno della griglia logica del livello, si procede alla creazione delle connessioni fisiche che consentono al giocatore di spostarsi tra le diverse aree. Tale operazione implica la trasformazione della struttura topologica definita dal *Mission Graph* in collegamenti spaziali concreti all'interno del livello.

L'approccio adottato prevede la generazione dei corridoi a partire dalle relazioni di adiacenza tra le stanze definite nel grafo. Ogni arco del grafo rappresenta, infatti, una connessione logica tra due stanze che deve essere tradotta in un passaggio navigabile.

Il processo si articola nell'analisi delle posizioni delle stanze nella griglia e nell'identificazione, per ciascuna coppia di stanze adiacenti, della direzione relativa della connessione. In base alla loro disposizione spaziale, è possibile distinguere due casi

principali: connessioni orizzontali, quando le stanze si trovano sulla stessa riga della griglia, e connessioni verticali, quando si trovano sulla stessa colonna.

Al fine di garantire la coerenza tra le diverse configurazioni delle stanze, ogni template di stanza definisce una serie di punti di connessione (i connettori) posizionati sui suoi bordi. Tali punti stabiliscono le direzioni in cui una stanza può essere collegata ad altre e consentono di mantenere la compatibilità tra stanze di dimensioni o forme differenti.

Quando due stanze risultano collegate nel grafo e possiedono punti di connessione compatibili, viene generato un corridoio che unisce tali punti. Il corridoio viene quindi costruito nello spazio compreso tra le due stanze e successivamente vengono aperti i varchi necessari sui bordi delle stanze per rendere il passaggio effettivamente attraversabile.

Il risultato finale è una struttura spaziale in cui tutte le stanze risultano collegate secondo la topologia del grafo, garantendo al contempo la consistenza geometrica dell'ambiente di gioco.

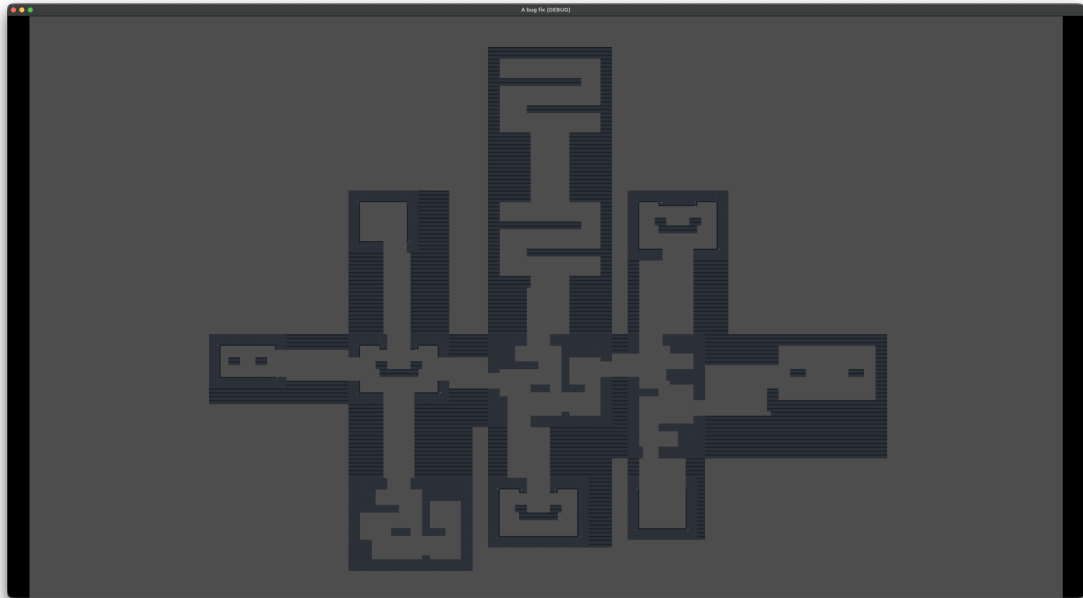


Figura 3.12: Costruzione dei corridoi tra stanze.

3.5 Riempimento e gestione delle stanze

Nell’ambito del processo di generazione del livello, la fase finale prevede il popolamento dello stesso con le entità di gioco. Tali entità comprendono elementi integrativi quali oggetti, raccogliibili, nemici, trappole e decorazioni ambientali, alcuni dei quali vengono specificati nei nodi del grafo generato a monte del processo. Questa fase non è puramente casuale, bensì è regolato da una serie di vincoli geometrici e regole di distribuzione che garantiscono sia la giocabilità sia una distribuzione visivamente plausibile degli elementi.

Ogni stanza gestisce autonomamente le proprie entità, limitate ai confini della stanza stessa. Per ogni stanza vengono eseguite le seguenti operazioni:

1. calcolo delle posizioni in cui inserire le entità di gioco, tenendo conto dei vincoli geometrici e delle regole di distribuzione;
2. posizionamento delle entità nello spazio fisico, garantendo la loro coerenza con l’ambiente circostante;
3. attivazione delle entità in risposta a determinati eventi, al fine di garantire un’interazione fluida e dinamica con il giocatore.

Gli elementi statici, quali trappole e decorazioni, sono già presenti nel livello e non richiedono un evento scatenante specifico per la loro visualizzazione. L’evento scatenante per l’apparizione dei nemici, invece, coincide con l’ingresso del giocatore nella stanza. Il sistema tiene traccia della stanza in cui si trova il giocatore. Se la stanza prevede un combattimento, le porte si chiudono e il combattimento ha inizio; al termine del combattimento scatta un secondo evento cardine che apre le porte e ricompensa il giocatore con oggetti rappresentanti il tesoro della stanza. Una volta sconfitti tutti i nemici, la stanza viene considerata “*sconfitta*” e il giocatore può procedere nel livello.

Inoltre, viene inizializzato il sistema di *autotiling*. Tale sistema ha il compito di determinare automaticamente quale variante grafica di una tile utilizzare in base alle tile adiacenti. In questo modo, è possibile ottenere bordi, angoli e transizioni tra superfici in maniera coerente, senza dover definire manualmente ogni combinazione possibile, garantendo una qualità visiva elevata.



Figura 3.13: Livello finale raggiunto nel processo di traduzione dalla rappresentazione astratta del grafo al livello fisico.

3.5.1 Individuare punti di spawn (Voronoi)

Prima di collocare le diverse tipologie di entità all'interno di una stanza generata proceduralmente, è necessario individuare un insieme di posizioni candidate geometricamente valide. Il sistema sviluppato esegue questa operazione in due fasi principali: l'identificazione delle celle interne alla stanza e la successiva selezione di un sottoinsieme di punti distribuiti nello spazio tramite un approccio ispirato ai diagrammi di Voronoi.

Il primo passo consiste nell'individuare tutte le celle che rappresentano spazio libero all'interno della stanza. Tale operazione viene effettuata tramite un algoritmo di *flood fill* applicato alla griglia della stanza, partendo da una cella di ingresso interna e visitando iterativamente le celle adiacenti che non contengono elementi solidi. Il risultato è un insieme di celle che rappresentano l'area attraversabile della stanza, costituendo la base su cui vengono individuati i possibili punti di *spawn*.

A partire dalle celle interne viene costruita una classificazione di punti di interesse che rappresentano configurazioni geometriche utili per il posizionamento di diversi tipi di entità di gioco. In particolare, vengono individuate le seguenti categorie:

- **Celle pavimento:** celle libere con un elemento solido immediatamente al di sotto. Queste rappresentano i punti naturali per il posizionamento di entità che devono poggiare a terra.
- **Celle soffitto:** celle libere con un elemento solido immediatamente al di sopra. Sono utilizzate per il posizionamento di elementi sospesi o decorazioni.
- **Rientranze nelle pareti:** celle interne adiacenti lateralmente a una parete. Queste configurazioni sono adatte per inserire elementi ambientali o trappole.
- **Pozzi verticali (pit):** sequenze verticali di celle delimitate lateralmente da pareti solide. Quando tali configurazioni raggiungono una profondità minima, possono essere utilizzate come punti per il posizionamento di trappole o elementi ambientali.

Questa classificazione consente di ottenere insiemi distinti di posizioni candidate, che possono essere successivamente utilizzati dai diversi sistemi di *spawning*.

Una volta individuato l'insieme delle posizioni candidate, è necessario selezionare un sottoinsieme di punti che garantisca una distribuzione equilibrata all'interno della stanza. Una selezione puramente casuale, infatti, tenderebbe a generare *cluster* di oggetti eccessivamente ravvicinati. Per ovviare a tale problematica, viene adottato un approccio ispirato ai diagrammi di Voronoi. Il processo si articola in tre fasi:

1. selezione di un insieme di centroidi casuali all'interno delle posizioni candidate;
2. assegnazione di ogni cella candidata al centroide più vicino, determinando la formazione di una serie di regioni spaziali;
3. selezione, all'interno di ciascuna regione, di alcuni punti che massimizzino la distanza reciproca.

Questa metodologia consente di ottenere una distribuzione più omogenea dei punti di *spawn* all'interno dello spazio della stanza, migliorando l'equilibrio visivo dell'ambiente e garantendo una disposizione spaziale più coerente con le dinamiche di gioco. Le diverse categorie di entità (quali decorazioni, trappole, nemici e oggetti)

utilizzano successivamente questi insiemi di punti di interesse applicando specifici criteri di selezione, definiti in base alle caratteristiche intrinseche di ciascun tipo di elemento.

3.5.2 Decorazioni

Le decorazioni sono concepite per migliorare l'estetica visiva degli ambienti generati proceduralmente, contribuendo all'immersione e alla varietà degli spazi senza influire direttamente sulle dinamiche di gioco. Il sistema di posizionamento delle decorazioni è ottimizzato per garantire una distribuzione organica degli elementi, prevenendo sovrapposizioni e assicurando una coerenza con la geometria della stanza.

Il processo di generazione delle decorazioni si articola nelle seguenti fasi: determinazione del *budget* della stanza, selezione delle decorazioni da posizionare, individuazione dei punti di *spawn* e validazione finale del posizionamento.

Il numero di decorazioni inserite in una stanza non è predeterminato, ma è correlato alla dimensione dello spazio disponibile. In particolare, il sistema calcola un *budget* di decorazioni proporzionale all'area interna della stanza, ovvero al numero di celle attraversabili individuate durante la fase di analisi geometrica descritta nella sezione precedente.

Ogni categoria di decorazione (ad esempio, elementi a terra, a parete o a soffitto) dispone di un budget base, che viene incrementato in funzione dell'area della stanza. Tale approccio consente di mantenere una densità visiva coerente tra stanze di dimensioni differenti.

Una volta determinato il *budget* disponibile, il sistema seleziona le decorazioni da istanziare da un catalogo predefinito di elementi. Ogni decorazione è caratterizzata da una serie di parametri, tra cui:

- il tipo di decorazione (terra, parete o soffitto);
- il costo associato alla sua generazione;
- il numero massimo di istanze consentite per stanza;
- un peso probabilistico che determina la frequenza di selezione.

La selezione delle decorazioni avviene in maniera pesata, questo consente di introdurre varietà visiva mantenendo un controllo sulla distribuzione degli elementi.

Per determinare le posizioni di collocamento delle decorazioni vengono utilizzate le categorie di punti di interesse individuate nella fase precedente di analisi della stanza. In particolare, il sistema utilizza differenti insiemi di celle, candidate, queste ultime, a seconda del tipo di decorazione:

- celle di pavimento per le decorazioni a terra;
- rientranze nelle pareti per le decorazioni murali;
- celle di soffitto per gli elementi sospesi.

Le posizioni candidate vengono successivamente filtrate mediante un algoritmo di distribuzione spaziale basato su un approccio Voronoi-like, come descritto nella sezione precedente. Tale passaggio consente di ottenere una disposizione più uniforme degli elementi decorativi, prevenendo concentrazioni eccessive di oggetti nella medesima area della stanza.

Prima di procedere all'istanziatura definitiva di una decorazione, il sistema verifica la validità della posizione selezionata. In particolare, vengono controllati diversi vincoli geometrici, tra cui:

- la compatibilità della posizione con l'ingombro dell'oggetto;
- l'assenza di collisioni con elementi solidi dell'ambiente circostante;
- il rispetto di una distanza minima tra decorazioni adiacenti.

Solo le posizioni che soddisfano tutti questi criteri vengono accettate, garantendo l'integrazione corretta delle decorazioni nella geometria della stanza e l'assenza di sovrapposizioni indesiderate.

Questo approccio consente di ottenere ambienti visivamente più ricchi e variegati, mantenendo al contempo un controllo preciso sulla distribuzione e sulla coerenza spaziale degli elementi decorativi.

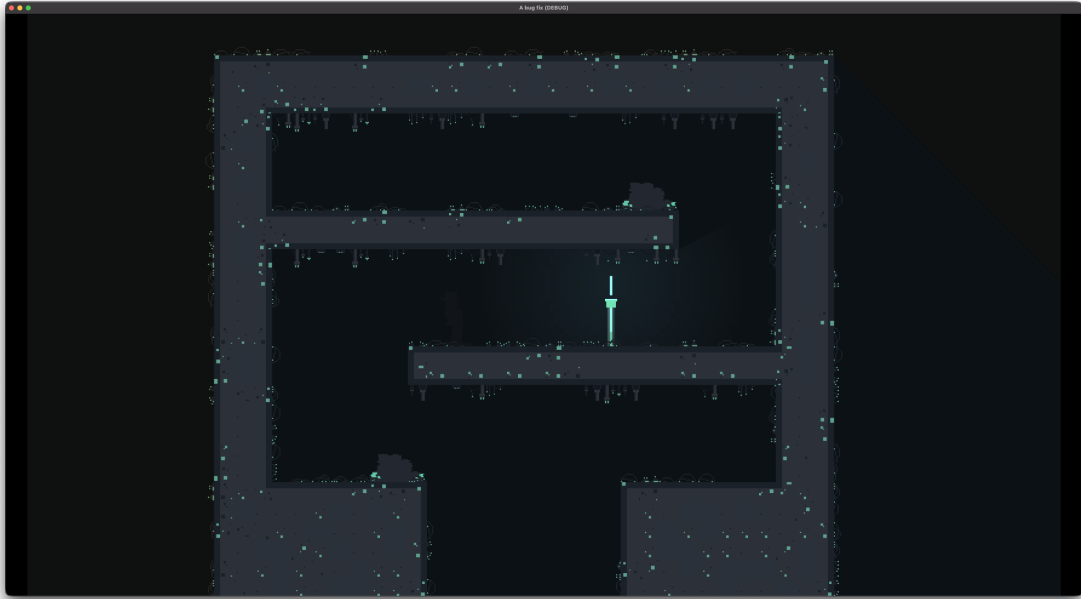


Figura 3.14: Una stanza con delle decorazioni.

3.5.3 Gestione trappole

Le trappole costituiscono un elemento fondamentale per l'introduzione di sfida e varietà all'interno dei livelli generati proceduralmente. A differenza delle decorazioni, le trappole interagiscono direttamente con il giocatore, richiedendo un posizionamento strategico per evitare situazioni che possano compromettere la giocabilità o causare *soft-lock*.

Il sistema di generazione delle trappole si articola in tre fasi principali: la determinazione del *budget* della stanza, la selezione delle tipologie di trappola e l'individuazione dei punti di *spawn* validi.

Il numero di trappole presenti in una stanza non è predeterminato, bensì viene calcolato dinamicamente in base a diversi fattori. In particolare, il *budget* disponibile è influenzato da:

- il livello di progressione della *run*;
- la difficoltà logica della stanza (contenuta nel nodo) all'interno del grafo di missione;
- eventuali direttive specifiche associate al nodo logico della stanza.

Questi parametri vengono combinati mediante una funzione di *scaling* che consente un incremento graduale del numero di trappole nelle fasi avanzate della partita, garantendo al contempo limiti superiori e inferiori per evitare configurazioni eccessive.

Una volta determinato il *budget* disponibile, il sistema procede alla selezione delle trappole da inserire all'interno della stanza da un catalogo predefinito. Ogni trappola è caratterizzata da diversi parametri, tra cui:

- il costo associato alla sua generazione;
- il numero massimo di istanze consentite per stanza;
- una finestra di difficoltà entro la quale la trappola può comparire;
- un peso probabilistico che determina la frequenza di selezione (questo parametro consente di ampliare la gamma di opzioni disponibili nella selezione delle trappole).

La selezione avviene tramite un meccanismo di scelta pesata. Questo favorisce variabilità tra le trappole individuate per una specifica difficoltà.

Le posizioni candidate per il posizionamento delle trappole vengono determinate mediante un'analisi geometrica approfondita della stanza. Il sistema identifica specifiche categorie di punti caratteristici dell'ambiente, quali:

- pozzi o dislivelli del terreno;
- rientranze presenti nelle pareti;
- celle interne della stanza potenzialmente percorribili.

Tali posizioni costituiscono i candidati iniziali, successivamente sottoposti a un processo di filtraggio per assicurarne la validità di posizionamento.

Considerando la potenziale interferenza delle trappole con la navigabilità dello spazio di gioco, il sistema implementa una politica di protezione per prevenire configurazioni che possano ostacolare il movimento del giocatore. In particolare, viene stimata la capacità di movimento del personaggio (ad esempio l'altezza massima raggiungibile con un salto) e vengono individuate le celle che devono rimanere libere per garantire l'accesso alle diverse aree della stanza.

Le posizioni candidate che interferiscono con queste regioni protette vengono automaticamente eliminate. Tale approccio consente di preservare la percorribilità dello spazio mantenendo, al contempo, una distribuzione significativa degli ostacoli.

A seguito della fase di filtraggio, i punti di *spawn* rimanenti vengono ordinati e valutati attraverso una funzione di *scoring* topologico che considera la struttura della stanza. Questo processo consente di privilegiare, ad esempio, posizioni situate in corridoi stretti, in prossimità di pareti o in aree di passaggio strategiche.

Successivamente, viene applicato un ulteriore passaggio di distribuzione spaziale, volto a prevenire concentrazioni eccessive di trappole nella stessa area della stanza e a garantire una disposizione più equilibrata.

Infine, le trappole selezionate vengono istanziate nelle celle di *spawn* individuate.

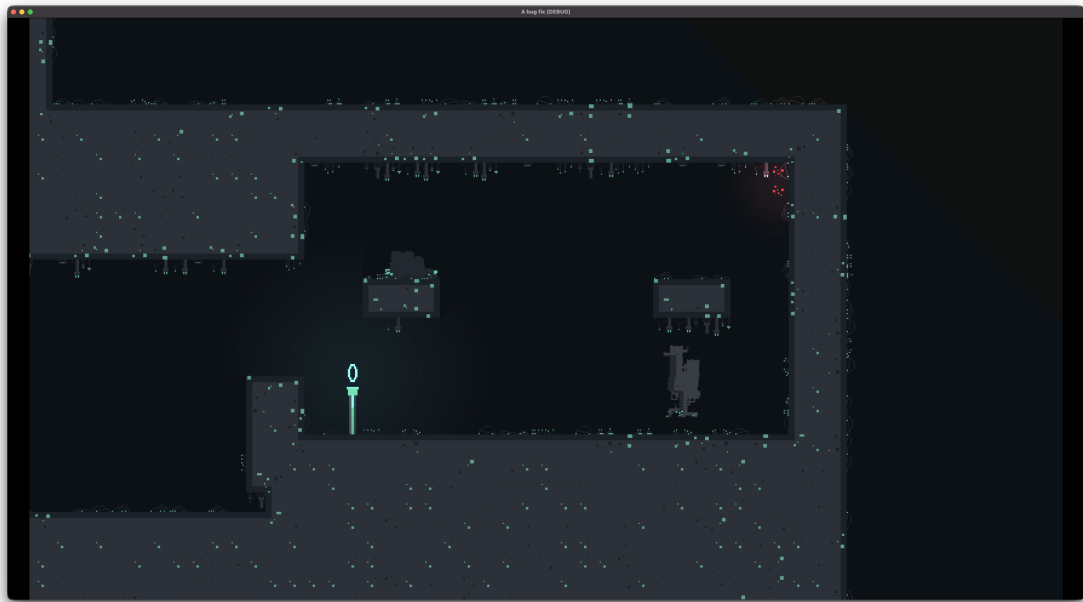


Figura 3.15: Stanza con trappole.

3.5.4 Gestione nemici

I nemici rappresentano l'elemento principale attraverso cui viene definita la difficoltà delle stanze generate proceduralmente. A differenza di trappole e decorazioni, i nemici sono entità dinamiche in grado di muoversi liberamente nello spazio di gioco. Le celle vengono quindi utilizzate esclusivamente come punti di spawn iniziali,

mentre il comportamento successivo è gestito dal sistema di intelligenza artificiale delle singole entità ostili.

Il sistema di generazione dei nemici è strutturato in quattro fasi principali: determinazione del *budget* della stanza, selezione dei tipi di nemico, individuazione dei punti di *spawn* e istanziamento finale delle entità.

Il numero di nemici presenti in una stanza viene determinato dinamicamente attraverso un *budget* che dipende da diversi fattori legati alla progressione del gioco. In particolare, il *budget* è funzione di:

- livello di avanzamento della *run*;
- difficoltà logica della stanza (contenuto nel nodo) nel grafo di missione;
- eventuali moltiplicatori associati alle direttive del nodo logico.

Questi parametri vengono combinati attraverso una funzione di *scaling* che permette di aumentare gradualmente il numero di nemici nelle fasi avanzate della partita. Il valore finale viene poi limitato entro un intervallo definito, in modo da evitare configurazioni eccessivamente dense o troppo povere di avversari.

Una volta determinato il *budget* disponibile, il sistema seleziona i tipi di nemico da generare utilizzando un catalogo di entità predefinite. Ogni tipo di nemico è caratterizzato da una serie di parametri, tra cui:

- il costo associato alla sua generazione;
- un peso probabilistico che ne regola la frequenza di apparizione;
- un intervallo di difficoltà entro cui il nemico può comparire;
- il numero massimo di istanze consentite per stanza.

La selezione avviene tramite un meccanismo di scelta pesata. Questo favorisce variabilità nella composizione degli incontri.

Le posizioni candidate per lo spawn dei nemici vengono individuate a partire dalle celle di pavimento attraversabili all'interno della stanza. Queste celle rappresentano le aree in cui un'entità mobile può essere inizialmente collocata senza generare collisioni immediate con la geometria dell'ambiente. Per evitare una distribuzione troppo concentrata, le celle candidate vengono riorganizzate tramite una

procedura di distribuzione spaziale che tende a separare i punti di *spawn* tra loro. Questo passaggio permette di ottenere configurazioni più leggibili e bilanciate, evitando situazioni in cui numerosi nemici compaiono nello stesso punto della stanza. Durante la validazione delle posizioni viene inoltre imposto un vincolo di distanza minima tra i nemici già posizionati e i nuovi punti di *spawn*. Un controllo analogo viene applicato anche rispetto ai connettori della stanza, in modo da evitare che il giocatore incontri immediatamente un nemico appena attraversato un ingresso. Una volta determinati i tipi di nemico e le posizioni di *spawn* disponibili, le entità vengono istanziate all'interno della stanza.

Poiché i nemici sono agenti mobili, la validazione delle posizioni viene eseguita solo nella fase iniziale di spawn. Dopo l'istanziatura, il comportamento degli agenti è interamente gestito dal sistema di movimento e dall'intelligenza artificiale, che permettono ai nemici di esplorare liberamente lo spazio della stanza.

Questo approccio consente di generare incontri variabili e dinamici, mantenendo al contempo un controllo sulla densità e sulla distribuzione spaziale degli avversari.



Figura 3.16: Stanza in cui sono presenti dei nemici.

3.5.5 Gestione oggetti

Gli oggetti costituiscono gli elementi interattivi con cui il giocatore interagisce durante l'esplorazione del livello, quali risorse, chiavi o punti di salvataggio. Il loro posizionamento deve rispettare vincoli geometrici, evitando sovrapposizioni con altri elementi e garantendo la corretta accessibilità da parte del giocatore.

Il sistema di generazione degli oggetti si articola in tre fasi principali: costruzione del catalogo di oggetti da generare, individuazione delle posizioni candidate e istanziamento finale all'interno della stanza.

Gli oggetti da inserire all'interno della stanza vengono determinati a partire dal nodo logico. Ogni nodo può definire un catalogo di oggetti che descrive le possibili ricompense o elementi interattivi associati a quella stanza.

Ogni oggetto è caratterizzato da una serie di parametri, tra cui:

- una quantità massima generabile;
- una probabilità di comparsa;
- un livello di priorità.

Gli oggetti con priorità obbligatoria vengono sempre generati nella quantità specificata, mentre gli altri vengono selezionati in modo probabilistico fino al raggiungimento della quantità massima prevista. Il risultato di questa fase è una mappa che associa a ciascun tipo di oggetto il numero di istanze da generare.

Le posizioni candidate per il posizionamento degli oggetti vengono individuate a partire dalle celle di pavimento attraversabili della stanza. Queste celle rappresentano le aree in cui un oggetto può essere collocato senza interferire con la geometria dell'ambiente.

Al fine di prevenire una distribuzione eccessivamente irregolare, le celle candidate vengono riorganizzate mediante una procedura di distribuzione spaziale che tende a massimizzare la separazione tra i punti selezionati. Tale processo consente di ottenere una disposizione più uniforme degli oggetti all'interno della stanza, dunque una maggiore fruibilità dello spazio di gioco.

Prima di procedere all'istanza di un oggetto, il sistema verifica la validità della posizione scelta attraverso un rigoroso controllo di diversi vincoli, tra cui:

- la compatibilità con la geometria della stanza, garantendo che l'oggetto si inserisca correttamente all'interno dei limiti definiti;
- l'assenza di collisioni con elementi già presenti, prevenendo sovrapposizioni indesiderate e garantendo l'integrità della struttura spaziale;
- il rispetto di una distanza minima dagli altri oggetti e dagli elementi statici (trappole e decorazioni), assicurando una distribuzione spaziale equilibrata e una visuale ottimale per il giocatore.

Considerando le diverse dimensioni degli oggetti, il sistema valuta anche l'ingombro dell'oggetto in termini di celle occupate. Una volta individuata una posizione valida, le celle corrispondenti vengono riservate per impedire che altri sistemi di generazione utilizzino la stessa area, garantendo l'esclusività della posizione per l'oggetto istanziato.

Capitolo 4

A Bug Fix

4.1 Descrizione

A Bug Fix è un *action-platform* 2D con visuale laterale ambientato all'interno di un sistema operativo di un elaboratore. Il giocatore interpreterà un processo software dotato di specifici obiettivi e caratteristiche che ne determineranno le meccaniche e il gameplay. Il progetto consiste in un prototipo del gioco completo realizzato per l'esame di Sviluppo di Videogiochi nell'anno accademico 2024-2025.

4.2 Personaggi

Il gioco è caratterizzato da tre categorie di personaggi, ciascuno distinto da obiettivi, abilità e background specifici:

1. **Il personaggio principale:** esso è controllato dal giocatore, rappresenta un malware informatico che ha infettato il sistema operativo di un computer. Gli obiettivi assegnati da un personaggio non giocante (NPC) includono l'eliminazione dei processi dell'antivirus che ne impediscono il progresso, la corruzione di file chiave del sistema operativo e, infine, l'acquisizione del controllo totale del computer.
2. **NPC:** rappresenta un altro processo dello stesso malware che è riuscito a penetrare nel sistema. Fornisce informazioni cruciali all'utente per la comprensione della narrazione, degli obiettivi e di alcune meccaniche di gioco.



Figura 4.1: Personaggio principale.



Figura 4.2: NPC.

3. **Avversari:** rappresentano un ostacolo al raggiungimento degli obiettivi del giocatore. Si tratta dei processi dell'antivirus attivati in risposta all'infezione da malware. Si distinguono i *Minion*, che si spostano lungo la mappa e attaccano in mischia ravvicinata, e i *Gunner*, che occupano posizioni strategiche e utilizzano attacchi a distanza.



Figura 4.3: *Minion*.

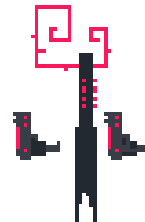


Figura 4.4: *Gunner*.

4.3 Storia

La narrazione si svolge all'interno di un sistema informatico. Il protagonista è un software malevolo introdotto nel sistema a seguito dell'esecuzione di un'applicazione che ha stabilito una connessione tra il computer bersaglio e un endpoint. Dopo l'avvio del software malevolo, l'antivirus del sistema minacciato verrà attivato per

tentare di proteggerlo, contrastare l'avanzata dell'applicativo malevolo attraverso l'invio di processi ostili e la generazione di strutture labirintiche per intrappolare il giocatore. L'utente non scoprirà la propria identità di software malevolo fino all'ultimo livello. Le informazioni fornite per la comprensione della storia sono volutamente ambigue, ma sono presenti indizi che suggeriscono all'utente che stia compiendo azioni dannose (ad esempio, completando gli obiettivi, l'utente noterà come il sistema presenterà diversi glitch visivi nei livelli successivi). Al completamento di tutti gli obiettivi, l'utente acquisirà il controllo dell'intero sistema, fondendosi con esso.

4.3.1 Tema

Il tema centrale del gioco ruota attorno alla percezione ingannevole e ambigua delle azioni del giocatore, che conducono al raggiungimento degli obiettivi assegnati al protagonista. L'obiettivo iniziale del gioco è fornire informazioni ambigue sulla situazione, suggerendo l'infiltrazione di un malware nel sistema e la conseguente mobilitazione dell'antivirus per contrastarlo. Non viene, tuttavia, specificata la fazione di appartenenza del personaggio giocante, inducendo il giocatore a interrogarsi sulla natura delle proprie azioni e a cogliere dettagli che possano rivelare la sua affiliazione. Il colpo di scena finale elimina ogni dubbio, mostrando le conseguenze delle azioni del protagonista, che conquista il sistema e ne diventa parte integrante.

4.4 Trama

L'esperienza di gioco ha inizio quando un utente, operante nel mondo reale, avvia il proprio computer e procede all'esecuzione di un file eseguibile. Tale file viene temporaneamente interrotto da un messaggio di errore generato dal *software antivirus*, che segnala la rilevazione di *malware* e l'attivazione del *Defender*. In seguito, il protagonista viene trasportato da una nuvoletta, metafora del *cloud*, all'interno del sistema vittima. In questo contesto, l'utente si imbatte in un personaggio non giocante (NPC) che fornisce una spiegazione preliminare della situazione. L'identità dell'NPC rimane avvolta nel mistero, generando un'ambiguità che stimola la curiosità del giocatore, spingendolo a riflettere sulla propria posizione all'interno

della narrazione, in particolare se stia interpretando il ruolo del virus o dell'antivirus. Il giocatore si muove attraverso diverse directory, risalendo il file system del sistema operativo. Durante i vari livelli, l'NPC offre informazioni preziose sulle capacità del giocatore e sulla narrazione, delineando al contempo obiettivi specifici che il giocatore deve raggiungere per progredire nel gioco. Nei livelli iniziali, l'NPC richiede al protagonista di "eliminare" i processi che ostacolano il passaggio all'area successiva, consentendo così l'escalation dei privilegi di sistema e la riduzione di quelli dei processi avversari. In seguito, il giocatore dovrà "allineare i file" con il sistema interno del *malware* da cui il giocatore e l'NPC hanno origine. Tale allineamento implica la modifica e la corruzione di file di sistema essenziali. In risposta a tali attacchi, il sistema operativo tenterà un'ultima e disperata manovra di difesa, cercando di intrappolare definitivamente il giocatore all'interno di labirinti generati proceduralmente. Il giocatore dovrà quindi navigare attraverso i livelli, acquisendo potenziamenti per raggiungere il nucleo centrale del sistema. Giunto a questo punto, l'NPC richiede la corruzione di un ultimo file rimanente, rivelando i veri obiettivi della loro operazione: la completa conquista del sistema. Il gioco si conclude con una scena finale in cui il protagonista si fonde con il sistema e lo controlla.

4.5 Gameplay

La struttura di una sessione di gioco si origina dal menù principale, che presenta inizialmente al giocatore due opzioni: l'avvio della partita o la chiusura del gioco. A seguito dell'avvio della partita, il giocatore accede alla scena introduttiva, al termine della quale assume il controllo operativo del personaggio. Il suo compito consiste nel progredire attraverso i diversi livelli, completando gli obiettivi prefissati per sbloccare l'accesso all'area successiva.

4.5.1 Obiettivi

Per il completamento del gioco è necessario conseguire gli obiettivi specifici assegnati a ciascun livello. Di seguito si riporta la descrizione dettagliata degli obiettivi per livello:

- **Livello 1:** eliminare tutti gli avversari presenti nell'area di gioco per ottenere l'accesso all'area successiva e testare le meccaniche del sistema di combattimento;
- **Livello 2:** allineare i file presenti nel livello al sistema interno del giocatore, un'espressione ambigua che implica la loro corruzione, per ottenere l'accesso all'area successiva;
- **Livello 3:** uscire dai labirinti creati dal sistema operativo;
- **Livello 4:** corrompere il file di sistema, assumendo così il controllo totale.

L'obiettivo del gioco è stimolare la curiosità del giocatore e instillare in lui il dubbio che le sue azioni non siano benevole per il sistema, inducendolo a interrogarsi sulla vera natura del protagonista e a comprendere infine la sua posizione.

4.5.2 Abilità del giocatore

Le abilità del personaggio giocante possono essere suddivise in due categorie principali:

- **Combattimento:** questo sistema si basa su una sequenza di attacchi e contrattacchi che, se eseguiti con precisione, consentono di sbloccare un attacco unico;
- **Movimento:** oltre ai movimenti di base e al salto, è disponibile la funzione “*dush*”, un breve scatto che rende temporaneamente invulnerabile il protagonista, permettendogli di eludere attacchi nemici, proiettili e trappole. Inoltre, il personaggio è in grado di eseguire il “*wall jump*” e lo “*slide*” sui muri, consentendogli di muoversi verticalmente. Infine, come power-up è previsto il “*double jump*”.

4.5.3 Meccaniche di gioco

Combat System Durante il gameplay, il giocatore si troverà a confrontarsi con due categorie di avversari: i *Minion* e i *Gunner*. Queste entità sono assegnate a specifiche aree della mappa con l'intento di ostacolare il progresso del giocatore.

I *Minion* pattuglieranno diligentemente le loro zone di competenza e, qualora il giocatore si avvicinasse alla loro portata, verranno allertati, dando inizio a un combattimento corpo a corpo.



Figura 4.5: Un *Minion* che presidia una zona.

Durante tale scontro, il giocatore avrà a disposizione le seguenti azioni:

- **Attacco base:** l'attacco è una manovra, eseguibile sia in aria che a terra, che infligge un'unità di danno all'avversario;



Figura 4.6: Il giocatore attacca il *Minion* con un attacco base.

- **Contrattacco:** il contrattacco può essere eseguito esclusivamente quando il giocatore si trova a terra, se eseguito con successo, interrompe l'attacco nemico in corso e consente al giocatore di effettuare un attacco speciale;



Figura 4.7: Il giocatore esegue un controattacco con successo.

- **Attacco Speciale:** l'attacco speciale può essere eseguito unicamente dopo un contrattacco eseguito con successo, tale attacco consente di eliminare definitivamente l'avversario.



Figura 4.8: Il giocatore esegue l'attacco speciale.

- **Schivata:** la schivata può essere attivata sia durante il volo del giocatore sia a terra, conferendo temporaneamente invulnerabilità agli attacchi nemici.

I nemici, a loro volta, sono in grado di attaccare il giocatore infliggendo un punto di danno.

Movements Oltre a fornire i comandi direzionali, il giocatore avrà la possibilità di eseguire due azioni speciali che ampliano il parco movimento del personaggio.

- **Dush/Schivata:** un breve scatto eseguibile sia in aria che a terra. Consente al giocatore di superare ostacoli, trappole, evitare attacchi nemici e incrementare le distanze percorse durante il salto.
- **Slide:** durante la caduta del personaggio, se il giocatore lo dirige verso una superficie verticale, questo inizierà a scivolare lungo la parete, rallentando la caduta. La velocità della scivolata è regolabile tramite un comando relativo impartito dal giocatore.
- **Wall jump:** i *wall jump* è un'abilità che si attiva quando il giocatore esegue un salto in direzione di una superficie verticale (dopo l'attivazione dello *slide*). Ripremendo il tasto del salto, il giocatore può staccarsi dalla superficie, eseguendo un salto aggiuntivo e sfruttando la superficie come punto di appoggio per tale azione. Questa abilità consente di ampliare lo spazio di movimento anche sull'asse verticale.
- **Double jump:** il doppio salto è un power-up ottenuto durante la progressione del gioco. Tale potenziamento consente al giocatore di raggiungere altezze superiori, agevolando l'esplorazione del gioco.

Le azioni che consentono di dominare il gioco, e che quindi innescano la *mastery*, sono il **contrattacco** e il **dush**. Tali azioni, se correttamente utilizzate, consentono di velocizzare notevolmente il ritmo di gioco, appagando così l'esperienza del giocatore.

4.5.4 Oggetti

Durante l'interazione con il gioco, il giocatore avrà la possibilità di interagire con diversi oggetti, ciascuno dei quali svolge una funzione specifica:

1. **Celle di energia:** queste celle vengono raccolte dal giocatore per ripristinare i punti vita persi durante le fasi di combattimento e *platforming*.



Figura 4.9: Cella di energia.

2. **Backdoor:** le backdoor fungono da *checkpoint*. Il giocatore dovrà semplicemente attraversarle per attivarle. In caso di sconfitta, il giocatore verrà riprodotto in corrispondenza del *checkpoint* attivato. A livello informatico, una backdoor è un punto di accesso nascosto e non autorizzato a un sistema informatico che consente di bypassare le normali procedure di sicurezza. Questa caratteristica è stata scelta per rappresentare i *checkpoint* del protagonista, essendo esso un *malware*.

Figura 4.10: *Backdoor: checkpoint*.

3. **Directory:** le directory rappresentano i punti di transizione tra i diversi livelli di gioco. Vengono sbloccate al completamento di specifici obiettivi.

Figura 4.11: *Directory*: cambio di livello.

4. **File da Corrompere:** quando il personaggio si trova su tali file, verrà richiesto di interagire con essi. Questi file sono necessari per il completamento di alcuni obiettivi.

Figura 4.12: *File* corrotto.

5. **Chiavi di Accesso:** queste chiavi sono utilizzate per aprire porte o oggetti bloccati.



Figura 4.13: Chiave di accesso.

4.6 Progressione

La progressione del gioco si sviluppa attraverso una struttura articolata in più fasi. Nella fase iniziale, il giocatore affronta sfide necessarie ad apprendere le meccaniche principali di gioco. Il livello di difficoltà segue un andamento lineare, incrementando gradualmente il numero di nemici, trappole e ostacoli progettati. Nelle fasi avanzate del gioco, il giocatore si trova ad esplorare livelli labirintici di natura *dungeon-like*. All'interno di tali livelli, il giocatore avrà l'opportunità di acquisire abilità e oggetti essenziali per il proseguimento dell'esperienza di gioco, affrontando nemici e trappole la cui quantità e difficoltà dipendono da:

- valore della run, che rappresenta una difficoltà progressiva;
- difficoltà di ciascun nodo definito nella fase progettuale;
- moltiplicatori applicati alle difficoltà.

Questi fattori influenzano alcuni elementi, quali: il numero di nemici e il loro tipo, nonché il numero di trappole e il loro tipo. Al completamento di ciascun livello procedurale, il giocatore dovrà affrontare un *boss*, che rappresenta un ostacolo significativo da superare per accedere al livello successivo.

4.7 Sconfitta

Il giocatore subisce una sconfitta da parte del gioco qualora i suoi punti vita si riducano a zero, evento che può verificarsi a seguito dell'interazione con nemici o trappole. In caso di sconfitta del personaggio, esso viene ripristinato all'ultimo checkpoint visitato. Nei livelli procedurali, inoltre, le stanze che determinano la sconfitta del giocatore vengono ripristinate al loro stato iniziale, richiedendo al

giocatore di sconfiggere nuovamente i nemici precedentemente eliminati e quelli che lo hanno sconfitto nella medesima stanza.

4.8 Art Style

Il gioco presenta una grafica *pixel-art* a 32/64 bit. L'obiettivo principale è stato quello di sviluppare uno stile grafico minimalista, caratterizzato da una palette cromatica limitata e da pochi colori predominanti. Gli *asset* grafici originali sono stati realizzati da *Hugues Laborde*, un utente della piattaforma web *itch.io*, e ulteriori *asset* sono stati creati ispirandosi al suo stile.

4.9 Musica e Suono

Il progetto attualmente incorpora tracce sonore *placeholder* generate tramite l'applicazione online *Suno* mediante l'utilizzo di prompt specifici. Tali tracce presentano una particolare sonorità retro elettronica a 8-bit. I suoni associati alle azioni dei personaggi, dei nemici e degli oggetti sono stati invece realizzati da *Helton Yan* e si possono scaricare dalla piattaforma web *itch.io*.

Capitolo 5

Implementazione

Il presente capitolo si propone di fornire una descrizione dettagliata dell'architettura e delle specifiche implementazioni dei concetti presentati nel capitolo 3 all'interno del motore di gioco *Godot 4*, con particolare attenzione alla generazione procedurale dei livelli per il gioco descritto nel capitolo 4.

L'architettura della fase di generazione del livello si articola in tre fasi distinte: la fase di generazione del grafo e la sua relativa solvibilità, la fase di traduzione del grafo logico nel livello fisico e, infine, la gestione delle stanze del livello durante l'intera sessione di gioco.

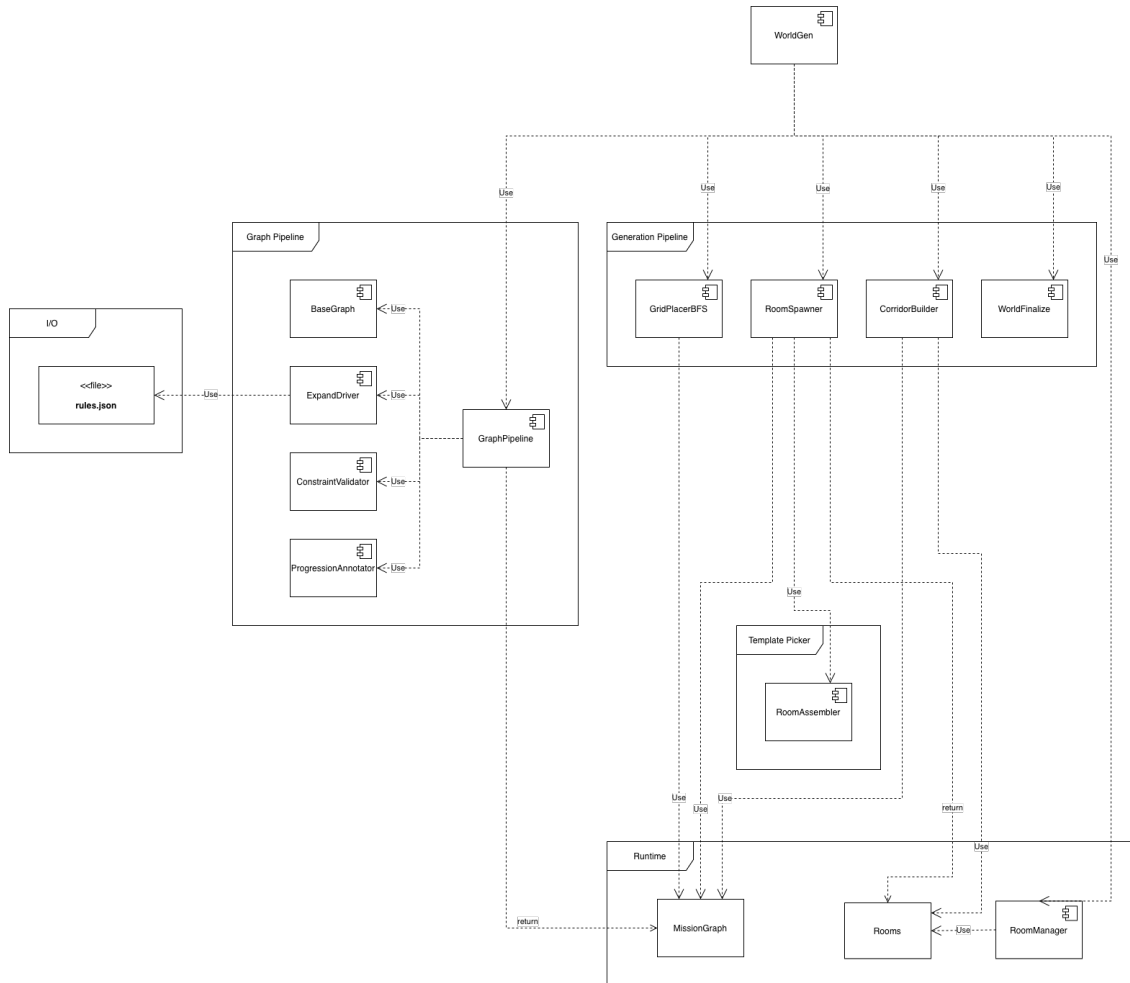


Figura 5.1: Diagramma a blocchi del sistema di generazione.

5.1 Moduli - Generazione dei livelli

5.1.1 WorldGen

Il modulo `WorldGen` opera come classe orchestratrice, coordinando e invocando i moduli subordinati incaricati della risoluzione della classe dei sottoproblemi identificati. Tale processo consente di ottenere un output valido per il problema superiore, ovvero la generazione procedurale di un livello.

Questo modulo espone nell'editor di gioco quattro parametri configurabili dal *designer*:

1. `budget_nodes`: rappresenta un vincolo imposto sul numero di nodi del grafo durante l'applicazione della grammatica generativa;
2. `seed`: un valore intero utilizzato per inizializzare il generatore pseudocasuale, garantendo la riproducibilità del livello.
3. `run_level`: un numero intero utilizzato per rappresentare la difficoltà complessiva del livello;
4. `player_scene`: la scena del *player* che dovrà essere istanziato al termine della generazione procedurale.

Inoltre, all'interno del modulo viene definita la lista di abilità possedute dal giocatore all'inizio del livello, in modo che la selezione dei *template* durante la fase di traduzione avvenga sulla base di queste abilità iniziali e di quelle acquisite durante il livello.

La funzione che avvia il processo di generazione del livello esposta dal presente modulo è la funzione `build`. Essa utilizza i seguenti moduli:

- `GraphPipeline`: generazione del grafo del livello e verifica della sua solvibilità;
- `GridPlacerBFS`: sistema di *embedding* del grafo in una griglia logica;
- `RoomAssembler`: sistema di selezione dei template delle stanze;

- **RoomSpawner**: istanziamiento all'interno del mondo di gioco delle stanze scelte, rispettando i vincoli di adiacenza della griglia logica e degli archi del grafo;
- **CorridorBuilder**: creazione dei corridoi per il transito tra le stanze;
- **WorldFinalize**: rifiniture finali del livello generato;
- **RoomsManager**: modulo utilizzato per gestire gli eventi all'interno delle stanze.

Listing 5.1: Pseudocodice della procedura `build`

```
1 def build() -> void:
2     pipeline: GraphPipeline.Result = GraphPipeline.run()
3     #Se la missione non è solvibile esegui un'altra generazione con un nuovo
4     ↪ seed
5     if not pipeline.solvable:
6         reload_current_scene()
7
8     graph = pipeline.graph
9
10    #-----
11    # Embedding del grafo nella griglia logica
12    placer_result: GridPlacerBFS.Result = GridPlacerBFS.place(graph)
13    #-----
14
15    #-----
16    # Scelta e istanziamento delle stanze
17    assembler: RoomAssembler = RoomAssembler.new()
18    spawner: RoomSpawner = RoomSpawner.new(graph, placer_result, assembler)
19    rooms = spawner.spawn_all()
20    #-----
21
22    #-----
23    # Costruisci i corridoi tra le stanze
24    CorridorBuilder.connect_adjacent(graph, placer_result, rooms)
25    #-----
26
27    #-----
28    # Applica il sistema di autotailing e istanzia il giocatore
29    WorldFinalize.merge_and_spawn()
30    #-----
31
32    #-----
33    # Aggiungi il manager alla scena per gestire gli eventi delle stanze
34    add_child(RoomsManager.new(graph, placer_result))
35    #-----
```

5.1.2 GraphPipeline

Il modulo in oggetto si occupa della definizione e dell'espansione del grafo di base rappresentante il livello, della validazione della sua solvibilità e dell'annotazione delle abilità necessarie per la selezione dei template nelle fasi successive. Tale processo culmina nella generazione della struttura finale, che fungerà da riferimento

primario per la creazione del livello fisico e dei suoi contenuti. Di seguito vengono descritti in dettaglio i moduli impiegati per affrontare i sottoproblemi sopra menzionati.

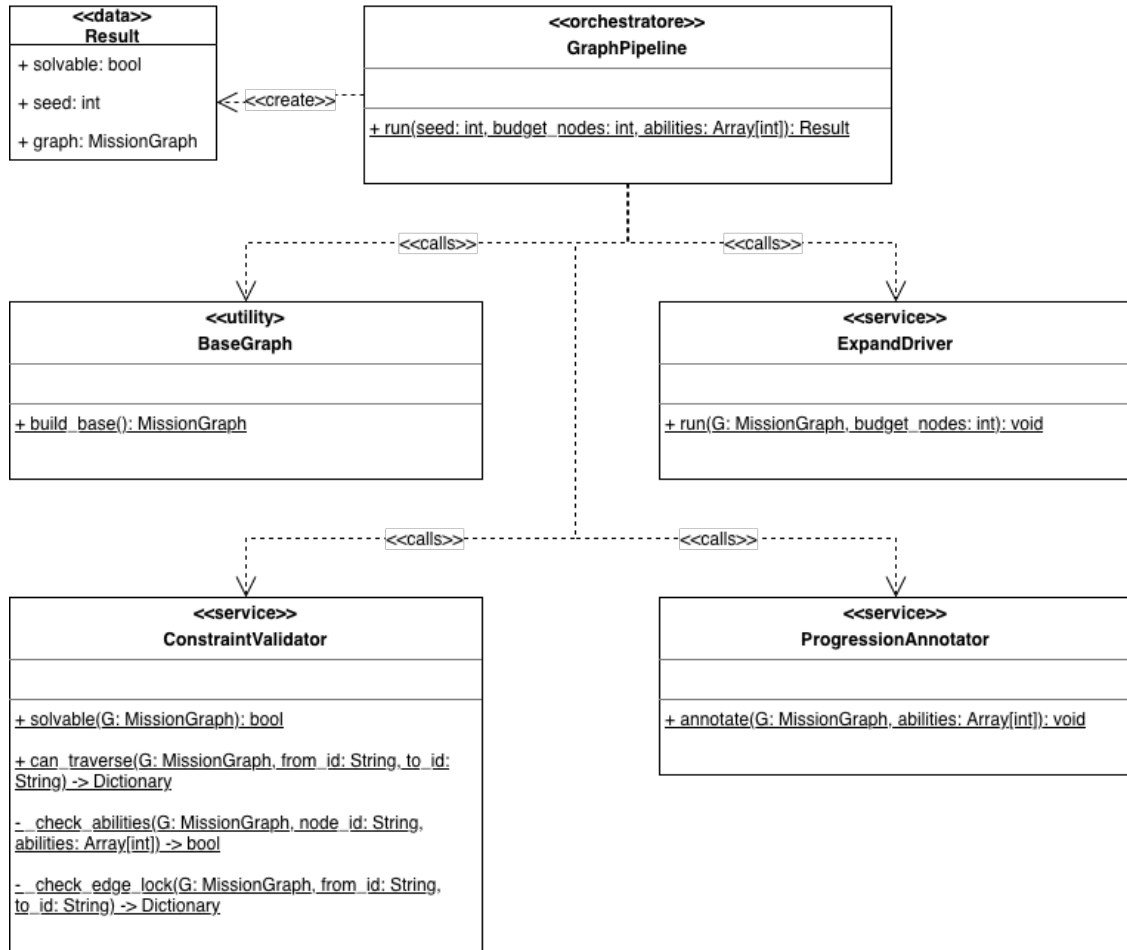


Figura 5.2: Diagramma delle classi del modulo **GraphPipeline**.

BaseGraph Il modulo **BaseGraph** è responsabile della definizione del percorso fondamentale che verrà successivamente ampliato dalle regole grammaticali. In particolare, la funzione `build_base` genera il grafo delle missioni (**MissionGraph**) e lo popola di nodi appartenenti alla classe **MissionNode**. Ogni nodo è caratterizzato dai seguenti parametri:

- **id**: una stringa che identifica in modo univoco una stanza all'interno del **MissionGraph**;
- **kind**: un enumerativo che specifica il tipo di stanza, utilizzato per selezionare il template appropriato durante la traduzione nel livello fisico;
- **grants**: una lista di abilità conferite al giocatore al completamento di una stanza;
- **require**: una lista di abilità necessarie per superare una stanza;
- **logical_abilities_before**: una lista di abilità possedute dal giocatore al suo ingresso iniziale nella stanza;
- **diff**: un intero che rappresenta la difficoltà della singola stanza;
- **catalog**: una lista di oggetti che la stanza deve contenere, ordinati per priorità;
- **enemy_directive**: una direttiva utilizzata per determinare il tipo di combattimento nella stanza e il *budget* che limita il numero massimo di nemici presenti;
- **trap_directive**: una direttiva utilizzata per stabilire il *budget* che limita il numero massimo di trappole presenti.

Una volta popolato l'insieme dei nodi del grafo, vengono definiti gli archi tra ciascun nodo iniziale, stabilendo i vincoli necessari per il passaggio da una stanza all'altra. Tali vincoli sono specificati tramite un enumerativo che indica se il passaggio richiede un oggetto chiave o l'eliminazione di tutti i nemici presenti nella stanza.

ExpandDriver Il modulo **ExpandDriver** è responsabile dell'espansione del grafo precedentemente definito attraverso l'applicazione delle regole di produzione stabilite dalla grammatica realizzata. Tali regole sono memorizzate in un file **JSON** e vengono caricate dalla memoria di massa. I nodi contenuti nel grafo base vengono caricati all'interno di un dizionario, in cui la chiave rappresenta l'identificativo del nodo e il valore è il **MissionNode** che deve essere sottoposto ad elaborazione. Per ogni nodo da elaborare, viene effettuata una ricerca all'interno delle regole di produzione per individuare un *match*. In caso di corrispondenza, la regola viene applicata, restituendo un altro dizionario con la stessa struttura descritta precedentemente. L'applicazione di ciascuna regola corrisponde alla generazione di un nuovo sottografo $G' = (V', E')$. Ad ogni applicazione, il grafo base $G = (V, E)$, dove V rappresenta l'insieme dei nodi e E l'insieme degli archi, viene ampliato con il nuovo sottografo G' . Formalmente, si avrà che $G = (V, E)$, con $V = V \cup V'$ e $E = E \cup E'$. I nuovi nodi del sottografo generato vengono copiati nel dizionario da elaborare, in modo che possano anch'essi essere sottoposti ad espansione. Infine, viene rimossa la chiave k dal dizionario di elaborazione che ha avviato il processo. La condizione di terminazione dell'espansione del grafo si verifica fintanto che il numero di nodi che lo popolano risulta inferiore al **budget_nodes** definito dal designer, formalmente:

$$|V| < \text{budget_nodes}.$$

Listing 5.2: Pseudocodice della procedura `expand`.

```
1 rules: Array = load_rules(path)
2 def expand(G: MissionGraph, budget_nodes) -> void:
3     to_elaborate: Dictionary[String, MissionNode] = G.nodes.duplicate()
4
5     while G.node_count() < budget_nodes:
6         for k: String in to_elaborate.keys():
7             node: MissionNode = to_elaborate.get(k)
8
9             for rule in rules:
10                 if _match(node, rule.get("match", {})):
11                     sub_graph: Dictionary[String, MissionNode] = _apply_rule(G,
12                                     ↪ k, rule)
13                     to_elaborate.merge(sub_graph)
14
15     to_elaborate.erase(k)
```

Ogni regola di produzione è strutturata secondo il seguente schema:

- `match`: utilizzato per identificare la *LHS* che avvia il processo di riscrittura del grafo;
- `expand_nodes`: elenco di nodi che espande il grafo base, specificando le proprietà di ciascuna stanza e il suo contenuto;
- `expand_edges`: regole impiegate per determinare gli archi da generare tra i nodi del nuovo sottografo, definendo anche i vincoli di traversabilità;
- `attach`: regola che stabilisce la modalità di connessione dei nodi del nuovo sottografo tramite archi al nodo che ha avviato il processo di espansione.

ConstraintValidator Il modulo `ConstraintValidator` svolge la funzione cruciale di verifica della solvibilità del `MissionGraph`, determinando la fattibilità di raggiungere il nodo obiettivo a partire dal nodo iniziale, nel pieno rispetto dei vincoli imposti dal sistema. La funzione principale `solvable` implementa un algoritmo di visita in BFS sul grafo, ampliando il concetto tradizionale di stato con informazioni aggiuntive essenziali per la progressione: l'insieme delle abilità acquisite, il

numero di chiavi disponibili e lo stato degli archi già sbloccati. Ogni stato esplorato è univocamente identificato da una chiave che integra tali informazioni, al fine di prevenire esplorazioni ridondanti e ottimizzare l'efficienza computazionale.

Durante il processo di esplorazione, il nodo corrente contribuisce all'aggiornamento dello stato: eventuali abilità permanenti vengono integrate nell'insieme posseduto, mentre la presenza di oggetti di tipo chiave comporta un incremento di un contatore. Per ciascun nodo adiacente, la validità della transizione è determinata dalla funzione `can_traverse`, che verifica sia i vincoli sulle abilità richieste dal nodo destinazione, sia le condizioni specifiche di attraversamento dell'arco.

Il controllo sugli archi considera tre tipologie di `LockType`:

- **FREE**: gli archi liberi sono sempre attraversabili in qualsiasi momento dal giocatore;
- **ENEMIES_CLEARED**: questi archi sono vincolati dalla sconfitta dei nemici della stanza da parte del giocatore e sono considerati sempre superabili in fase di validazione;
- **KEY**: tali archi sono bloccati da chiavi: per rendere il passaggio attraversabile è necessario consumare un oggetto chiave, a meno che non siano già stati precedentemente sbloccati; in quest'ultimo caso, lo stato conserva memoria degli archi aperti per evitare consumi di chiavi non necessari e ottimizzare la gestione delle risorse.

La procedura di verifica termina con esito positivo non appena il nodo obiettivo viene raggiunto; in caso contrario, se tutti gli stati possibili vengono esplorati senza successo, il grafo è considerato non solvibile.

ProgressionAnnotator Il modulo `ProgressionAnnotator` è concepito per arricchire il `MissionGraph` con informazioni dettagliate circa la progressione logica del giocatore. Questo modulo annota per ciascun nodo l'insieme minimo di abilità possedute al momento del suo raggiungimento, fornendo una rappresentazione accurata del percorso intrapreso dal giocatore. La procedura `annotate` implementa una visita BFS a partire dal nodo iniziale, propagando uno stato che comprende l'identificativo del nodo corrente e l'insieme delle abilità disponibili al giocatore. Per ottimizzare l'efficienza del processo e prevenire esplorazioni ridondanti, ogni stato

viene univocamente identificato tramite una chiave composta dall'identificativo del nodo e dalla configurazione corrente delle abilità.

Durante la visita del grafo, per ogni nodo raggiungibile vengono memorizzate le abilità possedute dal giocatore al momento dell'ingresso nel nodo stesso, salvandole nell'attributo `logical_abilities_before`. Tale insieme di abilità viene inizialmente propagato ai nodi successivi nel percorso e può essere aggiornato in presenza di nuove abilità acquisite dal giocatore. In particolare, quando un nodo fornisce una o più abilità (tramite il parametro `grants`), queste vengono aggiunte allo stato corrente del giocatore e influenzano la successiva esplorazione del grafo, determinando i nodi accessibili e le strategie di gioco possibili. Parallelamente, i nodi che presentano vincoli di accesso (definiti nel parametro `requires`) vengono visitati solo se tali requisiti risultano soddisfatti dall'insieme di abilità possedute dal giocatore al momento dell'accesso.

Questo meccanismo consente di costruire una rappresentazione esplicita e dettagliata della progressione del giocatore all'interno del grafo, rendendo disponibile, per ogni nodo, il contesto logico in cui esso viene raggiunto. Tali informazioni risultano fondamentali nelle fasi successive di generazione del livello, in particolare nella selezione dei template, permettendo di scegliere configurazioni di livello coerenti con le abilità effettivamente disponibili al giocatore. In questo modo, il livello fisico può adattarsi dinamicamente alla progressione del giocatore, introducendo gradualmente stanze e sfide che richiedono abilità sempre più avanzate.

5.1.3 GridPlacerBFS

Il modulo `GridPlacerBFS` è responsabile dell'assegnazione delle coordinate bidimensionali ai nodi del `MissionGraph` su una griglia discreta mediante la funzione `place`. L'algoritmo implementa una visita in `BFS` a partire dal nodo iniziale, posizionato nell'origine della griglia avente coordinate $(x = 0, y = 0)$. Questo nodo funge da punto di ancoraggio, a partire dal quale i nodi adiacenti vengono progressivamente esplorati. Per ogni nodo non ancora posizionato, la funzione `_free_adjacent_cells` individua le posizioni cardinali candidate libere nell'area circostante al nodo ancorato. Tra le celle adiacenti al nodo corrente, viene selezionata una posizione libera. La scelta tra le celle candidate è determinata da un

generatore pseudo-casuale, garantendo un comportamento deterministico a partire da un **seed** noto.

Durante l'intero processo, viene mantenuta una doppia rappresentazione: una mappa diretta che associa ogni nodo alla propria posizione e una mappa inversa che consente di verificare in tempo costante l'occupazione delle celle della griglia. Qualora non fosse disponibile alcuna posizione libera nelle immediate vicinanze, il nodo viene temporaneamente rimandato e inserito in una lista di elementi differiti. Al completamento della visita principale, qualora la lista non risulti vuota, i nodi in essa contenuti vengono gestiti mediante una fase di recovery, attraverso l'invocazione della procedura `_place_deferred_nodes`. In tale fase, si tenta di collocare i nodi rimandati utilizzando come riferimento nodi già posizionati. L'algoritmo, durante questa operazione, può apportare modifiche locali alla struttura del grafo, rimuovendo e ricreando archi al fine di garantire la connettività e individuare una configurazione spaziale valida. Per ogni nodo rimandato, si considerano i suoi vicini già posizionati e li si inserisce in una coda di elaborazione contenente i nodi di ancoraggio. Fino a quando la coda non risulta vuota, si verifica l'esistenza di una posizione libera in cui collocare il nodo rimandato. Qualora tale posizione venga individuata, essa viene registrata e si procede con il posizionamento del successivo nodo. Nel caso in cui non risulti disponibile alcuna posizione, la ricerca nel grafo viene espansa radialmente ai vicini del nodo di ancoraggio già posizionati e ubicati nelle posizioni cardinali, i quali vengono aggiunti alla coda di elaborazione. All'individuazione di una posizione libera, il nodo rimandato viene registrato nella mappa delle posizioni. Particolare attenzione è rivolta alla gestione dei vincoli sugli archi (lock), i quali vengono preservati anche a seguito di eventuali modifiche strutturali, garantendo la coerenza tra la rappresentazione logica e quella spaziale del livello. Questo approccio consente di ottenere una disposizione dei nodi compatta e navigabile, mantenendo al contempo la struttura del grafo e i vincoli di progressione definiti nelle fasi precedenti della *pipeline*.

Listing 5.3: Classe usata per memorizzare l'istanza del risultato della funzione 5.4.

```
1  # Classe utilizzata come risultato della funzione place.
2  class Result:
3      # Griglia logica: key = node.id, value = coordinata.
4      positions: Dictionary[String, Vector2i] = {}
5
6      # Lookup per il controllo costante delle posizioni già occupate.
7      occupied: Dictionary[Vector2i, String] = {}
8
9      # Lista di nodi rimandati.
10     _deferred: Array[String] = []
11
```

Listing 5.4: Pseudocodice della funzione `place`, essa è responsabile di eseguire l'*embedding* del grafo in una griglia discreta.

```
1 def place(G: MissionGraph) -> Result:
2     out: Result = Result.new()
3
4     # Guard: grafo nullo o senza start.
5     if G == null or G.start_id == "":
6         return out
7
8     # Piazzo la stanza iniziale.
9     start: String = G.start_id
10    out.positions[start] = Vector2i.ZERO
11    out.occupied[Vector2i.ZERO] = start
12
13    # Coda di elaborazione.
14    var queue: Array[String] = [start]
15    while not queue.is_empty():
16        # Selezione il nodo che sarà il punto di partenza della ricerca.
17        anchor: String = queue.dequeue()
18        anchor_pos: Vector2i = out.positions[anchor]
19        for adjacent: String in G.neighbors(anchor):
20            # Se il nodo adiacente è stato già piazzato vai al prossimo.
21            if out.positions.has(adjacent):
22                continue
23
24            # Individua le posizioni candidate libere.
25            candidates: Array[Vector2i] = _free_adjacent_cells(anchor_pos,
26                ↪ out.occupied)
27
28            # Se non ci sono posizioni libere rimanda il nodo.
29            if candidates.is_empty():
30                out._deferred.append(adjacent)
31                continue
32
33            chosen: Vector2i = candidates[randi_range(0, candidates.size() - 1)]
34
35            # Registra la nuova posizione.
36            out.positions[adjacent] = chosen
37            out.occupied[chosen] = adjacent
38            queue.append(out)
39
40            # Se ci sono nodi devono essere piazzati vengono gestiti.
41            if not r._deferred.is_empty():
42                _place_deferred_nodes(G, out)
43
44    return out
```

5.1.4 RoomSpawner

Il modulo `RoomSpawner` è incaricato dell'istanziamento delle stanze fisiche del livello a partire dalla rappresentazione logica e dal layout su griglia precedentemente generati. Per il corretto funzionamento, il modulo richiede le seguenti informazioni:

- **root**: il nodo radice della scena in cui devono essere istanziate le stanze;
- **positions**: le posizioni logiche di ciascuna stanza nella griglia precedentemente calcolate;
- **occupied**: le posizioni occupate dai nodi, parametro di utilità per velocizzare il *lookup*;
- **graph**: il grafo di missione che rappresenta il livello;
- **assembler**: il modulo `RoomAssembler` è responsabile della selezione dei *template*;
- **cell_tiles**: rappresenta la larghezza e l'altezza delle celle che conterranno le stanze; tale rettangolo avrà larghezza e altezza pari alla massima altezza e larghezza tra tutti i *template* disponibili, formalmente $cell_tiles = (w, h) + padding$: $w = \max\{s_1w, s_2w, \dots, s_nw\}$, $h = \max\{s_1h, s_2h, \dots, s_nh\}$, con s_i rappresentante una stanza qualsiasi e w e h la rispettiva larghezza e altezza.

Il processo di traduzione si svolge iterando su tutti i nodi logici posizionati. Per ciascun nodo, vengono innanzitutto determinati i requisiti dei connettori, ovvero le direzioni in cui la stanza deve presentare aperture, in base alla presenza di nodi adiacenti nella griglia. Tali requisiti vengono successivamente utilizzati per selezionare un *template* compatibile mediante **assembler**.

Una volta selezionato il prefabbricato, esso viene istanziato e posizionata nello spazio di gioco mediante la funzione `instantiate_room`. Le coordinate discrete della griglia vengono convertite in coordinate continue nel mondo moltiplicando `cell_tiles` per la dimensione in *pixel* della singola *tile*, indicata dal parametro `tile_size`. La formula di conversione è la seguente:

$$cell_px = (w_{px}, h_{px})$$

$$\begin{aligned} w_{px} &= cell_tiles.w \times tile_size.w, \\ h_{px} &= cell_tiles.h \times tile_size.h \end{aligned} \tag{5.1}$$

La stanza viene quindi inserita nel mondo di gioco moltiplicando le coordinate della griglia logica per `cell_px`.

Successivamente, la stanza fisica viene associata al nodo logico corrispondente nel grafo, garantendo una corrispondenza diretta tra la rappresentazione astratta e l'istanza concreta. Infine, l'oggetto viene inserito nella `root`. Il modulo implementa un approccio *fail-safe*: eventuali errori nella selezione del *template* o nell'istanziamento non interrompono il processo complessivo, ma comportano semplicemente l'esclusione della stanza problematica.

RoomAssembler Il modulo `RoomAssembler` rappresenta il fulcro dell'integrazione tra la rappresentazione logica del dungeon e la fase di traduzione dei nodi in stanze prefabbricate, le quali vengono successivamente implementate nel mondo di gioco. Esso coordina l'utilizzo dei moduli `RoomTemplatePicker` e `RoomTemplateCatalog`, responsabili, rispettivamente, della gestione del processo di selezione e *listing* dei prefabbricati.

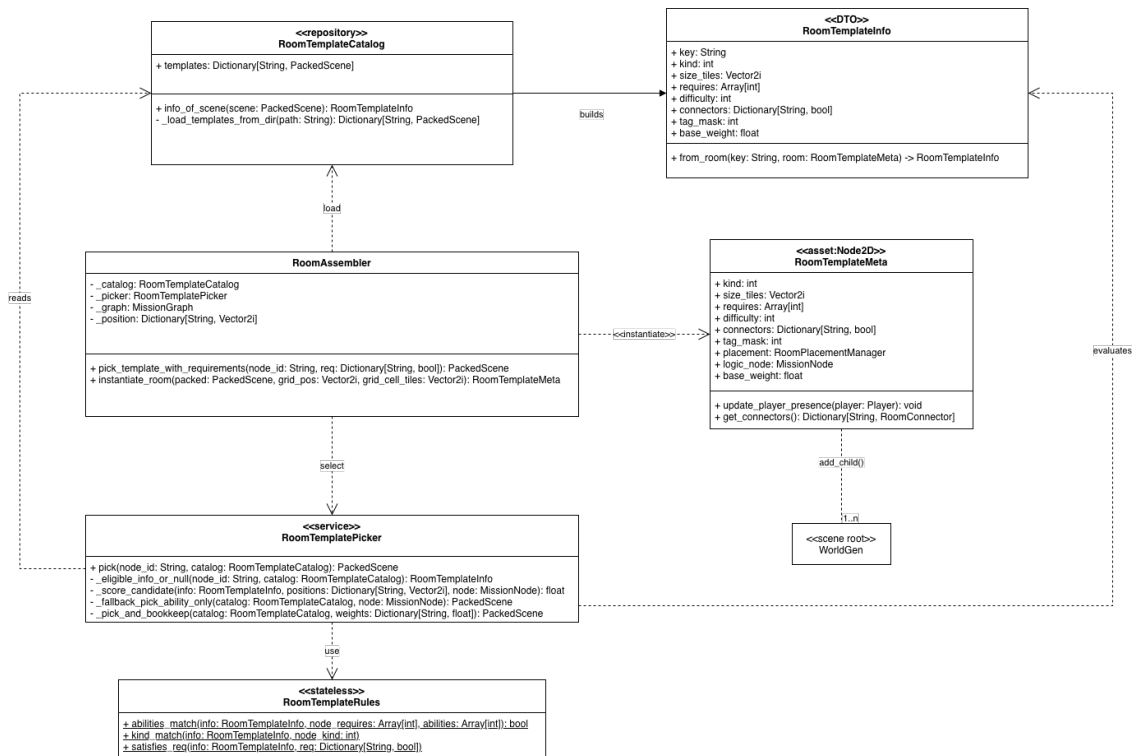


Figura 5.3: Diagramma delle classi del modulo RoomAssembler.

Il `RoomTemplateCatalog` svolge un ruolo fondamentale nella semplificazione dell'accesso ai prefabbricati disponibili. Tali prefabbricati vengono inizialmente caricati mediante la funzione `_load_templates_from_dir` e successivamente organizzati in una struttura indicizzata per chiave. La funzione `from_room` consente di estrapolare le informazioni relative a una stanza necessarie per la selezione del template a partire dall'oggetto `RoomTemplateMeta`.

La selezione dei prefabbricati è delegata al modulo `RoomTemplatePicker`, richiamando la funzione `pick`. Per ogni `nodo.id` posizionato nella griglia vengono filtrati i prefabbricati letti dal `RoomTemplateCatalog` attraverso la funzione `_eligible_info_or_null`, ottenendo, per ciascun template, una rappresentazione compatta sotto forma di `RoomTemplateInfo`. Durante questa fase vengono impiegate le funzioni del modulo `RoomTemplateRules`, in particolare:

- La funzione `abilities_match` verifica la compatibilità tra le abilità richieste dal nodo e quelle richieste dal prefabbricato. Qualora un nodo richieda una competenza specifica (`node.require`), il template dovrà necessariamente includere la medesima abilità. Nel caso in cui un nodo possa richiedere opzionalmente determinate capacità (`node.logical_abilities_before`), verranno considerati anche i *template* che presentano opzionalmente tali competenze, oltre a quelle obbligatorie.
- La funzione `satisfies_req` verifica il soddisfacimento dei requisiti relativi ai connettori della stanza. In particolare, in base alla posizione spaziale della stanza rispetto al nodo precedente, viene richiesto un prefabbricato che disponga del connettore necessario per il collegamento alla stanza precedentemente inserita. Ad esempio, se una stanza deve essere posizionata a destra di un'altra, il prefabbricato dovrà essere dotato di un connettore posizionato sul lato ovest.
- La funzione `kind_match` garantisce la coerenza tra il tipo di stanza e il `kind` del nodo. Per rappresentare in quali template un nodo può essere tradotto, si utilizza una `bit_mask`. Ciò consente al designer di assegnare a ciascun template ruoli differenti, offrendo una maggiore varietà nella scelta durante la conversione da nodo a stanza fisica.

Una volta individuati i candidati, il modulo procede alla loro valutazione tramite la funzione `_score_candidate`, che assegna un punteggio a ciascun template tenendo conto di fattori quali:

- **cover**: il numero di connettori posseduti dal template (un numero maggiore di connettori aumenta la probabilità di compatibilità con altri connettori situati nelle stanze adiacenti);
- **w_difficulty**: un parametro di ponderazione che valuta la compatibilità tra la difficoltà di un template e quella di un nodo. Questo si basa su una distribuzione *gaussiana* centrata su differenza zero. Il peso aumenta con la similitudine tra le due difficoltà, mentre diminuisce rapidamente con l'aumentare della loro differenza. Formalmente:

$$w = e^{-\frac{(d_t - d_n)^2}{k \cdot \sigma^2}}$$

dove d_t indica la difficoltà del template, d_n la difficoltà del nodo, σ controlla la larghezza della distribuzione (tolleranza alla differenza) e k è una costante;

- **w_diversity**: il fattore di diversità penalizza i template selezionati con maggiore frequenza, incentivando l'utilizzo di quelli scelti meno frequentemente.

Lo score finale è determinato dalla seguente espressione:

$$score = w_base \times cover^{cover} \times w_difficulty \times w_diversity$$

dove **w_base** rappresenta il peso base specificato dal designer nel template.

Al termine della fase di scoring, il metodo `_pick_and_bookkeep` seleziona il template finale in base ai pesi calcolati. Qualora nessun candidato soddisfi i criteri di selezione, il meccanismo di *fallback* `_fallback_pick_ability_only` viene attivato, rimuovendo alcuni vincoli pur garantendo la coerenza minima richiesta rispetto alle abilità.

5.1.5 CorridorBuilder

Il modulo `CorridorBuilder` è incaricato della generazione di corridoi che connettono le stanze precedentemente posizionate nel livello fisico, traducendo le connessioni

descritte dagli archi del `MissionGraph` in connessioni spaziali concrete. La funzione `connect_adjacent` sovrintende all'intero processo, suddividendo la costruzione dei corridoi nelle seguenti fasi. Inizialmente, viene generato un dizionario in cui le chiavi rappresentano le posizioni logiche dei prefabbricati inseriti, mentre i valori sono i prefabbricati stessi. Successivamente, viene istanziato un nuovo layer per la costruzione dei corridoi nel contesto del gioco, utilizzando un oggetto di tipo `TileMapLayer`. Prima di procedere con lo scavo dei corridoi tra le stanze, viene eseguito il metodo `_fill_between_rooms`, il cui scopo è quello di creare blocchi solidi tra le stanze adiacenti. Per ogni coppia di nodi connessi nel grafo, viene calcolata la *bounding box*, un rettangolo che racchiude entrambe le stanze. Tale rettangolo viene completamente riempito inserendo *tile* solide nel *layer* dei corridoi, escludendo le aree occupate dalle stanze per evitare sovrapposizioni. Questa operazione garantisce una continuità visiva tra le stanze. La generazione dei corridoi traversabile avviene analizzando le adiacenze del grafo e prevenendo duplicazioni attraverso un criterio di ordinamento stabile delle coppie di nodi. Per ogni coppia di stanze adiacenti, viene determinata la direzione di connessione (orizzontale o verticale) e viene verificata la presenza di connettori compatibili sui lati coinvolti (ad esempio, "E" per la stanza sinistra e "W" per quella destra). Il processo di connessione si articola in due fasi: in primo luogo, viene scavato un passaggio continuo nel `TileMapLayer` dei corridoi tra i *marker* dei connettori; in secondo luogo, vengono aperti varchi nei layer di collisione delle stanze, rimuovendo i *tile* lungo i bordi e scavando verso l'interno a partire dalla posizione dei *marker*. I connettori determinano la posizione e l'estensione del varco tramite parametri di *offset* nelle direzioni cardinali (su, giù, destra e sinistra), consentendo la creazione di passaggi di ampiezza variabile. Un algoritmo di *carving* opera all'interno del rettangolo precedentemente costruito, utilizzando la posizione dei connettori come punti di partenza e di arrivo, scavando di una larghezza pari agli *offset* stabiliti dai connettori stessi. Infine, a partire dai connettori che hanno dato origine al *carving*, vengono aperte le pareti delle stanze tra cui è stato costruito il corridoio, rispettando gli stessi valori di *offset*.

5.1.6 WorldFinalize

Il modulo **WorldFinalize** rappresenta la fase conclusiva della pipeline di generazione procedurale del livello, incaricato dell'esecuzione del *merge* dei diversi *layer* prodotti, dell'attivazione del sistema di **autotiling** e dell'istanziamento in scena del personaggio controllato dal giocatore. Tali operazioni sono implementate nella funzione `merge_and_spawn`.

Inizialmente, viene eseguito il *merge* dei diversi **TileMapLayer** generati nelle fasi precedenti, in particolare quelli relativi alle collisioni delle stanze e ai corridoi, al fine di ottenere un unico *layer* finale coerente, che fungerà da base per la navigazione e le interazioni fisiche. Successivamente, il sistema di autotiling viene applicato al *layer* finale per garantire la coerenza visiva tra stanze e corridoi. Infine, il giocatore viene istanziato all'interno della stanza di partenza del livello.

5.1.7 RoomsManager

Il modulo **RoomsManager** è responsabile del popolamento delle stanze e della loro gestione durante la fase di gioco. Il flusso operativo si articola in diverse fasi: la generazione dei punti di spawn per ciascuna stanza, effettuata tramite i moduli **SmartPlacement** e **RoomPlacementManager**; l'utilizzo dei punti di spawn calcolati per l'istanziamento delle entità di gioco, quali decorazioni, nemici, oggetti e trappole, descritte dai nodi logici; e la gestione del comportamento della stanza durante l'esecuzione del gioco, affidata al modulo **RoomsManager**.

Implementazione

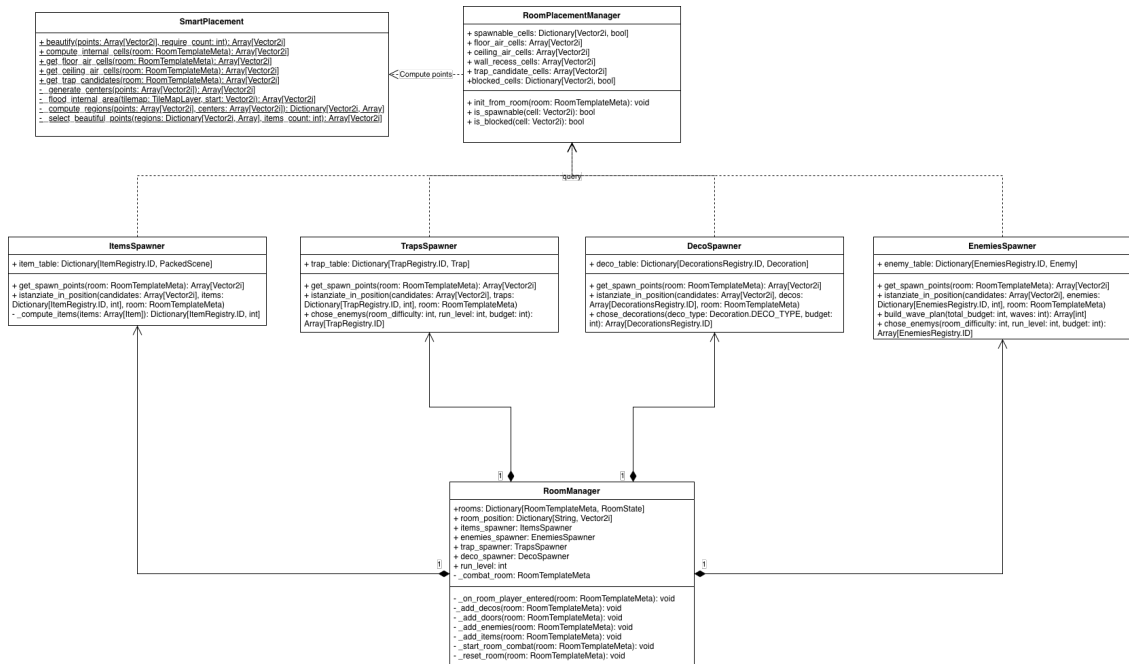


Figura 5.4: Diagramma delle classi del modulo RoomsManager.

Durante la fase di inizializzazione, le stanze precedentemente posizionate vengono connesse a un segnale che notifica il sistema in merito all'ingresso o all'uscita di un giocatore da una stanza. Questo si rende necessario per avviare determinati eventi nelle fasi successive di gioco. Successivamente, la funzione `init_from_room` del `RoomPlacementManager` memorizza per ciascuna stanza i punti di interesse di quest'ultima mediante le funzioni esposte dal modulo statico `SmartPlacement`:

- `floor_air_cells`: insieme delle posizioni che sono immediatamente sopra il pavimento;
- `ceiling_air_cells`: insieme delle posizioni che sono immediatamente sotto il soffitto;
- `wall_recess_cells`: insieme delle posizioni che si trovano in rientranze di muri;
- `trap_candidate_cells`: insieme delle posizioni che sono utili per il piazzamento delle trappole.

Il `RoomPlacementManager` conserva un registro delle posizioni precedentemente occupate da altre entità, al fine di prevenire sovrapposizioni, attraverso l'utilizzo di un dizionario. In tale dizionario, la chiave rappresenta la posizione, mentre il valore è un indicatore booleano che ne determina l'utilizzo. È disponibile una funzione `is_blocked` che consente di verificare la disponibilità di una cella. A seguito dell'acquisizione della lista dei punti, i diversi `spawner` richiedono i punti di interesse pertinenti al `RoomPlacementManager` per ogni stanza e applicano le *policy* decisionali relative all'istanziamento delle entità di gioco. Successivamente, le entità selezionate vengono inserite nelle rispettive stanze mediante l'esecuzione delle funzioni definite dal `RoomManager`:

- `_add_items`: aggiunge gli oggetti di gioco interattivi con il giocatore;
- `_add_traps`: aggiunge le trappole che ostacolano il giocatore;
- `_add_enemies`: aggiunge le entità ostili che il giocatore deve sconfiggere;
- `_add_decos`: aggiunge le decorazioni della stanza.

Infine, la funzione `_add_doors` aggiunge le porte che limitano il passaggio, come descritto dai vincoli degli archi del grafo. Gli oggetti e i nemici sono inizialmente nascosti al giocatore e vengono rivelati e resi interattivi solo al completamento di specifici eventi. In particolare, la fase di esecuzione mantiene traccia della stanza in cui si trova il giocatore tramite la funzione `_on_room_player_entered`. Quando il giocatore accede a una stanza con un combattimento, le porte si chiudono e vengono visualizzati i nemici che devono essere sconfitti per proseguire, tramite la funzione `_start_room_combat`. Al termine del combattimento, vengono visualizzati gli oggetti della stanza che rappresentano la ricompensa ottenuta, e successivamente le porte vengono riaperte. Qualora il giocatore perdesse la vita durante questa fase, viene riportato all'ultimo checkpoint e la stanza viene ripristinata alla sua fase iniziale, con tutti i nemici nuovamente attivi e gli oggetti nascosti, tramite la funzione `_reset_room`.

SmartPlacement Il modulo `SmartPlacement` implementa un sistema deterministico per il calcolo di posizioni di *spawn* all'interno di una stanza, operando interamente nello spazio discreto della stanza.

Inizialmente, la funzione `compute_internal_cells` identifica la geometria interna attraversabile mediante un algoritmo di *flood fill* (`_flood_internal_area`) limitato all'area utilizzata della `TileMap` della stanza, producendo un insieme di celle "aria" accessibili. Tale insieme è rappresentato sia come lista sia come struttura di *lookup* a tempo costante (dizionario), consentendo l'ottimizzazione di operazioni frequenti quali controlli di adiacenza e filtri topologici.

Su questa base, vengono costruiti sottoinsiemi semantici applicando regole locali (ad esempio, una cella pavimento è identificata dall'essere una cella non solida e avente una cella sotto solida), fungendo da candidati per diverse categorie di *spawn*. Successivamente, i punti vengono distribuiti mediante un algoritmo di clustering "Voronoi-like", che suddivide lo spazio in regioni attraverso la funzione `_compute_regions` a partire da centroidi casuali calcolati mediante la funzione `_generate_centers`. I punti selezionati vengono distribuiti in modo proporzionale alla densità delle regioni `_select_beautiful_points`.

All'interno di ciascuna regione, i punti vengono selezionati mediante una strategia di *farthest point sampling*, che assicura una distribuzione spazialmente uniforme dei punti selezionati. Formalmente, tale distanza viene calcolata come:

$$p^* = \arg \max_{p \in R \setminus S} \left(\min_{s \in S} d(p, s)^2 \right)$$

- R rappresenta l'insieme completo di tutti i punti disponibili all'interno della regione geografica considerata;
- S indica l'insieme dei punti già selezionati nelle iterazioni precedenti del processo di selezione;
- $R \setminus S$ denota l'insieme dei punti candidati ancora disponibili per la selezione;
- $d(p, s)$ esprime la distanza euclidea tra il punto candidato p e un punto già selezionato s ;
- $\min_{s \in S} d(p, s)^2$ calcola, per ciascun punto candidato p , la distanza minima rispetto a tutti i punti già selezionati;
- $\arg \max_{p \in R \setminus S}$ individua il punto p che massimizza tale distanza minima, garantendo una distribuzione spazialmente uniforme dei punti selezionati.

5.2 Moduli - Generazione delle stanze (Offline)

5.2.1 RoomLayoutMask

Il presente modulo fornisce le informazioni essenziali per rappresentare i layout di una stanza mediante una struttura a griglia discreta bidimensionale detta *mask* (matrice), la cui dimensione (**size**) viene determinata casualmente all'interno di un intervallo definito dai limiti inferiori (**min_size**) e superiori (**max_size**) specificati dal designer prima della fase di generazione. Ogni cella interna alla griglia può essere classificata come:

- solida, rappresentata dal valore 1 nella griglia;
- vuota, rappresentata dal valore 0 nella griglia.

Inizialmente, ogni valore della griglia viene impostato su solido (valore 1) e successivamente viene modificato dagli operatori strutturali. Per la rappresentazione di tale struttura viene impiegato un **PackedByteArray**, la cui dimensione è impostata su **size.x × size.y**.

5.2.2 EvolutionEngine

Il processo di generazione delle stanze si fonda su un algoritmo evolutivo implementato nel modulo **EvolutionEngine**, il cui scopo primario è la selezione automatica del layout più idoneo tra un insieme di configurazioni candidate. L'algoritmo opera mantenendo una popolazione di genomi, ciascuno dei quali contiene le informazioni necessarie per la costruzione di una possibile configurazione della stanza. Tali parametri sono inizialmente generati in modo casuale tramite una **LayoutGenomeFactory**. Questo modulo definisce la struttura e il contenuto dei genomi che rappresentano i layout delle stanze. Ciascun genoma è composto da una lista di geni che codificano un operatore di trasformazione del layout sotto forma di coppia (tipo operatore, dizionario di parametri). La costruzione di un genoma segue una strategia ibrida che distingue tra operatori strutturali obbligatori e operatori secondari opzionali. In particolare, gli operatori di tipo **CONNECTOR** e **BACKBONE** sono sempre inclusi, in quanto garantiscono rispettivamente la presenza di punti

di connessione con altre stanze e una struttura portante del layout basata sui connettori, assicurando che ogni candidato soddisfi i requisiti minimi di percorribilità. A questi vengono affiancati un numero variabile di operatori secondari, selezionati tramite un meccanismo di estrazione pesata che introduce diversità strutturale tra i genomi. Ciascun operatore è responsabile della generazione dei propri parametri in modo stocastico attraverso la funzione `create_random_params`, che restituisce un dizionario avente come chiave una stringa che identifica il valore del parametro generato e come valore un componente casuale generato a partire dai vincoli imposti dal designer. Questi vincoli specificano gli intervalli in cui i valori dei parametri generati dagli operatori devono rientrare (ad esempio, è possibile definire i limiti superiori e inferiori circa la larghezza e l'altezza dell'operatore che genera le piattaforme di una stanza). Gli operatori sono inoltre responsabili della mutazione dei propri parametri `mutate_params` e della costruzione della forma geometrica a partire dai parametri `applay`.

Il ciclo evolutivo si sviluppa per un numero predeterminato di generazioni e, ad ogni iterazione, tutti gli individui della popolazione vengono sottoposti a valutazione mediante una funzione di fitness che assegna un punteggio numerico in base alla qualità del layout. Il modulo `LayoutFitnessEvaluator` è responsabile dell'implementazione della sequenza di validazione e valutazione del layout.

Listing 5.5: Pseudocodice della fase evolutiva del modulo EvolutionEngine

```
1 factory: GenomeFactory
2 mutator: GenomeMutator
3 evaluator: FitnessEvaluator
4 selection: SelectionStrategy
5 reproduction: ReproductionStrategy
6
7 def evolve() -> Genome:
8     population: Array[Genome] = []
9
10    for i in range(population_size):
11        population.append(factory.random_genome())
12
13    best_genome: Genome = nil
14    best_score: float = -INF
15
16
17    for gen in range(generations):
18
19        var evaluated: Array[EvaluationResult] = _evaluate(population)
20
21        if evaluated.is_empty():
22            population.clear()
23            for i in range(population_size):
24                population.append(factory.random_genome())
25            continue
26
27        if evaluated[0].score > best_score:
28            best_score = evaluated[0].score
29            best_genome = evaluated[0].genome
30
31        var survivors: Array[Genome] = selection.select(evaluated,
32            ↪ survival_rate, rng)
33
34        population = reproduction.reproduce(
35            survivors, factory, mutator, rng, population_size,
36            ↪ mutation_rate
37        )
38
39    return best_genome
```

5.2.3 LayoutFitnessEvaluator

Per quanto riguarda la validazione, il modello del giocatore viene astratto per recuperare informazioni quali:

- la massima altezza di salto in *tiles*;
- la massima altezza di salto dal muro in *tiles*;
- la massima lunghezza di salto in *tiles*;
- il movimento su pavimento in *tiles*.

Tali informazioni sono impiegate per verificare la raggiungibilità di tutte le celle pavimento (`are_floor_tile_reacable`) e di tutti i connettori per l'uscita dalla stanza (`are_connectors_reacable`) mediante i movimenti effettivi del giocatore all'interno della mask. Al fine di ottimizzare la velocità della fase di validazione per ciascun layout, la raggiungibilità di tali celle è verificata utilizzando come punto di partenza la cella che nella mask risulta la più bassa (con valore *y* più elevato) nel caso delle celle pavimento e, analogamente, per i connettori, il connettore più in basso (quello posizionato sul lato esterno *sud*). Qualora tali celle non risultassero raggiungibili, il layout viene scartato; in caso contrario, può proseguire con la fase di valutazione. Tale metodologia consente di separare, nella maggior parte dei casi, i layout che presentano almeno un percorso che permetta al giocatore di risalire la stanza e raggiungere le posizioni di interesse dalle stanze che presentano vincoli che impediscano la salita (piattaforme che non consentono il passaggio fisico del giocatore o muri con rientranze che non permettono la scalata). Tale assunzione si basa sul presupposto che, se esiste un percorso che dal basso consenta al giocatore di raggiungere una cella situata su una porzione elevata mediante i suoi movimenti, e poiché dall'alto è sempre possibile cadere, lo stesso percorso per la salita può essere utilizzato per la discesa. Questo approccio consente di ridurre i tempi di validazione per ciascun layout della popolazione, ma non garantisce sempre la giocabilità della stanza, in quanto sarebbe necessario il controllo della raggiungibilità delle celle di interesse da più punti, il che aumenterebbe drasticamente i tempi di ricerca.

La fase di valutazione di un layout ne determina le qualità attraverso la definizione delle seguenti funzioni:

Bilanciamento di densità La funzione `_density_balance` valuta la prossimità della densità dei blocchi solidi a un valore di densità ottimale. Il punteggio viene normalizzato nell'intervallo $[0,1]$, penalizzando linearmente la deviazione dalla densità target e saturando ai limiti dell'intervallo. Tale metrica consente di mantenere un equilibrio tra stanze eccessivamente affollate, quindi poco esplorabili, e stanze eccessivamente vuote, quindi poco interessanti.

$$r = \frac{S}{N}$$

$$f(r) = \max(0, \min(1, 1 - k \cdot |r - r_{ideal}|))$$

Dove:

- N rappresenta il numero totale di celle;
- S rappresenta il numero di celle solide;
- r rappresenta la densità dei solidi;
- r_{ideal} (30%) è una costante che rappresenta la densità totale;
- k è una costante che rappresenta il fattore di tolleranza.

Distribuzione verticale La metrica `_height_distribution` quantifica la distribuzione dello spazio vuoto lungo l'asse verticale. Si definisce come l'intervallo compreso tra la cella vuota più elevata e quella più bassa, normalizzato rispetto all'altezza complessiva della griglia. Tale metrica consente di valutare la struttura della stanza preferendo una stratificazione verticale (valori alti) a stanze piatte (valori bassi).

$$y_{min} = \min\{y \mid (x, y) \in E\}, \quad y_{max} = \max\{y \mid (x, y) \in E\}$$

$$\Delta y = y_{max} - y_{min}$$

$$f = \max\left(0, \min\left(1, \frac{\Delta y}{H}\right)\right)$$

Dove:

- H rappresenta l'altezza totale della griglia;
- E costituisce l'insieme delle celle vuote $\{(x, y) \mid \text{cella vuota}\}$;
- $y_{min} = \min y \mid (x, y) \in E$;
- $y_{max} = \max y \mid (x, y) \in E$.

Simmetria La metrica `_symmetry` quantifica il grado di simmetria orizzontale di un layout confrontando ogni cella con la sua controparte speculare rispetto all'asse verticale centrale. Il valore restituito indica la proporzione di celle non corrispondenti, fornendo una misura normalizzata dell'asimmetria. Tale valore viene utilizzato come penalizzazione per i layout in caso di stanze simmetriche, poiché le stanze asimmetriche sono percepite come più organiche e naturali.

$$M = \sum_{y=0}^{H-1} \sum_{x=0}^{m-1} |S(x, y) - S(W - 1 - x, y)|$$

$$N = m \cdot H$$

$$f = \frac{M}{N}$$

$f = 1$ massima asimmetria, $f = 0$ simmetria perfetta,

Dove:

- W rappresenta la larghezza della griglia;
- H rappresenta l'altezza della griglia;
- $m = \frac{W}{2}$ è l'asse di simmetria verticale;
- $r_{ideal} = 0.3$ è una costante che rappresenta la densità totale;
- $S(x, y) \in \{0,1\}$ è la funzione indicatrice che stabilisce se una cella è solida (valore 1) o è vuota (valore 0);
- $|S(x, y) - S(W - 1 - x, y)|$ rappresenta il confronto tra una cella e la sua controparte speculare;
- M è il numero di mismatch;
- N è il numero di confronti.

Monotonia orizzontale La metrica `_horizontal_monotony`, valuta la presenza di segmenti continui di celle solide all'interno di un layout. Dopo aver identificato tali segmenti per ciascuna riga, viene applicata una funzione di penalità che si attiva esclusivamente oltre una soglia predefinita. Tale funzione di penalità presenta una crescita quadratica in funzione della lunghezza normalizzata del segmento. Il valore finale della metrica è determinato calcolando la media normalizzata sull'intera altezza del layout. Questo approccio consente di evitare piattaforme eccessivamente lunghe e uniformi, privilegiando layout meno ripetitivi e più stimolanti.

$$p(\ell, W) = \begin{cases} 0 & \text{se } \frac{\ell}{W} < \tau \\ \left(\frac{\ell}{W}\right)^2 & \text{altrimenti} \end{cases}$$

$$P = \sum_{y=0}^{H-1} \sum_{\ell \in R_y} p(\ell, W)$$

$$f = \max \left(0, \min \left(1, \frac{P}{H} \right) \right)$$

Dove:

- ℓ rappresenta la lunghezza del segmento considerato;
- W indica la larghezza massima della griglia;
- H corrisponde all'altezza totale della griglia;
- $\tau = 0.4$ è una costante che definisce la lunghezza oltre la quale un segmento è considerato eccessivamente lungo;
- P è la penalità totale;
- $p(\ell, W)$ è la funzione di penalità applicata a segmenti di lunghezza ℓ ;
- R_y denota l'insieme delle lunghezze dei segmenti continui di celle con altezza y .

Platform score La metrica `_internal_platform_score` assegna un punteggio alle superfici calpestabili interne, definite come celle solide con uno spazio vuoto immediatamente sovrastante. Il punteggio normalizzato è determinato dal rapporto tra il numero di tali celle e il numero totale di celle della griglia, moltiplicato per un fattore di scala e limitato nell'intervallo $[0,1]$. Questa funzione privilegia i layout che presentano strati intermedi, le piattaforme.

$$\mathcal{P} = \{(x, y) \mid 1 \leq y < H, S(x, y) = 1 \text{ e } E(x, y - 1) = 1\}$$

$$N = W \cdot H$$

$$f = \max \left(0, \min \left(1, \frac{|\mathcal{P}|}{N} \cdot \alpha \right) \right)$$

Dove:

- $S(x, y) \in \{0,1\}$ rappresenta la funzione indicatrice che determina se una cella è solida (valore 1) o vuota (valore 0);
- $E(x, y) = 1 - S(x, y)$ rappresenta la funzione indicatrice che determina se una cella è vuota (valore 1) o solida (valore 0);
- $\mathcal{P}$ denota l'insieme delle celle solide con celle vuote sopra (superfici calpestabili);
- N indica il numero totale di celle;
- α è una costante utilizzata per amplificare il punteggio.

Stratificazione verticale La metrica `_vertical_stratification` assegna un punteggio più elevato a layout che presentano una maggiore varietà di livelli distinti di blocchi solidi, promuovendo così la diversità verticale. Tale caratteristica contribuisce alla creazione di percorsi più stimolanti, ottimizza la distribuzione delle superfici calpestabili e incrementa la complessità esplorativa favorendo layout aventi più piani distinti.

$$L = \{y \in [0, H - 1] \mid \exists x \in [0, W - 1], S(x, y) = 1\}$$

$$f = \max \left(0, \min \left(1, \frac{|L|}{H} \right) \right)$$

Dove:

- $S(x, y) \in \{0, 1\}$ rappresenta la funzione indicatrice che determina se una cella è solida (valore 1) o vuota (valore 0);
- L denota l'insieme delle righe che contengono almeno una cella solida.

5.3 TopKSelection

La popolazione di individui valutati viene ordinata in base allo score ottenuto da ciascun layout. Il modulo `TopKSelection` implementa una strategia elitista, selezionando i migliori candidati in base allo score di fitness precedentemente determinato. Solo una frazione della popolazione, pari al 25% (`survival_rate`), viene preservata per la generazione successiva. Tale approccio garantisce che solo i genomi più promettenti contribuiscano all'evoluzione della popolazione, favorendo un rapido miglioramento della qualità media delle soluzioni. La selezione *Top-K* elimina completamente gli individui meno performanti, incrementando la pressione selettiva e accelerando il processo di convergenza.

5.4 ElitistHybridReproduction

Il modulo `ElitistHybridReproduction` è incaricato della generazione della nuova popolazione di genomi mediante l'implementazione di strategie di crossover e

mutazione. Inizialmente, i genomi sopravvissuti alla fase precedente vengono preservati integralmente e copiati direttamente nella nuova popolazione, garantendo la conservazione delle soluzioni ottimali tra le generazioni. Successivamente, la popolazione viene integrata mediante la generazione di nuovi individui a partire dai genomi sopravvissuti. Per ogni nuovo elemento, viene selezionato casualmente un genitore e, con una probabilità costante pari al 50% (`CROSSOVER_RATE`), viene applicato un operatore di crossover strutturale che fonde le informazioni genetiche di due genitori distinti, producendo un genoma figlio che eredita caratteristiche da entrambi. In alternativa, il nuovo individuo viene ottenuto mediante la clonazione di un genitore. A questa fase segue l'applicazione di una mutazione con probabilità dell'80% (`mutation_rate`), che introduce variazioni locali nel genoma e contribuisce al mantenimento della diversità della popolazione. L'impiego combinato di crossover e mutazione consente, da un lato, la ricombinazione di soluzioni già promettenti, accelerando la convergenza e, dall'altro, l'esplorazione di nuove configurazioni, mitigando il rischio di convergenza prematura verso ottimi locali.

5.5 LayoutGenomeMutator

La variabilità necessaria all'esplorazione dello spazio delle soluzioni è introdotta dal modulo `LayoutGenomeMutator`, il quale è responsabile dell'applicazione di mutazioni controllate sui genomi durante il processo evolutivo. Il mutatore opera generando una copia del genoma originale e applicando una singola trasformazione selezionata in modo stocastico da un insieme predefinito di operazioni:

- `_mutate_param`: mutazione dei parametri di un gene esistente;
- `_add_gene`: aggiunta di un nuovo operatore secondario;
- `_remove_gene`: rimozione di un operatore secondario;
- `_swap_genes`: riordinamento della sequenza degli operatori.

Tale metodologia consente di intervenire sia sulla configurazione interna degli operatori sia sulla struttura complessiva del genoma. Le mutazioni, tuttavia, sono soggette a vincoli strutturali rigorosi: i geni corrispondenti agli operatori fondamentali, quali `CONNECTOR` e `BACKBONE`, sono invariabilmente preservati e non possono

essere rimossi o alterati nella loro posizione, garantendo così la validità minima del layout generato. La modifica dei parametri avviene mediante istanze temporanee degli operatori, utilizzate esclusivamente per applicare variazioni coerenti con il dominio del problema. L'aggiunta e la rimozione di geni consentono di esplorare configurazioni strutturalmente differenti, mentre il riordinamento dei geni permette di variare l'ordine di applicazione degli operatori, influenzando indirettamente il layout risultante.

5.5.1 **LayoutToRoomTemplateBuilder**

Al termine del processo evolutivo, il modulo `LayoutToRoomTemplateBuilder` è usato per convertire il fenotipo che si dimostra più idoneo tra tutte le generazioni precedenti in una stanza fisica. Per fare questo la maschera del fenotipo selezionato viene convertita direttamente nella geometria della stanza, trasformando ogni cella solida della `mask` in una cella solida del `TileMapLayer`.

Capitolo 6

Sperimentazione

6.1 Obiettivi della sperimentazione

La presente sperimentazione si propone di rispondere empiricamente alla domanda formulata a monte di questa tesi 1.1:

In che misura la generazione procedurale dei livelli incide sull'esperienza utente rispetto ai livelli progettati manualmente in un videogioco *action-platform 2D*, e quali differenze significative si riscontrano tra i due approcci?

Si adotta un approccio quantitativo basato su questionari standardizzati per misurare e confrontare l'esperienza di gioco nelle due versioni del progetto:

- **Versione base:** implementata per l'esame di Sviluppo di videogiochi, che presenta livelli realizzati manualmente;
- **Versione procedurale:** implementa algoritmi procedurali al fine di generare livelli esplorabili e controllabili dal designer, descritti nella sezione 5.

Le due versioni del gioco saranno testate da un campione di utenti selezionati. Ogni utente parteciperà a entrambe le versioni e, al termine di ciascuna, compilerà un questionario standardizzato. I membri dei gruppi di test non saranno a conoscenza della versione del gioco a cui stanno partecipando. L'analisi comparativa dei dati raccolti dall'esperienza utente consentirà di valutare le differenze tra le due versioni.

I risultati ottenuti da questa valutazione hanno come obiettivo primario quello di fornire raccomandazioni e spunti di miglioramento nell'applicazione della Procedural Content Generation (*PCG*) su categorie di giochi appartenenti al genere *action-platform 2D*.

6.2 Metodologia sperimentale

Al fine di effettuare una valutazione sistematica dell'influenza della generazione procedurale sull'esperienza di gioco, è stato implementato un approccio sperimentale fondato sull'utilizzo di questionari standardizzati. La presente sezione fornisce una descrizione dettagliata del design sperimentale, del campione di partecipanti e del protocollo operativo adottato.

6.2.1 Design sperimentale

Lo studio è stato progettato per garantire che ogni partecipante del campione sperimentasse entrambe le versioni del gioco, valutando l'esperienza percepita attraverso la compilazione di un questionario specifico per ciascuna versione. Le due versioni del gioco sono state presentate al campione di utenti in sequenze diverse, senza alcuna indicazione relativa alla natura dei livelli. I dati raccolti saranno utilizzati per valutare le differenze tra le due versioni. In particolare, si intende:

- valutare l'impatto della generazione procedurale sull'esperienza videoludica del campione, verificandone il coinvolgimento, l'immersione, la sfida, le emozioni positive e negative;
- identificare differenze significative post-game in termini di soddisfazione, desiderio di rigiocabilità ed esperienza complessiva tra i livelli generati e quelli realizzati manualmente, al fine di individuare spunti di miglioramento nella generazione procedurale;
- valutare se le tecniche algoritmiche di generazione abbiano migliorato l'esperienza di gioco.

6.2.2 Partecipanti

Il campione è costituito da venti partecipanti con differenti livelli di esperienza videoludica, età e genere. I criteri di inclusione prevedevano che i partecipanti:

- possedessero almeno una minima esperienza pregressa con i videogiochi;
- non avessero partecipato allo sviluppo del progetto;
- non avessero testato in precedenza alcuna versione di *A Bug Fix*.

6.2.3 Protocollo sperimentale

Ogni sessione di test è stata condotta secondo un protocollo rigoroso al fine di garantire la coerenza nella raccolta dei dati. Il protocollo prevedeva le seguenti fasi:

1. **Introduzione:** i partecipanti sono stati informati in modo esaustivo sulla natura generale dello studio e hanno fornito il loro consenso alla partecipazione;
2. **Tutorial:** è stato fornito un breve tutorial che illustrava i controlli e le meccaniche di base del gioco;
3. **Prima Sessione di Gioco:** i partecipanti hanno preso parte a una sessione di gioco con una delle due versioni disponibili, con una durata compresa tra 15 e 30 minuti;
4. **Prima Somministrazione GEQ:** al termine della prima sessione di gioco, è stata effettuata la somministrazione del questionario sulla prima versione provata;
5. **Seconda Sessione di Gioco:** i partecipanti hanno preso parte a una seconda sessione di gioco con la versione rimanente da provare, anch'essa con una durata compresa tra 15 e 30 minuti;
6. **Seconda Somministrazione GEQ:** al termine della seconda sessione di gioco, è stata effettuata la somministrazione del questionario circa la seconda versione.

La durata complessiva di ogni sessione sperimentale è stata di circa *30-60* minuti, inclusa la compilazione dei questionari.

6.3 Strumenti di valutazione

Al fine di quantificare l'esperienza di gioco in maniera standardizzata e comparabile, è stato impiegato il *Game Experience Questionnaire (GEQ)*, uno strumento ampiamente riconosciuto e utilizzato nella ricerca scientifica relativa all'esperienza videoludica [9].

6.3.1 Game Experience Questionnaire

Il Game Experience Questionnaire (GEQ) si configura come uno degli strumenti più completi e validati per la misurazione dell'esperienza di gioco, ampiamente utilizzato in numerosi studi nel campo dell'interazione uomo-macchina e del game design. Nella sua configurazione completa, il GEQ si articola in quattro moduli complementari, ciascuno progettato per valutare specifici aspetti dell'esperienza videoludica.

1. **Core Module:** Valuta l'esperienza di gioco generale attraverso *33* item.
2. **Social Presence Module:** Valuta l'esperienza di gioco con altri giocatori (sia in cooperazione che in competizione), attraverso *17* item. Questo modulo è rilevante esclusivamente per esperienze di gioco multigiocatore.
3. **Post-game Module:** Composto da *17* item, valuta le sensazioni del giocatore al termine della sessione di gioco.
4. **In-game Module:** Versione concisa del *Core Module*, con soli *14* item, progettata per essere somministrata durante la sessione di gioco senza interrompere significativamente il flusso di gioco.

Per la presente sperimentazione, sono stati utilizzati due dei quattro moduli disponibili:

- **Core Module:** per valutare l'esperienza complessiva di gioco.

- **Post-game Module:** per misurare le sensazioni successive alla sessione di gioco.

Il *Social Presence Module* non è stato utilizzato in quanto il gioco in esame è un'esperienza *single-player*. L'*In-game Module* non è stato somministrato per evitare interruzioni durante le sessioni di gioco, privilegiando una valutazione completa al termine di ciascuna sessione.

Core Module: dimensioni Il *Core Module* del *GEQ* è stato progettato per misurare sette dimensioni distinte dell'esperienza di gioco. Di seguito sono riportate le dimensioni con i rispettivi *item*:

- **Competenza** (items 2, 10, 15, 17, 21): valuta la sensazione di abilità, maestria e successo sperimentata durante il gioco;
- **Immersione** (items 3, 12, 18, 19, 27, 30): valuta il coinvolgimento narrativo, l'interesse nella storia e l'assorbimento nell'universo di gioco;
- **Flusso** (items 5, 13, 25, 28, 31): misura lo stato di concentrazione ottimale in cui il giocatore è completamente immerso nell'attività, con una distorta percezione del tempo;
- **Tensione/Frustrazione** (items 22, 24, 29): valuta sentimenti negativi quali irritazione, frustrazione e nervosismo;
- **Sfida** (items 11, 23, 26, 32, 33): misura la pressione, la sfida percepita e lo sforzo richiesto dal gioco;
- **Emozioni Negative** (items 7, 8, 9, 16): misurano la noia, la mancanza di interesse o il disagio;
- **Emozioni Positive** (items 1, 4, 6, 14, 20): valuta emozioni positive quali felicità, piacere e soddisfazione.

Numero item	voce
1	Mi sentivo interessato/a
2	Mi sentivo abile
3	Ero interessato/a alla storia del gioco
4	Ho pensato che fosse divertente
5	Ero completamente preso/a dal gioco
6	Mi sentivo felice
7	Mi ha fatto sentire male
8	Ho pensato ad altre cose
9	L'ho trovato stancante
10	Mi sono sentito/a competente
11	Ho pensato che fosse difficile
12	Era esteticamente piacevole
13	Ho dimenticato tutto quello che mi stava intorno
14	Mi sono sentito/a bene
15	Ero bravo/a al gioco
16	Mi sono sentito/a annoiato/a
17	Mi sono sentito/a di successo
18	Mi sono sentito/a immaginativo/a
19	Ho sentito di poter esplorare cose
20	Ho apprezzato il gioco
21	Ero veloce a raggiungere gli obiettivi del gioco
22	Mi sono sentito/a infastidito/a
23	Ho sentito la pressione
24	Mi sono sentito/a irritato/a
25	Ho perso la cognizione del tempo
26	Mi sono sentito/a sfidato/a
27	L'ho trovato impressionante
28	Ero profondamente concentrato/a nel gioco
29	Mi sono sentito/a frustrato/a
30	Mi sembrava un'esperienza ricca
31	Ho perso il contatto con il mondo esterno
32	Ho sentito di essere sotto pressione per il tempo
33	Ho sentito di dover metterci molto impegno

Tabella 6.1: Items del *Core Module*.

Post-game Module: dimensione Il *Post-game Module* è uno strumento progettato per valutare l'esperienza del giocatore immediatamente al termine della sessione di gioco. Questo modulo fornisce dati preziosi sul come l'esperienza di gioco influenzi lo stato psicologico del giocatore in seguito. Il modulo misura quattro dimensioni specifiche:

- **Esperienza Positiva** (items 1, 5, 7, 8, 12, 16): valuta sentimenti positivi quali soddisfazione, senso di potere e orgoglio;
- **Esperienza Negativa** (items 2, 4, 6, 11, 14, 15): misura emozioni negative come stanchezza, colpa o imbarazzo;
- **Ritorno alla Realtà** (items 3, 9, 17): valuta la sensazione di disorientamento e la difficoltà di transizione alla realtà dopo l'esperienza di gioco;
- **Stanchezza** (items 10, 13): misura la percezione che il tempo di gioco sia stato una perdita di tempo o l'idea che ci si sarebbe potuti dedicare ad attività più produttive.

L'analisi di queste dimensioni consente di comprendere l'impatto complessivo dell'esperienza di gioco sul benessere psicologico del giocatore, fornendo indicazioni utili per il miglioramento delle future sessioni di gioco.

Numero item	voce
1	Mi sono sentito/a rinvigorito/a
2	Mi sono sentito/a male
3	Ho trovato difficile tornare alla realtà
4	Mi sono sentito/a in colpa
5	È stato come una vittoria
6	L'ho trovato una perdita di tempo
7	Mi sono sentito/a energico/a
8	Mi sono sentito/a soddisfatto/a
9	Mi sono sentito/a disorientato/a
10	Mi sono sentito/a esausto/a
11	Ho sentito che avrei potuto fare cose più utili
12	Mi sono sentito/a potente
13	Mi sono sentito/a stanco/a
14	Ho provato rimorso
15	Mi sono sentito/a imbarazzato/a
16	Mi sono sentito/a orgoglioso/a
17	Ho avuto la sensazione di essere tornato/a da un viaggio

Tabella 6.2: Items del *Post-game Module*.

6.3.2 Scala di misura

Ogni item del *GEQ* viene valutato mediante una scala *Likert* a cinque punti, con i seguenti valori:

- 0: Per niente
- 1: Leggermente
- 2: Moderatamente
- 3: Abbastanza
- 4: Estremamente

L'adozione di una scala uniforme consente una comparabilità agevole dei risultati tra le diverse dimensioni e moduli del questionario.

6.4 Analisi dei dati

L'analisi dei dati raccolti mediante il Questionario sulla Gestione delle Emozioni si basa su un approccio metodico e rigoroso, incentrato sull'esame delle statistiche fondamentali.

Calcolo delle dimensioni: per ogni versione del gioco, sono stati determinati i punteggi medi delle diverse dimensioni del *GEQ*, in conformità con le indicazioni fornite dagli autori.

Presentazione dei risultati: i dati sono rappresentati graficamente, evidenziando: i valori medi di ciascuna dimensione per entrambe le versioni del *GEQ* e la differenza tra i valori medi delle due versioni.

6.5 Risultati della valutazione

6.5.1 Risultati del Core Module

Il Core Module fornisce dati dettagliati sull'esperienza di gioco di un utente durante una sessione. La figura 6.1 illustra un confronto tra i punteggi ottenuti dalle due versioni, mentre la figura 6.2 mostra la distanza relativa e il suo peso effettivo sull'intervallo di misura.

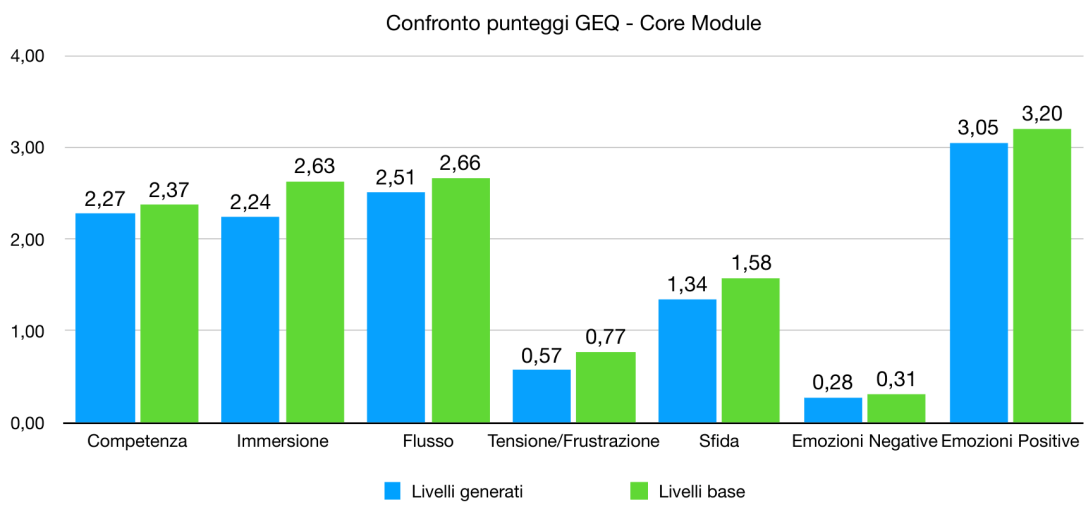


Figura 6.1: Confronto dei punteggi medi del *Core Module* tra livelli generati automaticamente e livelli creati manualmente.

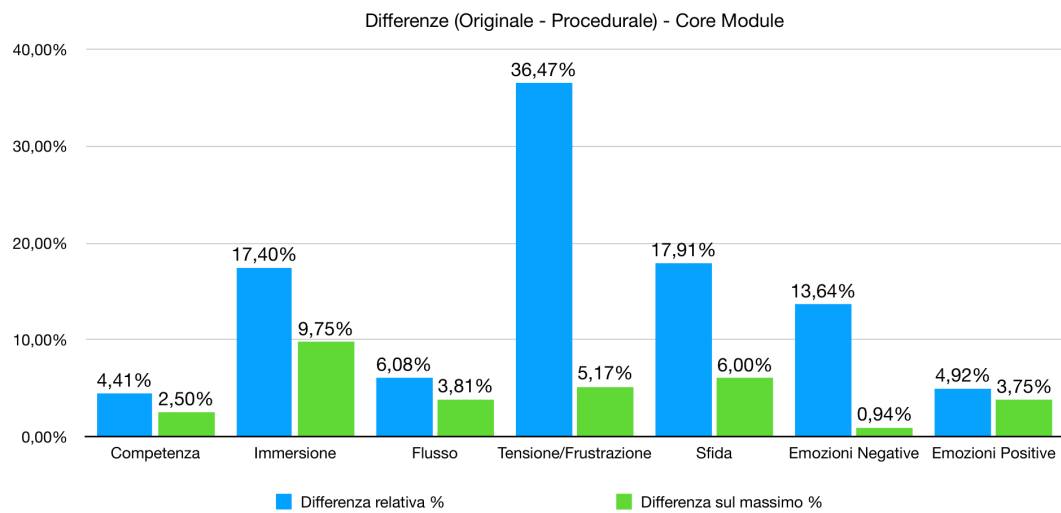


Figura 6.2: Le barre blu esprimono la differenza percentuale relativa tra i livelli manuali e procedurali, le barre verdi rappresentano invece la differenza rapportata al valore massimo della scala.

Un'analisi preliminare dei dati rileva che le variazioni in ciascuna dimensione tra le due versioni confrontate risultano nel complesso contenute, pur avvantaggiando i livelli definiti dal designer. In particolare:

- **Competenza:** in media i giocatori si sono sentiti competenti in entrambe le versioni, riscontrando una lieve differenza pari a 0,10 tra livelli generati 2,27 e livelli base 2,37 percependo una maggiore padronanza in questi ultimi.
- **Immersione:** i risultati evidenziano come i livelli progettati manualmente forniscano una maggiore immersione rispetto a quelli generati proceduralmente. Risulta doveroso sottolineare che la storia del prototipo è narrata nei livelli base ed è assente da quelli procedurali. Questa scelta potrebbe aver ridotto notevolmente i risultati ottenuti nella versione generata. In ogni caso i livelli base hanno totalizzato un punteggio medio di 2,63 rispetto al 2,24 dei livelli generati, con una differenza di 0,39, pari a circa il 17,4% in termini relativi ma che corrisponde a meno del 10% dell'intera scala, indicando uno scostamento che nel complesso è moderato.
- **Flusso:** la concentrazione e la continuità dei livelli si presentano soddisfacenti in entrambe le versioni, con un lieve vantaggio per i livelli progettati manualmente (2,66 rispetto a quelli generati 2,51). La differenza è pari a 0,15 (circa il 6% in termini relativi) e suggerisce che entrambe le versioni supportano adeguatamente il flusso di gioco.
- **Tensione/Frustrazione:** entrambe le versioni hanno totalizzato punteggi relativamente bassi, ciò indica che il gioco risulta poco frustrante ma anche poco stimolante. I livelli generati forniscono un punteggio medio di 0,57 mentre i livelli base 0,77. Sebbene la differenza relativa risulti significativamente elevata occorre tenere in considerazione che tale valore è condizionato da un basso punteggio assoluto. La distanza tra le due versioni è di 0,20 che rappresenta approssimativamente il 5% nella scala di misura. Possiamo concludere che lo scostamento è complessivamente contenuto.
- **Sfida:** i livelli generati manualmente presentano una maggiore difficoltà, con un punteggio medio di 1,58, rispetto alla versione procedurale, che ha ottenuto un punteggio medio di 1,34. La differenza tra i due punteggi è pari a 0,24

(circa il 17,9% in termini relativi); la distanza rimane comunque contenuta sulla scala complessiva.

- **Emozioni Negative:** i punteggi bassi di questa dimensione in entrambe le versioni suggeriscono che l'esperienza di gioco è prevalentemente priva di emozioni negative. In particolare, i livelli generati hanno suscitato un livello inferiore di tali emozioni, con un punteggio medio di 0,28, rispetto ai livelli progettati dal designer, che hanno ottenuto un punteggio medio di 0,31. I livelli generati hanno quindi registrato un lieve incremento di 0,03.
- **Emozioni Positive:** Entrambe le versioni hanno registrato punteggi elevati in relazione alle emozioni positive percepite durante la sessione di gioco. La versione procedurale ha ottenuto un punteggio medio di 3,05, mentre i livelli base hanno raggiunto un punteggio di 3,20. La differenza tra le due versioni risulta marginale, con uno scarto di 0,15.

6.5.2 Risultati del Post Game Module

Complessivamente anche nel modulo di Post Game le differenze tra la versione procedurale e la versione progettata manualmente risultano limitate. I livelli manuali mostrano comunque valori leggermente superiori.

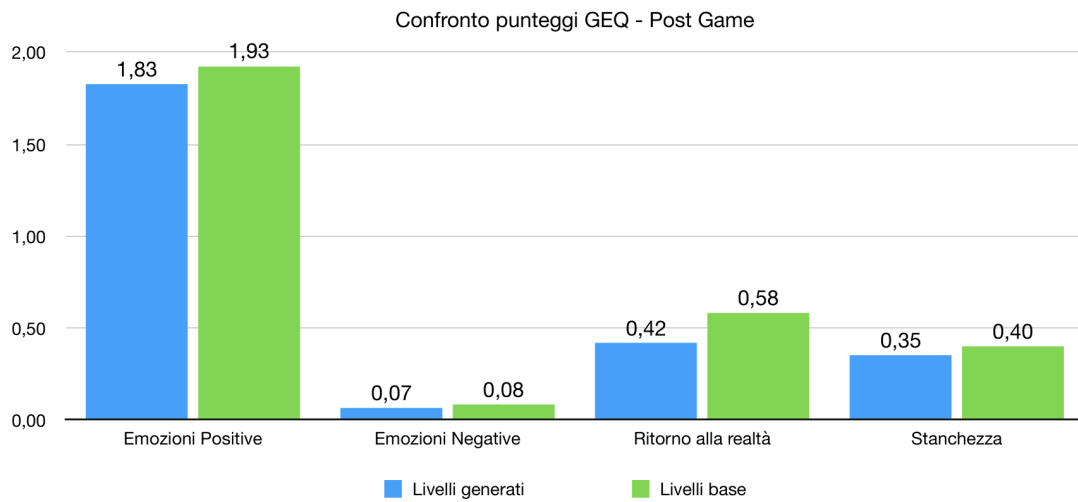


Figura 6.3: Confronto dei punteggi medi del *Post Game Module* tra livelli generati automaticamente e livelli creati manualmente.

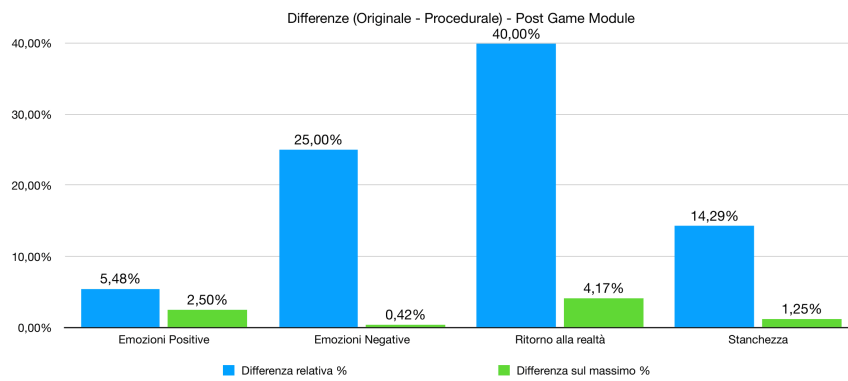


Figura 6.4: Le barre blu esprimono la differenza percentuale relativa tra i livelli manuali e procedurali, le barre verdi rappresentano invece la differenza rapportata al valore massimo della scala.

- **Emozioni Positive:** la distanza tra le due versioni è pari a 0,10 (1,93 per i livelli manuali e 1,83 per quelli procedurali) ed entrambe suscitano emozioni positive. In termini percentuali, lo scostamento è pari a circa il 5,48% in forma relativa e al 2,50% rispetto al valore massimo della scala. Ciò permette di affermare che le due versioni sono comparabili su questa dimensione.
- **Emozioni Negative:** entrambe le versioni hanno registrato punteggi estremamente bassi, indicando che l'esperienza complessiva di gioco non genera stati emotivi negativi di rilievo, con una differenza di soli 0,01 punti tra le due versioni. I livelli procedurali hanno totalizzato un punteggio pari a 0,07 e quelli manuali 0,08. Sebbene la differenza relativa è circa il 25% tra le versioni, tale valore è influenzato dai bassi punteggi assoluti e corrisponde ad appena lo 0,42% dell'intera scala, indicando uno scostamento privo di rilevanza pratica.
- **Ritorno alla realtà:** l'analisi dei dati raccolti evidenzia un livello di coinvolgimento relativamente superiore da parte del campione di utenti nei livelli progettati manualmente, i quali hanno conseguito un punteggio medio di 0,58, rispetto ai livelli procedurali, che hanno ottenuto un punteggio medio di 0,42. La differenza, pari a 0,16, corrisponde a circa il 40% in termini relativi, ma a soli 4,17% della scala complessiva. Questo suggerisce che, nonostante questa dimensione risulti essere quella con il divario più elevato, l'impatto reale rimane contenuto.
- **Stanchezza:** i punteggi relativi alla stanchezza risultano bassi in entrambe le versioni, suggerendo che l'esperienza non venga percepita come particolarmente affaticante. I livelli base evidenziano un valore leggermente superiore (0,40 rispetto a 0,35), con una differenza di 0,05. Tale scostamento rappresenta approssimativamente il 14,29% in termini relativi, ma solo l'1,25% della scala complessiva, a conferma della marginalità della differenza.

6.5.3 Interpretazione dei risultati

I risultati ottenuti evidenziano come entrambe le versioni offrono un'esperienza utente positiva e coerente nelle diverse dimensioni considerate. Nelle metriche definite dal *Core Module*, che sono state progettate dal designer riportano punteggi medi leggermente superiori rispetto a quelli generati tramite l'applicazione delle tecniche procedurali; tuttavia, queste differenze risultano nel complesso contenute. Analogamente, il modulo di *Post Game* possiede un andamento simile con valori medi confrontabili e scostamenti limitati.

Immersività e sfida: le differenze più evidenti emergono in queste dimensioni, dove i livelli manuali mostrano un vantaggio più marcato in termini relativi. Tuttavia, considerando tali differenze sull'intero intervallo della scala di misura definita dai moduli, il loro impatto risulta dimensionato, indicando che i livelli procedurali si collocano comunque in prossimità di quelli definiti dal designer. Questo comportamento è confermato anche durante l'analisi delle differenze percentuali, che evidenzia come valori relativamente elevati siano spesso associati a scostamenti assoluti ridotti.

Dimensione emotiva: le dimensioni che descrivono il lato emotivo e la tensione/frustrazione del giocatore, mostrano valori particolarmente bassi o alti in modo coerente tra le due versioni proposte. Ciò suggerisce che l'esperienza risulti stabile e comparabile. Possiamo notare come l'assenza di emozioni negative e i livelli limitati di stanchezza incidono sulla qualità generale dell'esperienza utente, indipendentemente dalla modalità di creazione dei livelli.

I risultati complessivi indicano che la generazione procedurale dei livelli è in grado di offrire un'esperienza utente paragonabile a quella dei livelli progettati manualmente, sebbene non raggiunga pienamente il medesimo livello di coinvolgimento in alcune specifiche dimensioni. Le differenze rilevate, pur presenti, risultano contenute sia in termini assoluti sia in relazione alla scala di misurazione, confermando l'efficacia della *PCG* come approccio valido, con potenziali margini di miglioramento principalmente legati al controllo dell'immersione, del ritmo e della difficoltà.

Capitolo 7

Conclusioni

Il presente lavoro ha esaminato l'impatto che la generazione procedurale di livelli ha nei giochi di tipo *action-platform 2D*, focalizzando l'attenzione su un caso di studio specifico *A Bug Fix*. Un campione di utenti è stato selezionato per testare la versione procedurale e quella base, in seguito è stato somministrato il modulo *Game Experience Questionnaire* che ha permesso di raccogliere dati circa l'esperienza utente delle due versioni. Tali dati sono stati sottoposti ad analisi per constatare se i livelli generati proceduralmente risultano comparabili qualitativamente a quelli definiti da un designer.

I risultati ottenuti evidenziano come entrambe le versioni offrano un'esperienza complessivamente positiva e bilanciata con differenze generalmente contenute tra i due approcci. In particolare, i livelli progettati manualmente mostrano valori medi leggermente superiori nella maggior parte delle dimensioni analizzate, soprattutto in termini di immersione e sfida. Tuttavia, l'analisi delle differenze percentuali rispetto alla scala complessiva mette in risalto come questi scostamenti sono in realtà limitati, evidenziando come i livelli generati siano in grado di avvicinarsi significativamente alla qualità dell'esperienza offerta dal *level design* tradizionale.

Nel complesso, questo studio conferma che la *PCG* rappresenta un approccio valido per la creazione di contenuti videoludici, ma occorre apporre ottimizzazioni affinché tale tecnica possa raggiungere livelli di qualità equivalenti a quelli realizzati da un designer umano.

7.1 Considerazioni

7.1.1 Punti di Forza

L'analisi complessiva dei risultati mette in evidenza la capacità della generazione procedurale di offrire un'esperienza utente solida e coerente con quella dei livelli progettati manualmente. Entrambe le versioni hanno prodotto valori elevati nella dimensione delle emozioni positive, sia durante la fase di gioco sia nel post-game, indicando un'esperienza complessivamente piacevole e soddisfacente. Questo viene valorizzato anche dai punteggi significativamente bassi totalizzati nella dimensione delle emozioni negative e alla stanchezza, suggerendo che il gioco non genera frustrazione o affaticamento rilevanti, mantenendo, dunque, un buon equilibrio generale. Infine, le dimensioni, quali competenza e flusso, presentano valori comparabili tra le due versioni, dimostrando che i livelli procedurali riescono a garantire accessibilità, continuità e comprensibilità dell'esperienza in misura analoga a quella del level design manuale.

7.1.2 Punti di Debolezza

Pur evidenziando risultati positivi, il lavoro non è esente da criticità che mettono in luce i limiti dell'approccio procedurale. La differenza più significativa concerne la dimensione dell'immersione: i livelli manuali ottengono valori sensibilmente superiori, attribuibili alla presenza di elementi narrativi e a una maggiore coerenza progettuale. Anche il livello di sfida risulta leggermente più elevato nei livelli progettati manualmente, così come la tensione percepita che aumenta il coinvolgimento del giocatore, suggerendo che il sistema procedurale incontra difficoltà nel bilanciare efficacemente la difficoltà generale dei livelli. Inoltre, nel post-game, i livelli base hanno ottenuto un punteggio maggiore nella dimensione: "ritorno alla realtà", questo è un indice di coinvolgimento leggermente superiore durante l'esperienza. Sebbene molte differenze risultino contenute in termini assoluti, tali aspetti indicano che la generazione procedurale, allo stato attuale, non riesce ancora a replicare pienamente la qualità del design intenzionale umano nelle dimensioni maggiormente correlate al coinvolgimento profondo.

7.1.3 Sviluppi Futuri

Alla luce dei risultati ottenuti, diversi accorgimenti possono contribuire a migliorare l'efficacia della generazione procedurale:

- **Miglioramento dell'immersione:** l'integrazione di elementi narrativi e strutture più coerenti all'interno dei livelli generati potrebbe incrementare significativamente il livello di immersione;
- **Bilanciamento della difficoltà:** l'adozione di tecniche più avanzate di *PCG*, come sistemi basati su vincoli o adattivi, potrebbe migliorare il bilanciamento della difficoltà e della tensione percepita dal giocatore migliorando nel contempo anche il ritmo di gioco e rendendo l'esperienza più stimolante;
- **Espansione dei contenuti:** l'espansione dei contenuti di gioco (quali nemici, abilità, oggetti, storia e molto altro) potrebbe aumentare il livello di coinvolgimento e immersione offrendo inoltre un'esperienza utente più interessante e accattivante.

Bibliografia

- [1] Alberto Alvarez, Steve Dahlskog, Jose Font, and Julian Togelius. Empowering quality diversity in dungeon design with interactive constrained map-elites. In *2019 IEEE Conference on Games (CoG)*, pages 1–8. IEEE, 2019.
- [2] Noam Chomsky. Three models for the description of language. *IRE Transactions on information theory*, 2(3):113–124, 1956.
- [3] Mark de Berg, Otfried Cheong, Marc van Kreveld, and Mark Overmars. *Voronoi Diagrams*, pages 147–171. Springer Berlin Heidelberg, Berlin, Heidelberg, 2008.
- [4] Joris Dormans. Adventures in level design: generating missions and spaces for action adventure games. In *Proceedings of the 2010 workshop on procedural content generation in games*, pages 1–8, 2010.
- [5] Alison Gazzard. The platform and the player: exploring the (hi) stories of elite. *Game Studies*, 13(2), 2013.
- [6] Daniele Gravina, Ahmed Khalifa, Antonios Liapis, Julian Togelius, and Georgios N Yannakakis. Procedural content generation through quality diversity. In *2019 IEEE Conference on Games (CoG)*, pages 1–8. IEEE, 2019.
- [7] Jake Gutierrez and Jacob Schrum. Generative adversarial network rooms in generative graph grammar dungeons for the legend of zelda. In *2020 IEEE Congress on Evolutionary Computation (CEC)*, pages 1–8, 2020.
- [8] Mark Hendrikx, Sebastiaan Meijer, Joeri Van Der Velden, and Alexandru Iosup. Procedural content generation for games: A survey. *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)*, 9(1):1–22, 2013.
- [9] Wijnand A IJsselsteijn, Yvonne AW De Kort, and Karolien Poels. The game experience questionnaire. 2013.

- [10] Roland Linden, Ricardo Lopes, and Rafael Bidarra. Designing procedurally generated levels. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 9, pages 41–47, 2013.
- [11] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- [12] Aitor Santamaría-Ibirika, Xabier Cantero, Sergio Huerta, Igor Santos, and Pablo G. Bringas. Procedural playable cave systems based on voronoi diagram and delaunay triangulation. In *2014 International Conference on Cyberworlds*, pages 15–22, 2014.
- [13] Julian Togelius and Noor Shaker. The search-based approach. In *Procedural Content Generation in Games*, pages 17–30. Springer, 2016.
- [14] Julian Togelius, Noor Shaker, and Mark J. Nelson. *Introduction*, pages 1–15. Springer International Publishing, Cham, 2016.
- [15] Breno MF Viana and Selan R dos Santos. Procedural dungeon generation: A survey. *Journal on Interactive Systems*, 12(1):83–101, 2021.
- [16] Georgios N Yannakakis, Antonios Liapis, and Constantine Alexopoulos. Mixed-initiative co-creativity. 2014.

Ringraziamenti

Ringrazio il Prof. Pierpaolo Basile per il sostegno e la guida fornita negli ultimi mesi per la realizzazione del presente progetto.

Desidero ringraziare la mia famiglia per il loro sostegno e la presenza durante le sfide incontrate lungo questo percorso.

Ringrazio la mia ragazza Sabrina per essere stata l'ispirazione che ha riacceso in me la motivazione per raggiungere questo traguardo. Ti ringrazio enormemente per aver ascoltato tutti i miei sfoghi e le mie lamentele (che sono sempre molte) e per aver saputo bilanciare i momenti in cui mi hai motivato e guidato a quelli in cui mi hai sostenuto e compreso.

Infine, voglio ringraziare tutti gli amici con cui ho condiviso questo percorso, che mi hanno fatto vivere momenti di spensieratezza tra un esame e l'altro.

Grazie di cuore a tutti voi.